

CAIF Engineering Reference

Version 3.0

Contributors: Charles Tang, Jason Liu, Zhihan Lin

Table of Contents

Part I — Foundation	2
Chapter 1. Project Mission	2
Chapter 2. Design Principles	3
Chapter 3. Overall Architecture	9
Chapter 4. Agentic Development Philosophy	13
Chapter 5. CAIF Shared Core	22
Chapter 6. Capability Development	31
Part II — Engineering	44
Chapter 7. Repository & Engineering Processes	44
Part III — Collaboration	54
Chapter 8. Cognitive Interoperability	54
Chapter 9. Collective Cognition	61
Chapter 10. Collaborative Reasoning & Negotiation	70
Chapter 11. Internet of Cognition	79
Chapter 12. Future Evolution of CAIF	88
Appendix A. Reference Architecture	96
Appendix B. Capability Specifications	108
Appendix C. Cognitive Collaboration Specifications	117

Appendix D. Engineering Contracts.....	130
Appendix E. Capability Maturity Model.....	140
Appendix F. Canonical Biblical AI.....	151
Appendix G. Internet of Cognition.....	163
Appendix H. Technology Mapping.....	181
Appendix I. Migration Strategy	192
Appendix J. Prototype Roadmap	201
Appendix K. Glossary.....	213
Appendix L. Design Decisions (Architecture Decision Records)	223

Part I — Foundation

Chapter 1. Project Mission

Why CAIF?

Churches have accumulated decades of valuable knowledge through sermons, ministries, administrative experience, fellowship activities, and faithful service. Unfortunately, much of this knowledge is fragmented across documents, emails, presentations, photographs, videos, and personal memories. As volunteers rotate and leaders change, institutional memory is often lost, requiring new workers to rediscover past experience.

The Church AI Foundation (CAIF) exists to preserve, govern, and share church knowledge through trustworthy artificial intelligence.

Rather than building a single application, CAIF provides a shared engineering foundation upon which multiple AI applications can be developed collaboratively.

CAIF intentionally separates engineering infrastructure from domain knowledge, allowing independent domain-specific AI systems to share a common engineering foundation while maintaining their own knowledge, governance, and evaluation frameworks.

Initial applications include:

- CKAIS (Central Kitchen AI System)
 - Biblical AI
 - Sermon Intelligence
 - Media Memory
 - Mentoring
 - Future ministry applications
-

Mission Statement

Build trustworthy AI capabilities that preserve church knowledge and empower ministry through open collaboration.

Chapter 2. Design Principles

Purpose

Technology changes rapidly.

Software frameworks come and go.

Programming languages evolve.

Agent runtimes will continue to change.

However, the fundamental principles guiding the Church AI Foundation should remain stable. These principles define the philosophy upon which CAIF is built and should continue to guide future contributors regardless of technological evolution.

For how these design principles are applied to CAIF, see Appendix L.

Principle 1. Mission Before Technology

Technology is a servant of ministry, not its master.

Every technical decision should ultimately support the mission of the Church.

The success of CAIF is not measured by the sophistication of its AI models, the popularity of its programming framework, or the number of GitHub stars. Rather, success is measured by whether it helps churches preserve knowledge, disciple believers, support volunteers, and serve future generations more faithfully.

Whenever technological convenience conflicts with ministry needs, ministry takes priority.

Implications

- Choose simpler solutions when they better serve ministry.
 - Avoid unnecessary technical complexity.
 - Evaluate every feature by ministry value rather than engineering elegance.
 - Never allow technology to redefine the Church's mission.
-

Principle 2. Specification Before Implementation

Good specifications produce good software.

In the era of Agentic AI, software is increasingly generated by coding agents rather than written entirely by humans.

Consequently, developers spend less time writing syntax and more time defining specifications, interfaces, policies, tests, and success criteria.

CAIF therefore treats specifications as first-class engineering assets.

Specifications include:

- Vision documents
- Requirements
- Capability definitions

- Agent Skill descriptions
- Governance policies
- Test cases
- Evaluation criteria

Implementation may evolve many times.

Specifications should remain comparatively stable.

Implications

- Write the specification before writing code.
 - Coding agents should generate implementations from approved specifications.
 - Every implementation should be traceable to a specification.
-

Principle 3. Capabilities Before Applications

Applications are temporary. Capabilities are reusable.

Rather than building isolated applications, CAIF builds reusable capabilities.

For example:

Instead of building a "Church History Search System,"

CAIF develops reusable capabilities such as:

- Document Retrieval
- Citation Generation
- Bible Reference Parsing
- Access Control
- Media Search
- Audit Logging

Multiple applications may reuse the same capability.

This approach reduces duplicated effort while improving consistency across projects.

Implications

- Applications should compose capabilities.
 - Capabilities should remain independent whenever practical.
 - Shared capabilities belong to CAIF Core rather than individual applications.
-

Principle 4. Evidence Before Generation

Trustworthy AI begins with trustworthy evidence.

Large language models are excellent at generating fluent responses but may produce inaccurate or fabricated information.

Whenever possible, CAIF should retrieve reliable evidence before generating responses.

Evidence may include:

- Church documents
- Meeting minutes
- Sermons
- Bible passages
- Approved policies
- Historical records
- Verified metadata

Generated responses should distinguish:

- Retrieved facts
- AI interpretation
- Human opinion

Implications

- Retrieval precedes generation.
- Citations should be preserved whenever possible.

- Confidence and uncertainty should be communicated honestly.
-

Principle 5. Governance Before Deployment

A correct answer is not necessarily a trustworthy answer.

Passing automated tests does not guarantee that software is ready for production.

Governance includes:

- Specification review
- Security review
- Privacy protection
- Auditability
- Human approval
- Release management
- Theological review where appropriate

Governance becomes increasingly important as coding agents assume greater responsibility for implementation.

Implications

- Every release requires governance.
 - High-risk capabilities require human approval.
 - AI-generated artifacts should remain auditable.
-

Principle 6. Replace Technologies, Preserve Knowledge

Technology evolves. Knowledge should endure.

The AI ecosystem changes rapidly.

Today's technologies—including Google ADK, Agent Skills, Claude Code, DeepSeek, and future frameworks—will eventually be replaced.

CAIF therefore avoids coupling its architecture to any single technology.

Instead, it preserves:

- Church knowledge
- Specifications
- Governance
- Capabilities
- Institutional memory

These assets should survive multiple generations of implementation technology.

Implications

- Separate architecture from implementation.
 - Minimize framework-specific dependencies.
 - Design migration rather than replacement.
-

Principle 7. Human Accountability

AI assists. Humans remain responsible.

Artificial intelligence should support human decision making rather than replace it.

Pastors remain responsible for pastoral care.

Elders remain responsible for church governance.

Developers remain responsible for software quality.

Contributors remain responsible for the artifacts they submit.

AI should assist each of these roles rather than assume them.

Implications

- Human review remains essential.
- AI recommendations should be explainable.
- Final responsibility belongs to people, not models.

Chapter 3. Overall Architecture

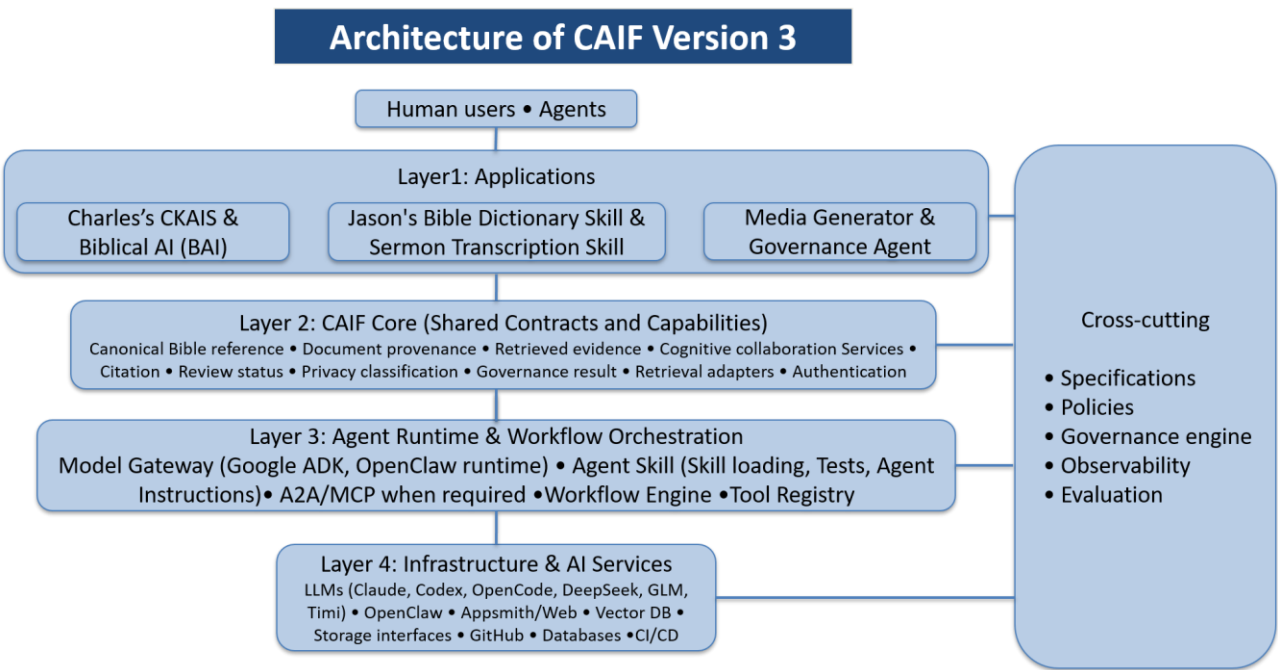
I also think Chapter 3 should be expanded to about **4 pages**. One thing I learned from Jason's emails is that **we must distinguish the architectural layers from today's implementation technologies**.

The architecture should explain **what each layer is responsible for**, not which framework implements it.

Architectural Overview

CAIF adopts a layered architecture that separates ministry applications from reusable capabilities, shared infrastructure, and execution technologies.

Each layer has a well-defined responsibility, allowing individual technologies to evolve without changing the overall architecture.



Please refer to **Appendix A “Reference Architecture”** in response to the following question:

"If another church, seminary, or open-source community wants to build a CAIF-compatible system, what architecture should they follow?"

Layer 1 — Ministry Applications

This layer contains applications that directly serve ministry needs.

Examples include:

- CKAIS
- Biblical AI
- Sermon Intelligence
- Mentoring Platform
- Church Administration Assistant

Applications are free to innovate according to their specific ministry requirements.

However, applications should avoid implementing functionality already provided by CAIF Core.

Responsibilities

- User experience
- Ministry workflows
- Application-specific business logic
- Integration of multiple capabilities

Examples

CKAIS combines:

- Church History Retrieval
- Media Memory
- Meeting Archive
- Volunteer Workflow

Biblical AI combines:

- Bible Retrieval
- Hermeneutics

- Citation
 - Theology Governance
-

Layer 2 — Shared Capabilities (CAIF Core)

This is the heart of the Foundation.

Rather than providing complete applications, CAIF Core provides reusable capabilities that multiple applications can share.

Examples include:

- Retrieval
- Citation
- Governance
- Evaluation
- Contracts
- Cognitive Collaboration Services
- Authentication
- Access Control
- Metadata Management
- Search
- Audit Logging

This layer represents the primary shared engineering effort among contributors.

Responsibilities

- Stable semantics
- Reusable implementations
- Standard interfaces
- Shared engineering effort

Layer 3 — Agent Runtime & Orchestration

This layer determines **how capabilities are discovered and executed**, not what they do.

Current examples may include:

- Google ADK
- Agent Skills
- A2A
- Dynamic Workflow (future)
- Other agent runtimes

Because AI technology evolves rapidly, this layer is intentionally replaceable.

Responsibilities

- Tool discovery
- Agent execution
- Workflow coordination
- Session management
- Multi-agent collaboration

Layer 4 — Infrastructure & AI Services

This layer contains the underlying technologies that support the system.

Examples include:

- Large Language Models
- Vector databases
- Document repositories
- Relational databases
- Object storage

- GitHub
- CI/CD
- Authentication systems

These technologies may evolve independently of the upper architectural layers.

Responsibilities

- Data storage
 - AI inference
 - Deployment
 - Security
 - Scalability
 - Reliability
-

Chapter 4. Agentic Development Philosophy

4.1 Why Software Development Is Changing

Artificial Intelligence is transforming software engineering in much the same way that high-level programming languages once transformed assembly programming.

In traditional software development, engineers spent most of their effort writing implementation code. Modern AI coding assistants can now generate a significant portion of that code automatically. Consequently, the value of human developers is shifting from writing syntax to defining architecture, specifications, policies, and system behavior.

CAIF adopts this new engineering philosophy.

The goal of contributors is no longer simply to produce software.

The goal is to design trustworthy capabilities that AI can help implement, test, maintain, and evolve.

4.2 Traditional Software Development

Traditional software engineering typically follows the lifecycle below.

Requirements → Architecture → Coding → Testing → Debugging → Deployment

Human developers spend much of their effort writing source code and manually debugging implementation details.

The resulting software quality depends largely upon the coding skills of individual developers.

4.3 Agentic Software Development

Agentic software development shifts the primary engineering effort.

Instead of focusing on implementation syntax, developers increasingly focus on describing the desired system behavior.

Vision → Specifications → Coding Agents → Generated Implementation → Automated Testing →
Governance Review → Deployment

Coding agents become implementation assistants.

Human developers become architects, reviewers, and governors.

This shift does **not** eliminate software engineering.

Instead, it raises the level of abstraction at which software engineers contribute.

4.4 The New Responsibilities of Developers

As coding becomes increasingly automated, developers assume new responsibilities.

System Architecture

Developers define the overall system structure, modularity, dependencies, and evolution strategy.

Questions include:

- What capabilities should exist?
- How should applications communicate?
- Which components belong in CAIF Core?

- How should future technologies be accommodated?
-

Specifications

Specifications become primary engineering artifacts.

Rather than telling a computer **how** to perform every operation, developers describe:

- desired behavior,
- required constraints,
- expected outputs,
- success criteria,
- quality requirements.

Coding agents use these specifications to generate implementations.

Knowledge Engineering

Church knowledge must be organized before AI can use it effectively.

Examples include:

- document classifications,
- discipleship tags,
- church terminology,
- metadata standards,
- citation policies,
- Bible reference conventions.

These knowledge structures become long-term organizational assets.

Governance

Developers also define policies governing AI behavior.

Examples include:

- security requirements,
- approval workflows,
- audit trails,
- privacy protection,
- theological review,
- release policies.

Governance ensures that automation remains trustworthy.

4.5 Engineering Artifacts

Traditional software projects primarily manage source code.

CAIF manages a broader collection of engineering artifacts.

Vision Documents

Describe why the project exists.

Example:

VISION.md

Architecture Documents

Describe the overall system design.

Example:

ARCHITECTURE.md

Specifications

Describe functional behavior and system requirements.

Examples:

SPECIFICATIONS/

```
church-history.md
```

```
media-memory.md
```

```
retrieval.md
```

```
governance.md
```

Capability Descriptions

Describe reusable capabilities.

Example:

```
capability.yaml
```

```
or
```

```
CAPABILITY.md
```

Each capability should define:

- purpose,
 - inputs,
 - outputs,
 - dependencies,
 - owner,
 - evaluation criteria.
-

Agent Instructions

Coding agents require guidance just as human developers require coding standards.

Typical examples include:

```
AGENTS.md
```

```
CLAUDE.md
```

```
GEMINI.md
```

These documents describe:

- project conventions,

- repository organization,
- coding style,
- testing expectations,
- approval requirements,
- preferred architectural patterns.

Unlike source code, these files are intended primarily for AI coding assistants.

Agent Skills

Agent Skills describe reusable abilities that runtime agents may invoke.

Example:

```
skills/  
bible-reference/  
SKILL.md  
church-history/  
SKILL.md  
citation/  
SKILL.md
```

Each Skill describes:

- purpose,
- when to use it,
- required inputs,
- expected outputs,
- available tools,
- examples,
- limitations.

Unlike AGENTS.md, which instructs **coding agents**, Skill descriptions enable **runtime agents** to

discover and use capabilities.

Source Code

Source code remains an important artifact.

However, it is no longer the only—or even the primary—representation of system knowledge.

Implementations may change repeatedly while specifications remain stable.

Tests

Automated testing becomes increasingly important as AI generates more code.

CAIF encourages:

- unit tests,
- integration tests,
- regression tests,
- evaluation datasets,
- governance validation.

Tests verify that generated implementations continue satisfying approved specifications.

4.6 Repository Documentation

The Git repository itself becomes an engineering platform.

A typical project may include:

```
README.md  
  
VISION.md  
  
ARCHITECTURE.md  
  
ROADMAP.md  
  
CONTRIBUTING.md
```

AGENTS.md
SPECIFICATIONS/
SKILLS/
PACKAGES/
TESTS/
EXAMPLES/
DOCS/

These documents are not merely documentation.

They are part of the engineering process itself.

Human contributors read them.

Coding agents also read them.

Consequently, repository documentation must be treated with the same care as production source code.

4.7 Continuous Evolution

AI technologies evolve rapidly.

New coding agents, development environments, and orchestration frameworks will continue to appear.

CAIF therefore distinguishes between:

Enduring Engineering Assets

- mission,
- specifications,
- knowledge,
- governance,
- capabilities,
- documentation.

and

Replaceable Implementation Technologies

- coding assistants,
- development environments,
- runtime frameworks,
- programming languages,
- deployment platforms.

The latter should be replaceable without changing the former.

4.8 The Role of Human Contributors

The purpose of Agentic Development is not to remove humans from software engineering.

Rather, it allows contributors to focus their effort where human judgment is most valuable.

Within CAIF:

- Architects design the system.
- Domain experts organize knowledge.
- Developers define specifications.
- Coding agents generate implementations.
- AI assists testing and maintenance.
- Human reviewers approve releases.
- Church leaders retain responsibility for ministry and doctrine.

AI becomes a collaborator.

Human contributors remain accountable.

Noticing the following is even more important than the .md files:

CAIF is not really shifting from "coding" to "specification."

It is shifting from **Software Engineering** to **Knowledge Engineering**.

Twenty years ago, software projects were primarily about algorithms and source code.

Today, successful AI projects are increasingly about:

- organizing knowledge,
- defining semantics,
- writing specifications,
- establishing governance,
- creating evaluation datasets,
- documenting capabilities.

The source code itself becomes only one artifact among many.

Chapter 5. CAIF Shared Core

5.1 Purpose

The Church AI Foundation is intended to support many independent AI applications developed by different contributors over many years.

These applications should not duplicate common functionality such as retrieval, governance, citations, document access, or AI model management.

Instead, reusable capabilities are collected into the **CAIF Shared Core**.

The Shared Core represents the permanent engineering foundation of CAIF.

Applications evolve.

Technologies evolve.

The Shared Core evolves much more slowly.

Its purpose is to preserve consistency, interoperability, and trust across all CAIF applications.

5.2 Architectural Philosophy

The Shared Core is **not** a software framework.

It is **not** a monolithic library.

It is **not** a runtime platform.

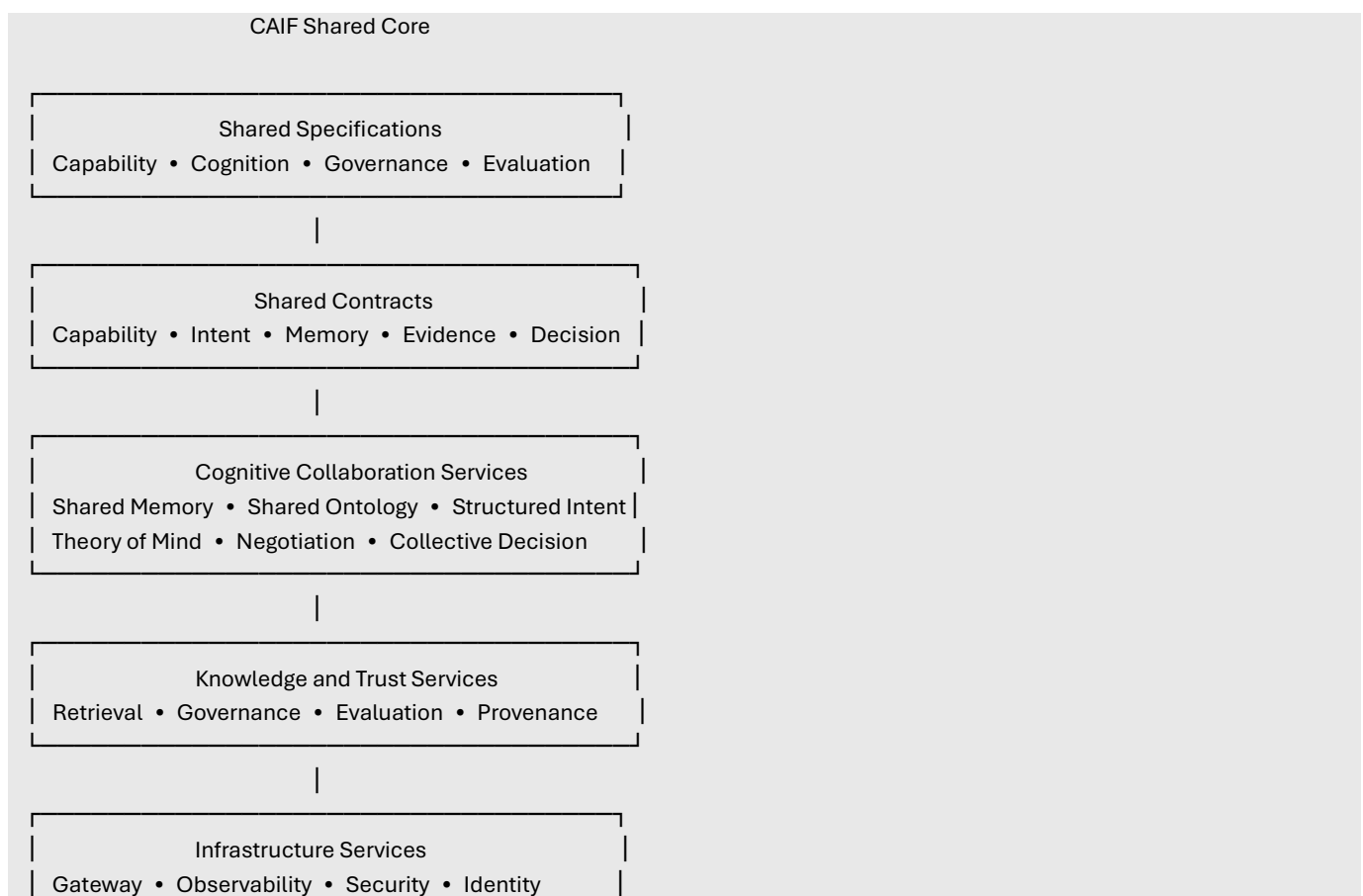
Instead, it is a collection of stable engineering capabilities shared by multiple applications.

The Shared Core follows four principles:

- reusable rather than application-specific;
 - capability-oriented rather than framework-oriented;
 - specification-driven rather than implementation-driven;
 - stable despite changing AI technologies.
-

5.3 Overall Structure

The Shared Core consists of several capability groups.



The layers shown in the CAIF Shared Core represent logical architectural responsibilities rather than a mandatory sequential execution pipeline. A capability invokes only the shared services required for its task. Cognitive Collaboration Services are used when an interaction requires shared semantic understanding, coordinated intent, negotiation, agent modeling, or collective judgment.

For example:

- A simple Bible lookup may use capability contracts, retrieval, and the model gateway without invoking collective cognition.
- A sermon interpretation involving several theological agents may use shared ontology, structured intent, negotiation, evidence evaluation, and collective decision making.
- A document-upload workflow may use retrieval, governance, security, and observability, but no Theory of Mind.

This diagram makes four distinctions clear:

1. **Specifications define expected behavior.**
2. **Contracts define exchanged structures and semantics.**
3. **Cognitive Collaboration Services perform cognitive coordination.**
4. **Infrastructure Services support execution but do not define cognition.**

The contracts and cognitive services may eventually become independent repositories or Python packages.

5.4 Shared Specifications

Specifications define **what** the system should accomplish.

Unlike source code, specifications remain relatively stable as implementations evolve.

Typical specifications include:

- capability descriptions;
- data semantics;
- governance policies;

- security requirements;
- privacy policies;
- evaluation criteria;
- ministry requirements.

Examples:

```
specifications/  
church-history.md  
retrieval.md  
media-memory.md  
biblical-ai.md  
governance.md
```

Specifications become the primary engineering artifacts in Agentic Development.

Coding agents consume specifications to generate implementations.

Human contributors review and improve specifications rather than repeatedly rewriting code.

5.5 Shared Contracts

Contracts define the common language used by all CAIF capabilities.

Without shared contracts, independently developed applications cannot reliably cooperate.

Unlike traditional API contracts, CAIF contracts primarily describe **semantics** rather than communication protocols.

Examples include:

```
RetrievedEvidence  
BibleReference  
CapabilityManifest  
InterpretationResult  
ReviewDecision  
AuditRecord
```

A contract answers questions such as:

- What is "evidence"?
- What constitutes a citation?
- What metadata accompanies a retrieved document?
- How is confidence represented?
- How is uncertainty expressed?
- How are human reviews recorded?

Stable contracts allow multiple applications to exchange trustworthy information without requiring identical implementations.

5.6 Retrieval Core

Retrieval is one of the most fundamental capabilities in CAIF.

Rather than allowing every application to implement retrieval independently, the Shared Core provides reusable retrieval services.

Responsibilities include:

- document retrieval;
- semantic search;
- metadata filtering;
- citation generation;
- source ranking;
- permission checking;
- evidence collection.

Applications remain free to define domain-specific retrieval logic while reusing the common retrieval infrastructure.

5.7 Governance Core

Governance ensures that AI systems remain trustworthy throughout their lifecycle.

Governance extends far beyond software testing.

Responsibilities include:

- specification review;
- approval workflow;
- audit logging;
- release policy;
- privacy enforcement;
- theological review;
- human approval;
- traceability.

Governance becomes increasingly important as coding agents assume greater responsibility for implementation.

5.8 Evaluation Core

AI systems should be evaluated continuously rather than only during development.

Evaluation measures whether capabilities continue satisfying their intended purpose.

Typical evaluations include:

- retrieval accuracy;
- citation correctness;
- hallucination detection;
- workflow consistency;
- capability regression;
- governance compliance;

- human review statistics.

Evaluation data also provides feedback for improving future implementations.

5.9 Model Gateway

The Shared Core separates applications from individual AI models.

Applications should never depend directly upon a particular model vendor.

Instead, requests pass through a model gateway.

Responsibilities include:

- model selection;
- model routing;
- provider abstraction;
- cost optimization;
- caching;
- authentication;
- fail-over.

Initially the gateway may support:

- DeepSeek
- GLM
- Gemini
- Claude
- OpenAI

Future providers can be added without changing application logic.

5.10 Observability

As AI systems become increasingly autonomous, visibility becomes essential.

Observability records how the system behaves rather than merely whether it succeeds.

Typical observations include:

- retrieval statistics;
- model usage;
- workflow execution;
- tool invocation;
- latency;
- failures;
- governance decisions;
- human approvals.

Observability enables contributors to understand and improve system behavior over time.

5.11 Shared Core Is Not a Framework

One common misconception is that the Shared Core defines how applications must be built.

This is not the intention.

Applications remain free to adopt:

- Google ADK;
- future agent runtimes;
- different user interfaces;
- different programming languages;
- different deployment environments.

As long as applications satisfy the Shared Core contracts and governance requirements, they remain interoperable.

The Shared Core therefore defines **shared semantics** rather than implementation technologies.

5.12 Relationship to Applications

Applications compose capabilities from the Shared Core.

For example:

```
CKAIS
|
├─ Retrieval
├─ Governance
├─ Citation
├─ Document Archive
└─ Media Memory
```

Biblical AI may reuse:

```
Biblical AI
|
├─ Retrieval
├─ Citation
├─ Governance
├─ Bible Reference
└─ Interpretation
```

Although these applications serve different ministry purposes, they benefit from a common engineering foundation.

5.13 Evolution Strategy

The Shared Core should evolve cautiously.

Changes affecting contracts or shared semantics may influence multiple applications simultaneously.

Therefore:

- shared capabilities should emphasize backward compatibility;
- specifications should evolve before implementations;
- contributors should review shared changes collaboratively;
- applications should remain insulated from rapidly changing AI technologies.

The Shared Core provides stability while allowing innovation at the application layer.

Proposed Repository Organization

```
CAIF/  
  
shared/  
  
    specifications/  
  
    contracts/  
  
    retrieval/  
  
    governance/  
  
    evaluation/  
  
    gateway/  
  
    observability/  
  
applications/  
  
    CKAIS/  
  
    BiblicalAI/  
  
    SermonAI/  
  
    Mentor/  
  
docs/  
  
examples/
```

Notice that **"shared/"** is not merely a Python package.

It is an architectural boundary.

Chapter 6. Capability Development

6.1 Purpose

The Church AI Foundation is organized around **capabilities**, rather than applications.

Applications are built by combining **reusable** capabilities.

This approach encourages collaboration, minimizes duplicated effort, and allows independently developed applications to share common engineering assets.

Every capability should be developed as an independent engineering component with clearly defined

responsibilities, interfaces, documentation, governance, and evaluation.

For detailed capability specifications, please refer to **Appendix B**.

6.2 Why Capability-Oriented Development?

Traditional software projects often begin by building complete applications.

For example,

Church Archive System

Biblical AI

Sermon Search

Mentoring Platform

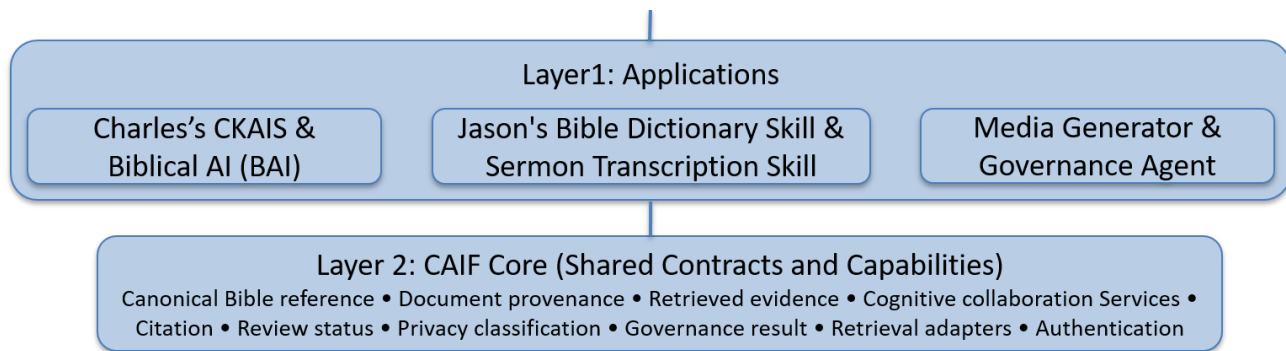
Each application may independently implement:

- document retrieval,
- authentication,
- citations,
- governance,
- search,
- metadata,
- audit logging.

This results in duplicated effort.

CAIF instead develops reusable capabilities.

Applications become compositions of capabilities.



6.3 Definition of a Capability

A capability is a reusable engineering service that performs one well-defined function and may be shared by multiple applications.

A capability should satisfy three principles:

Single Responsibility

Each capability solves one primary problem.

Examples:

- Bible Reference Parsing
- Church History Retrieval
- Citation Generation
- Media Search
- Audit Logging

rather than

"Church Knowledge System"

Reusability

A capability should be reusable by multiple applications.

If a component is useful only to one application, it probably belongs inside that application rather than CAIF Core.

Stable Semantics

The meaning of a capability should remain stable even if its implementation changes.

For example,

"Bible Reference Parsing"

should always produce the same logical result regardless of which LLM, runtime, or programming language performs the work.

6.4 Capability Lifecycle

Every capability follows the same engineering lifecycle.

```
graph TD; Idea --> Specification; Specification --> Design; Design --> Implementation; Implementation --> Testing; Testing --> Evaluation; Evaluation --> Governance_Review[Governance Review]; Governance_Review --> Publication; Publication --> Continuous_Improvement[Continuous Improvement];
```

Unlike traditional software engineering, implementation is **not** the final objective.

Continuous evaluation and improvement remain part of the lifecycle.

6.5 Capability Artifacts

Each capability consists of more than source code.

Typical engineering artifacts include:

- Capability/
- README.md

SPECIFICATION.md

SKILL.md

AGENTS.md

contracts/

implementation/

tests/

evaluation/

examples/

CHANGELOG.md

Each artifact serves a different purpose.

README.md

Introduces the capability.

Explains:

- purpose,
 - users,
 - dependencies,
 - quick examples.
-

SPECIFICATION.md

Defines:

- functional behavior,
- inputs,
- outputs,
- assumptions,

- constraints,
- success criteria.

This document represents the primary engineering specification.

SKILL.md

Provides runtime instructions for agent platforms.

Typical contents include:

- when to invoke the capability,
 - expected parameters,
 - available tools,
 - limitations,
 - examples.
-

AGENTS.md

Provides instructions for coding agents contributing to this capability.

Typical contents include:

- repository conventions,
- preferred architecture,
- coding standards,
- testing requirements,
- approval process.

Unlike SKILL.md, this document targets **development agents** rather than runtime agents.

Source Code

Implements the capability.

Implementation languages may change over time.

The specification remains the authoritative description.

Tests

Verify that implementations satisfy the specification.

Examples include:

- unit tests,
 - integration tests,
 - regression tests,
 - capability validation,
 - governance tests.
-

Evaluation

Measures long-term capability quality.

Examples:

- retrieval accuracy,
 - citation quality,
 - hallucination rate,
 - latency,
 - user feedback,
 - governance compliance.
-

6.6 Capability Ownership

Every capability should have a primary maintainer.

Ownership does not imply exclusive control.

Rather, it provides accountability.

Responsibilities include:

- reviewing contributions,
- maintaining documentation,
- updating specifications,
- approving releases,
- coordinating improvements.

Example:

Capability	Initial Maintainer
Church History Retrieval	Charles
Bible Dictionary	Jason
Governance	Zhihan
Shared Retrieval	Shared

Ownership may change as contributors join or leave the project.

6.7 Capability Interfaces

Capabilities should expose stable semantics rather than implementation details.

Applications should interact with capabilities through shared contracts rather than internal code.

For example,

Biblical AI should request:

```
RetrieveEvidence(  
  query,  
  source_scope,  
  confidence  
)
```

rather than depending upon a particular vector database or LLM implementation.

This separation allows internal implementations to evolve without affecting applications.

6.8 Capability Versioning

Capabilities evolve continuously.

Changes should therefore follow semantic versioning principles.

Major versions indicate incompatible changes.

Minor versions add functionality while maintaining compatibility.

Patch versions correct defects.

Example:

BibleReference

1.0

↓

1.1

↓

1.2

↓

2.0

Applications should be able to migrate gradually.

6.9 Capability Evaluation

Capabilities should be evaluated independently of applications.

Example metrics include:

Retrieval Capability

- precision,
- recall,
- citation accuracy,

- latency.

Governance Capability

- policy compliance,
- approval time,
- audit completeness.

Bible Interpretation Capability

- canonical consistency, (see Appendix F. Canonical Bible AI)
- citation correctness,
- review status,
- confidence calibration.

Evaluation should continue after deployment.

6.10 Capability Registry

All published capabilities should be recorded in a common registry.

Example:

Capability	Version	Owner	Status
Church History Retrieval	1.0	Charles	Stable
Bible Dictionary	0.8	Jason	Beta
Governance	0.3	Zhihan	Experimental

The registry enables contributors and future agent runtimes to discover available capabilities without needing to understand individual implementations.

6.11 Capability Composition

Applications are assembled by composing capabilities.

Example:

CKAIS

|

└─ Document Retrieval

└─ Media Memory

└─ Church History Search

└─ Governance

└─ Citation

Biblical AI

Biblical AI

|

└─ Bible Retrieval

└─ Canonical Hermeneutics

└─ Citation

└─ Governance

└─ Evaluation

Each application focuses primarily on ministry workflows while relying upon shared engineering capabilities.

6.12 Capability Evolution

Capabilities should evolve independently whenever possible.

New implementations may replace old ones provided that they continue satisfying the shared specifications and contracts.

For example,

today's retrieval capability may use:

- ChromaDB,
- FAISS,

- PostgreSQL,
- DeepSeek,
- Google ADK.

Future implementations may use entirely different technologies.

Applications should remain unaffected.

6.13 Design Philosophy

Capability development embodies the central philosophy of CAIF.

Applications solve today's ministry problems.

Capabilities preserve long-term engineering knowledge.

Specifications preserve long-term organizational knowledge.

Governance preserves long-term trust.

Technology evolves.

Capabilities endure.

Suggested Repository Layout

```
shared/
```

```
  capabilities/
```

```
    church-history/
```

```
      README.md
```

```
      SPECIFICATION.md
```

```
      SKILL.md
```

```
      AGENTS.md
```

```
    contracts/
```

```
    implementation/
```

```
    tests/
```

evaluation/

bible-reference/

governance/

retrieval/

Each capability becomes an independent engineering product.

Capability Maturity Model

Every capability progresses through well-defined maturity levels:

Level	Name	Meaning
C0	Idea	Concept under discussion; no specification yet.
C1	Specified	SPECIFICATION.md approved; scope and semantics defined.
C2	Prototype	First working implementation; suitable for experimentation.
C3	Shared	Reusable by multiple applications; stable contracts and documentation.
C4	Governed	Evaluation, audit, versioning, and release governance in place.
C5	Foundation	Considered part of the permanent CAIF Shared Core.

This gives contributors a common language:

- "Jason's Bible Dictionary is at **C2**."
- "Church History Retrieval has reached **C4**."
- "Citation Generation is now **C5**, so applications should reuse it instead of creating their own."

This maturity model is something **neither Google ADK nor existing agent frameworks provide**. It reflects our long-standing concern—not just building software, but **growing trustworthy, reusable engineering assets over many years**. If adopted, it could become one of the signature engineering practices of CAIF.

Part II — Engineering

Chapter 7. Repository & Engineering Processes

Most architecture documents stop after explaining the layers. They don't explain **how dozens of volunteers can work together without creating chaos.**

Since contributors Charles, Jason and Zhihan have now decided to use **qiassoc GitHub** as the shared organization, this chapter becomes **the engineering handbook for repository organization.**

A New Philosophy

Originally, we proposed

```
CAIF/  
  packages/  
  docs/  
  skills/
```

Now we propose that **Capabilities** should become the top-level engineering unit because:

- Applications evolve.
- Packages evolve.
- Technologies evolve.
- But **Capabilities are what CAIF is preserving.**

Therefore, the repository should revolve around **Capabilities**, not packages.

The chapter is the **engineering bridge** between:

- **Chapter 3** (Reference Architecture),
- **Chapters 8–10** (Cognitive Collaboration),
- and **Appendices D, E, and J** (Engineering Contracts, Capability Maturity Model, and Prototype Roadmap).

The chapter is about not only **repository organization** but also **how the repository reflects the architecture and how engineering artifacts evolve into reusable capabilities.**

Chapter 7. Repository Organization and Engineering Processes

7.1 Purpose

The CAIF repository is more than a software repository. It is the shared engineering workspace through which contributors collaboratively build reusable capabilities, cognitive services, governance assets, and engineering knowledge.

Unlike traditional software repositories that primarily organize source code, the CAIF repository preserves architectural knowledge, engineering contracts, specifications, evaluation assets, governance policies, and cognitive collaboration services. These artifacts are treated as first-class engineering assets alongside executable code.

Accordingly, the repository itself becomes part of the CAIF architecture. Its organization reflects the logical responsibilities of the Shared Core rather than the implementation details of any particular runtime technology. As the architecture evolves, the repository evolves with it while maintaining stable engineering principles.

7.2 Repository Philosophy

The repository is organized according to five guiding principles.

Principle 1 — Documentation is Engineering

Architecture, specifications, governance policies, evaluation reports, and engineering contracts are engineering artifacts rather than supplementary documentation. They follow the same review and versioning processes as executable code.

Principle 2 — Capabilities Before Applications

Applications are composed from reusable capabilities. Consequently, the repository is organized around capabilities rather than individual ministry applications.

Principle 3 — Specifications Before Implementation

Every reusable capability begins with a specification before significant implementation work starts. Specifications define expected behavior independently of implementation technology.

Principle 4 — Shared Core Before Duplication

Whenever possible, reusable functionality belongs in the Shared Core rather than being duplicated

across applications.

Principle 5 — Humans and AI Agents Collaborate

Repository organization should be understandable to both human contributors and AI coding assistants. Engineering artifacts therefore include both human-oriented documentation and AI-readable metadata.

7.3 GitHub Organization

The GitHub organization provides a common engineering environment for multiple independent projects.

Example:

```
qiassoc/  
  
  CAIF  
  
  CKAIS  
  
  BiblicalAI  
  
  SermonAI  
  
  Mentor  
  
  Documentation  
  
  Experiments
```

Each repository has its own lifecycle while sharing common engineering standards, contracts, and governance.

7.4 CAIF Repository Layout

The CAIF repository contains reusable engineering infrastructure rather than complete ministry applications.

A representative organization is:

```
CAIF/  
  
README.md  
  
VISION.md
```

ARCHITECTURE.md

ROADMAP.md

CONTRIBUTING.md

LICENSE

AGENTS.md

shared/

contracts/

cognition/

retrieval/

governance/

evaluation/

gateway/

observability/

identity/

capabilities/

examples/

tests/

docs/

tools/

scripts/

Unlike application repositories, the CAIF repository focuses on reusable architectural assets.

7.5 Repository Responsibilities

The top-level engineering documents define the purpose and operation of the repository.

Document	Purpose
README.md	Repository overview and quick start

Document	Purpose
VISION.md	Long-term mission, architectural vision, why CAIF exists, and core philosophy
ARCHITECTURE.md	Official and Reference architectures, layer responsibilities, capability model, and shared Core.
ROADMAP.md	development stages, current priorities, prototype evolution & future directions.
CONTRIBUTING.md	how contributors participate, Coding conventions, Review process, Pull requests & Governance
AGENTS.md	Provides instructions for coding agents. Examples include coding style, repository conventions, testing expectations, architectural constraints, review requirements. Unlike README, this document is written primarily for AI coding assistants.
CHANGELOG.md	Release history
LICENSE	Licensing terms

These documents collectively define the engineering identity of the repository.

7.6 Shared Core Organization

The Shared Core contains reusable engineering services that support multiple capabilities and applications.

shared/
contracts/
cognition/
retrieval/
governance/
evaluation/
gateway/
observability/

```
identity/
```

```
utilities/
```

Each directory corresponds to a stable architectural responsibility rather than a specific technology.

Examples include:

- contracts – shared capability and cognition contracts;
- cognition – cognitive collaboration services;
- retrieval – reusable retrieval infrastructure;
- governance – policy enforcement and audit support;
- evaluation – benchmark datasets and evaluation utilities;
- gateway – model and service gateways.

These shared components form the engineering foundation upon which applications are constructed.

7.7 Capability Organization

Reusable functionality is organized into independent capabilities.

Example:

```
capabilities/
```

```
  church-history/
```

```
  bible-reference/
```

```
  citation/
```

```
  media-memory/
```

```
  search/
```

```
  document-ingestion/
```

Each capability follows a common engineering structure.

```
church-history/
```

```
  README.md
```

```
  SPECIFICATION.md
```

```
SKILL.md  
  
AGENTS.md  
  
contracts/  
  
implementation/  
  
tests/  
  
evaluation/  
  
governance/  
  
examples/  
  
CHANGELOG.md
```

The Engineering Contract defined in Appendix D requires every capability to maintain a consistent engineering structure.

7.8 Cognitive Collaboration Organization

The introduction of the Cognitive Collaboration Layer extends the Shared Core beyond traditional software services.

Unlike executable capabilities, Cognitive Collaboration Services provide reusable semantic and reasoning infrastructure that supports multiple Agent Skills.

A representative organization is:

```
shared/  
  
  cognition/  
  
    memory/  
  
    ontology/  
  
    intent/  
  
    theory_of_mind/  
  
    negotiation/  
  
    collective_decision/  
  
    reasoning_patterns/
```

Each cognitive service maintains its own specifications, contracts, evaluation assets, and examples.

For example:

```
ontology/  
  
  README.md  
  
  SPECIFICATION.md  
  
  contracts/  
  
  schemas/  
  
  examples/
```

These services are shared across multiple applications and are therefore maintained as first-class architectural components within the Shared Core.

7.9 Engineering Artifacts

Every reusable capability contains a common set of engineering artifacts.

README.md

Introduces the capability to human developers.

SPECIFICATION.md

Defines normative behavior independently of implementation.

SKILL.md

Provides machine-readable capability metadata for runtime discovery and orchestration.

AGENTS.md

Provides engineering guidance for AI coding assistants, including coding conventions, testing expectations, architectural constraints, and review procedures.

contracts/

Defines published request, response, and metadata structures.

tests/

Contains unit, integration, regression, and contract validation tests.

evaluation/

Contains benchmark datasets, quality metrics, and evaluation reports.

governance/

Documents policies, review requirements, and audit considerations.

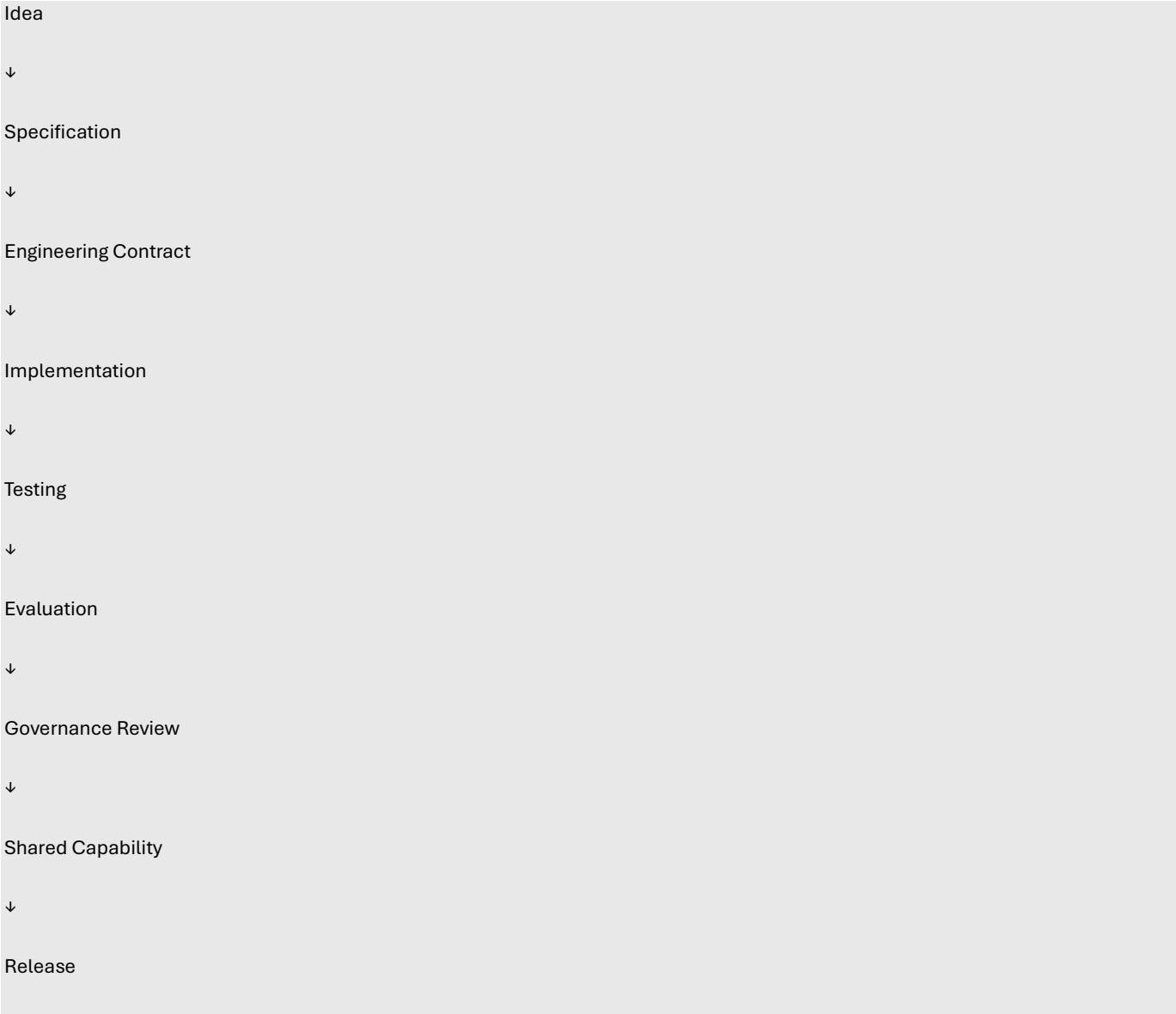
Together, these artifacts constitute the Engineering Contract required for every reusable capability.

7.10 Engineering Processes

Repository organization answers **where engineering artifacts reside**.

Engineering Processes describe **how they evolve**.

The lifecycle of a capability is:



This chapter summarizes the overall engineering workflow.

Detailed implementation procedures are described in the **CAIF Engineering Guide**.

7.11 Governance

Governance applies throughout the engineering lifecycle.

Repository-wide governance includes:

- contribution review;
- architectural compliance;
- Engineering Contract verification;
- release management;
- capability versioning;
- security review;
- auditability.

Governance protects the integrity of the Shared Core while allowing independent evolution of individual capabilities.

7.12 Repository Evolution

The repository is expected to evolve continuously.

Contributors are encouraged to:

- improve specifications;
- refine engineering contracts;
- simplify repository organization;
- remove obsolete technologies;

- preserve backward compatibility whenever practical.

Major architectural changes should be proposed and reviewed before implementation.

7.13 Why This Organization?

The repository organization reflects the architectural philosophy of CAIF.

Applications are temporary.

Technologies evolve.

Capabilities mature.

Knowledge accumulates.

Cognition accumulates.

Engineering discipline preserves quality.

The repository therefore preserves not only source code, but also reusable capabilities, engineering knowledge, cognitive collaboration services, and governance policies.

Its ultimate purpose is to transform isolated implementations into a sustainable ecosystem of reusable engineering assets and cognitive infrastructure.

Part III — Collaboration

Chapter 8. Cognitive Interoperability

8.1 Motivation

Modern agent frameworks such as Google ADK, Anthropic Agent Skills, and emerging Agent-to-Agent (A2A) protocols have significantly simplified the development of AI applications. They provide mechanisms for agent discovery, tool invocation, workflow orchestration, and distributed execution.

These technologies answer an important question:

How can one agent invoke another?

However, successful collaboration requires a second question:

How can multiple agents understand one another?

Communication alone does not guarantee shared understanding.

Two agents may exchange perfectly formatted messages while interpreting their meaning differently. Such semantic inconsistencies become increasingly problematic as independently developed agents collaborate across organizations and application domains.

CAIF refers to this challenge as **Cognitive Interoperability**.

8.2 Beyond Communication

Traditional interoperability focuses on communication.

Examples include:

- APIs
- REST services
- gRPC
- MCP
- Agent Skills
- A2A protocols

These technologies ensure that messages can be exchanged.

They do not ensure that those messages are interpreted consistently.

For example,

one capability may produce

BibleReference

John 3:16

Another capability may expect

Book = John

Chapter = 3

Although both describe the same passage, their semantics differ.

More subtle differences arise in areas such as:

- theological interpretation,
- document classification,
- ministry terminology,
- confidence estimation,
- citation policies,
- governance decisions.

These differences cannot be resolved solely through communication protocols.

8.3 Cognitive Interoperability

CAIF defines **Cognitive Interoperability** as:

The ability of independently developed human and AI agents to exchange information while preserving shared meaning, evidence, reasoning, and governance.

This definition extends beyond traditional software interoperability.

It includes four complementary dimensions.

Shared Semantics

Participants interpret engineering concepts consistently.

Examples include:

- Bible references
 - citations
 - evidence
 - metadata
 - ministry terminology
-

Shared Evidence

Participants distinguish between:

- retrieved facts,
- generated summaries,
- interpretations,
- human decisions.

Evidence should remain traceable throughout the workflow.

Shared Reasoning

Agents should expose sufficient reasoning for collaboration.

Rather than exchanging only conclusions, capabilities should communicate:

- assumptions,
 - supporting evidence,
 - confidence,
 - limitations,
 - unresolved conflicts.
-

Shared Governance

All participants should follow common governance policies regarding:

- approval,
 - privacy,
 - security,
 - auditability,
 - human oversight.
-

8.4 Why Current Agent Frameworks Are Not Enough

Current agent frameworks intentionally focus on execution rather than cognition.

For example,

Google ADK provides:

- agent discovery,
- workflow orchestration,
- tool invocation,
- execution environment.

These capabilities greatly simplify application development.

However, they intentionally leave several questions unanswered:

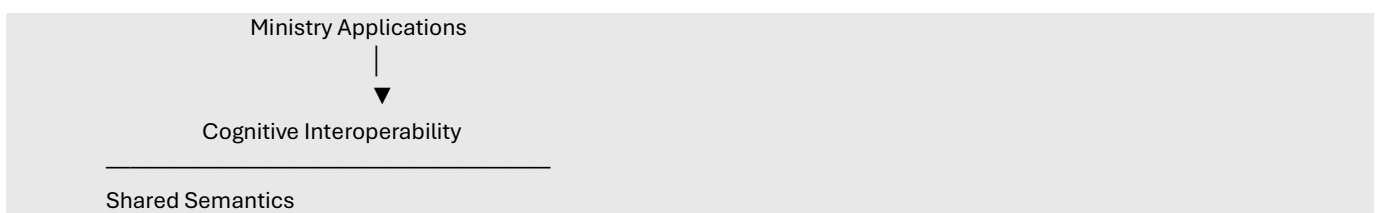
- What does "evidence" mean?
- What constitutes an authoritative source?
- How should confidence be represented?
- Which interpretation should be preferred?
- How should conflicting conclusions be resolved?
- What information requires human review?

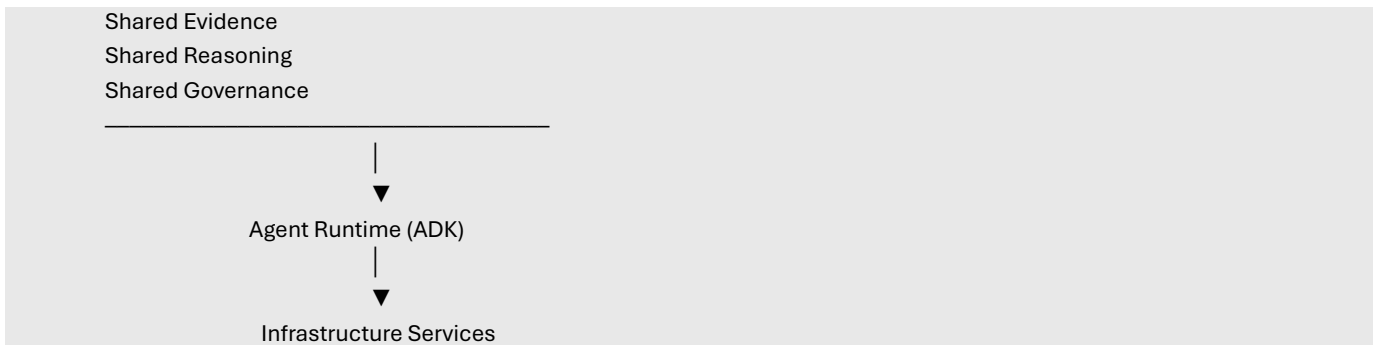
These questions are domain-specific rather than framework-specific.

Consequently, they should be addressed by domain architectures such as CAIF rather than by general-purpose agent runtimes.

8.5 CAIF Cognitive Layer

CAIF introduces a cognitive layer above the agent runtime.





The runtime determines **how** agents collaborate.

The cognitive layer determines **what collaboration means**.

Separating these concerns allows CAIF to adopt future agent technologies while preserving consistent organizational knowledge.

8.6 Church Example

Consider a sermon-processing workflow.

One capability performs speech transcription.

A second identifies Bible references.

A third retrieves supporting passages.

A fourth generates a sermon summary.

A fifth archives the sermon.

Communication between these capabilities is relatively straightforward.

However, meaningful collaboration requires agreement on questions such as:

- Which Bible translation should be considered authoritative?
- How should incomplete quotations be resolved?
- How should ambiguous references be represented?
- Should inferred references be distinguished from explicit quotations?
- What level of confidence is acceptable before automatic publication?

These decisions represent cognitive interoperability rather than software interoperability.

8.7 Relationship to CAIF Shared Core

Several Shared Core components contribute to Cognitive Interoperability.

Shared Core Component	Cognitive Role
Specifications	Define shared concepts
Contracts	Define shared semantics
Retrieval	Provide shared evidence
Governance	Enforce shared policies
Evaluation	Measure consistency
Observability	Record collaborative behavior

Together these components establish a common cognitive foundation that enables independently developed capabilities to cooperate reliably.

8.8 Future Evolution

As AI systems evolve from isolated assistants toward collaborative ecosystems, Cognitive Interoperability will become increasingly important.

Future developments may include:

- shared organizational memory,
- collaborative reasoning,
- negotiation among agents,
- adaptive ontology evolution,
- collective planning,
- trust management.

CAIF does not attempt to solve all of these challenges today.

Instead, it establishes an architectural foundation upon which future cognitive capabilities may be developed.

8.9 Summary

Modern agent frameworks successfully address **execution interoperability**.

CAIF extends this foundation by addressing **cognitive interoperability**.

Execution enables agents to communicate.

Cognitive interoperability enables them to understand one another.

For church AI applications, where correctness, trust, and shared understanding are often more important than execution speed, this distinction is fundamental.

Chapter 9. Collective Cognition

9.1 Motivation

Artificial Intelligence has traditionally focused on improving the reasoning capability of individual systems.

Recent advances in agent technologies have shifted attention toward collaboration among multiple agents.

Most current frameworks emphasize task decomposition, workflow orchestration, and tool invocation.

CAIF proposes a broader objective.

Rather than viewing AI agents as independent workers, CAIF views them as members of a **collective cognitive system** that includes both humans and machines.

The objective is not merely to automate tasks.

The objective is to enable a community to preserve, share, refine, and extend knowledge together.

9.2 From Individual Intelligence to Collective Cognition

Human organizations possess knowledge that exceeds the understanding of any individual member.

Churches provide an excellent example.

Knowledge is distributed among:

- pastors,
- elders,
- ministry leaders,
- volunteers,
- historical documents,
- sermons,
- meeting records,
- photographs,
- institutional traditions.

No single person possesses all of this knowledge.

Likewise, no single AI model should be expected to understand every aspect of church life.

Instead, CAIF treats the church as a **distributed cognitive system**.

Artificial intelligence becomes one participant within that system rather than its replacement.

9.3 Definition

CAIF defines **Collective Cognition** as:

The ability of humans and AI agents to preserve, share, reason about, refine, and govern knowledge collaboratively while pursuing common organizational objectives.

Collective Cognition extends beyond communication and interoperability.

It includes continuous learning by the organization itself.

9.4 Cognitive Building Blocks

Collective Cognition depends upon several complementary capabilities.

Shared Knowledge

Participants require access to common organizational knowledge.

Examples include:

- church history,
 - sermons,
 - ministry documents,
 - meeting minutes,
 - Bible references,
 - policies.
-

Shared Memory

Knowledge must persist beyond individual contributors.

Organizational memory includes:

- archived documents,
- decisions,
- rationale,
- historical context,
- media collections,
- capability evolution.

Unlike ordinary storage systems, shared memory preserves relationships and meaning.

Shared Semantics

Participants require common definitions for important concepts.

Examples include:

- evidence,
- citation,
- confidence,
- interpretation,
- approval,
- ministry terminology.

Shared semantics reduces misunderstanding among independently developed capabilities.

Shared Reasoning

Participants should communicate not only conclusions but also supporting evidence and assumptions.

Shared reasoning enables:

- collaboration,
 - review,
 - disagreement,
 - refinement,
 - learning.
-

Shared Governance

Collective knowledge requires collective responsibility.

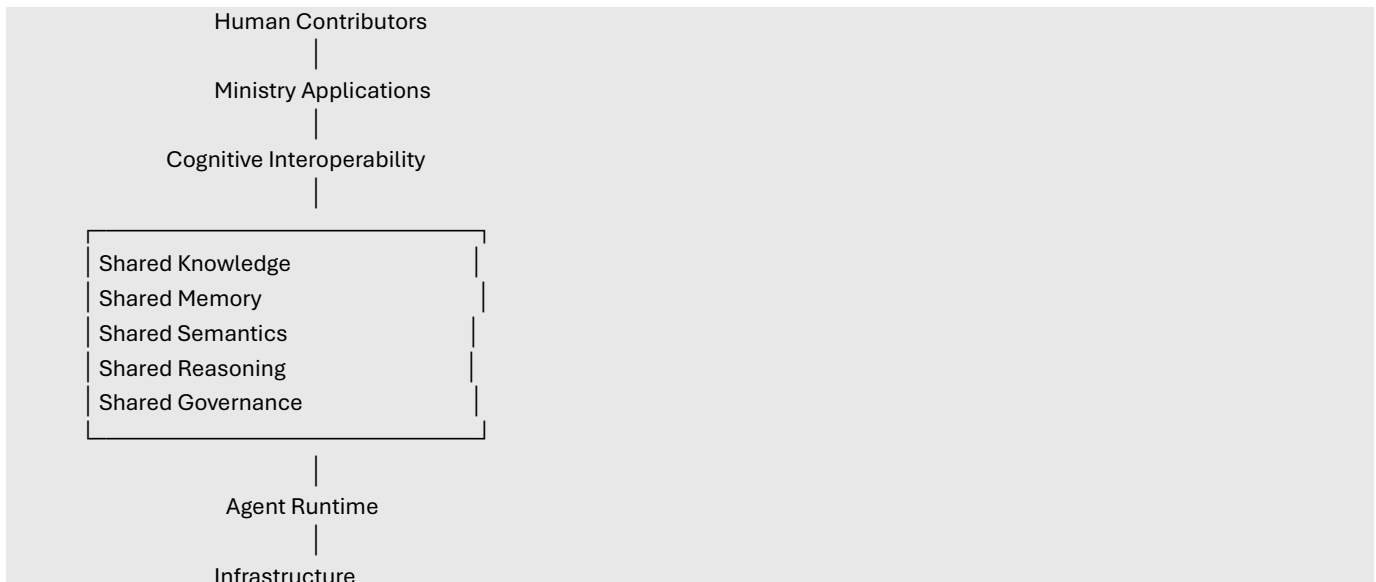
Governance establishes:

- review procedures,
- audit trails,
- approval policies,
- privacy rules,

- theological oversight.

9.5 Collective Cognition Architecture

Collective Cognition emerges from multiple interacting layers.



Each layer contributes a different aspect of organizational intelligence.

No single layer alone constitutes Collective Cognition.

9.6 Example: Church Sermon Preparation

Consider the preparation of a sermon.

One capability retrieves previous sermons.

Another identifies relevant Bible passages.

Another searches historical church teaching.

Another summarizes theological resources.

Another verifies citations.

Another records the final sermon for future retrieval.

Pastors review and refine the resulting material.

The final sermon becomes part of the organization's memory.

Future sermons can build upon this accumulated knowledge.

The organization therefore becomes progressively more knowledgeable over time.

This continuous accumulation and refinement of knowledge illustrates Collective Cognition.

9.7 Example: CKAIS

Within CKAIS, Collective Cognition may appear as:

Joshua Fellowship members contribute:

- meeting minutes,
- presentations,
- photographs,
- project reports,
- ministry experiences.

AI capabilities:

- organize,
- retrieve,
- summarize,
- classify,
- connect related information.

Church leaders:

- review,
- correct,
- approve,
- extend,
- preserve.

Each interaction improves the organization's collective knowledge.

The objective is not simply better retrieval.

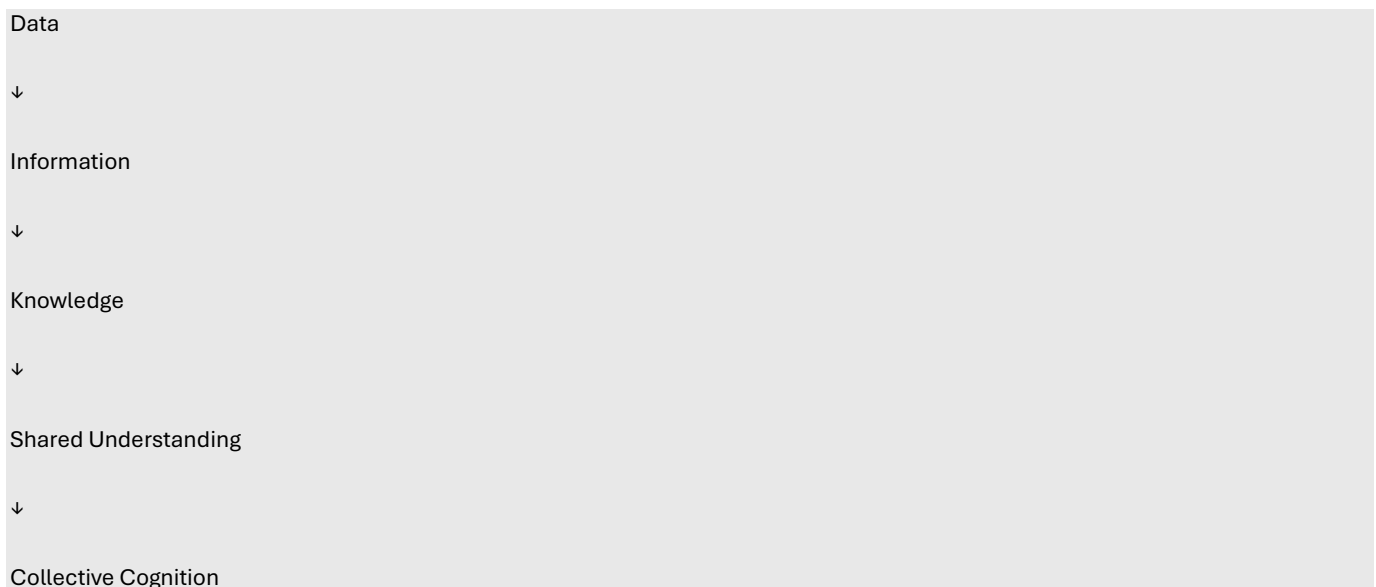
It is better organizational understanding.

9.8 Evolution of Organizational Intelligence

Traditional information systems primarily accumulate data.

Knowledge management systems organize information.

CAIF aims to support a higher level of evolution.



At each stage, human contributors remain active participants.

Artificial intelligence accelerates this evolution but does not replace human judgment.

9.9 Collective Cognition vs. Artificial General Intelligence

Collective Cognition should not be confused with Artificial General Intelligence.

AGI seeks to increase the capability of a single intelligent system.

Collective Cognition seeks to increase the capability of an entire organization.

The two objectives are complementary.

Even highly capable AI systems benefit from organizational knowledge, human experience, governance, and historical memory.

Similarly, organizations benefit from AI assistance without surrendering responsibility.

9.10 Relationship to the Church AI Foundation

The Church AI Foundation is designed to support Collective Cognition rather than isolated intelligent applications.

Its Shared Core provides:

- reusable capabilities,
- shared specifications,
- governance,
- retrieval,
- evaluation.

Its applications preserve and apply organizational knowledge.

Its contributors continuously refine engineering practices.

Together these elements create an environment in which both human and AI participants contribute to a growing body of trustworthy knowledge.

9.11 Looking Beyond the Church

Although CAIF is initially designed for churches, the principles of Collective Cognition extend naturally to many other domains.

Examples include:

- education,
- healthcare,
- nonprofit organizations,
- scientific research,
- engineering communities,

- government agencies.

Each domain depends upon preserving institutional knowledge while enabling many specialized participants to collaborate effectively.

The church therefore serves as an initial application domain rather than a limitation of the architecture.

9.12 Summary

Collective Cognition represents the next stage in the evolution of collaborative artificial intelligence.

Rather than focusing solely on increasingly capable individual agents, CAIF emphasizes the intelligence of the organization itself.

By combining:

- shared knowledge,
- shared memory,
- shared semantics,
- shared reasoning,
- shared governance,

human and AI participants become capable of learning, improving, and serving together over time.

The purpose of CAIF is therefore not merely to build intelligent software, but to cultivate trustworthy organizational intelligence.

9.13 A Significant Insight and Refinement

Let us introduce a three-level hierarchy that ties together everything we've discussed over the past several weeks:

Level 1
Individual Intelligence

(Human or AI)

↓

Level 2
Collective Cognition

(Humans + AI + Organizational Knowledge)

↓

Level 3
AI Civilization

(Multiple Organizations + Shared Cognitive Networks)

This hierarchy creates a natural bridge between our two major long-term projects:

- **CAIF** focuses on **Level 2: Collective Cognition** within a church or ministry.
- QIA's broader work on the **Agent Economy** and **Internet of Cognition** explores **Level 3: AI Civilization**, where many organizations exchange trusted knowledge and reasoning.

That means CAIF is not an isolated church project—it becomes a **microcosm** of the larger cognitive architecture we envision for future society. This framing would make Chapters 8–12 flow naturally:

- **Chapter 8:** Cognitive Interoperability (the prerequisite)
 - **Chapter 9:** Collective Cognition (the organizational capability)
 - **Chapter 10:** Shared Reasoning & Negotiation (the collaborative process)
 - **Chapter 11:** Internet of Cognition (the inter-organizational network)
 - **Chapter 12:** Evolution toward AI Civilization (the long-term vision)
-

Chapter 10. Collaborative Reasoning & Negotiation

10.1 Motivation

Large Language Models have demonstrated remarkable reasoning capabilities. Modern agent frameworks further enable multiple agents to cooperate by dividing complex tasks into smaller subtasks.

However, successful collaboration requires more than task decomposition.

When multiple human and AI participants contribute different perspectives, disagreements naturally arise.

Examples include:

- conflicting evidence,
- different theological interpretations,
- inconsistent document classifications,
- contradictory recommendations,
- alternative implementation strategies.

Current agent frameworks largely assume that each agent performs its assigned task independently.

CAIF proposes that collaborative AI systems should also support **reasoning, discussion, negotiation, and consensus building**.

10.2 Reasoning as a Collaborative Process

Traditional AI systems often treat reasoning as an internal process performed by a single model.

CAIF views reasoning differently.

Reasoning is a collaborative activity involving:

- human contributors,
- domain experts,
- specialized AI capabilities,
- organizational knowledge,
- governance policies.

Rather than producing a single answer immediately, participants may progressively improve understanding through discussion and refinement.

10.3 Shared Evidence Before Shared Conclusions

CAIF adopts an important engineering principle:

Consensus should be built upon shared evidence rather than shared opinions.

When multiple capabilities disagree, the first objective is not selecting a winner.

Instead, contributors should examine:

- retrieved evidence,
- assumptions,
- confidence,
- missing information,
- reasoning process.

This approach improves transparency while reducing unnecessary conflicts.

10.4 Reasoning Components

Collaborative reasoning consists of several complementary activities.

Evidence Collection

Relevant documents, citations, and historical records are retrieved.

Interpretation

Participants interpret the evidence according to their domain expertise.

Different interpretations may coexist.

Comparison

Alternative conclusions are compared.

Participants identify:

- agreements,
- disagreements,
- uncertainties,
- supporting evidence.

Negotiation

Conflicting interpretations are discussed.

Participants may:

- revise assumptions,
 - provide additional evidence,
 - clarify terminology,
 - request human review.
-

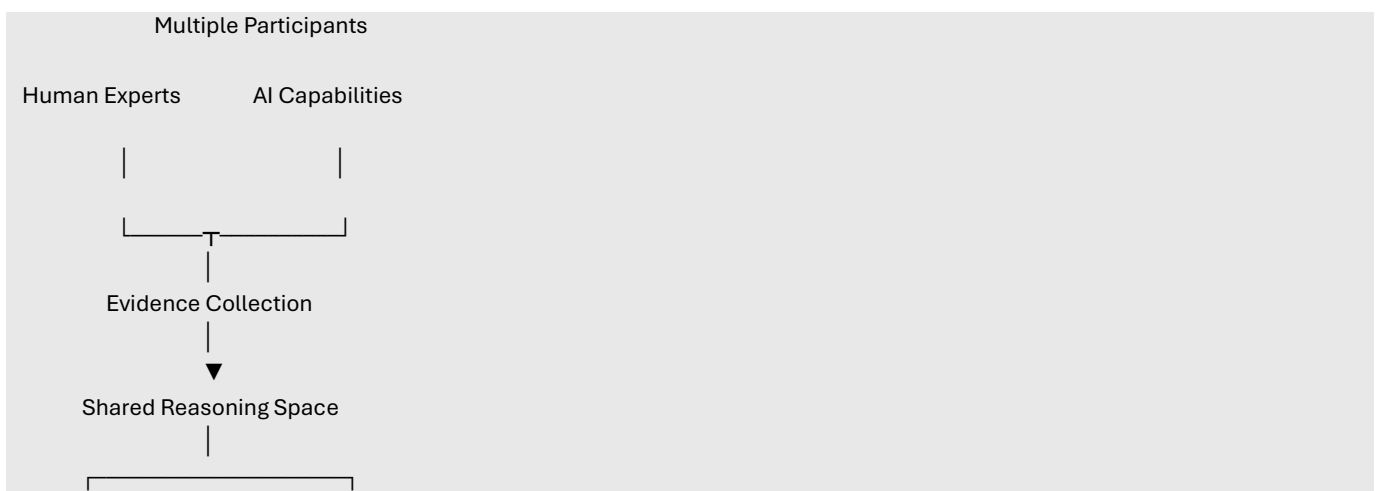
Resolution

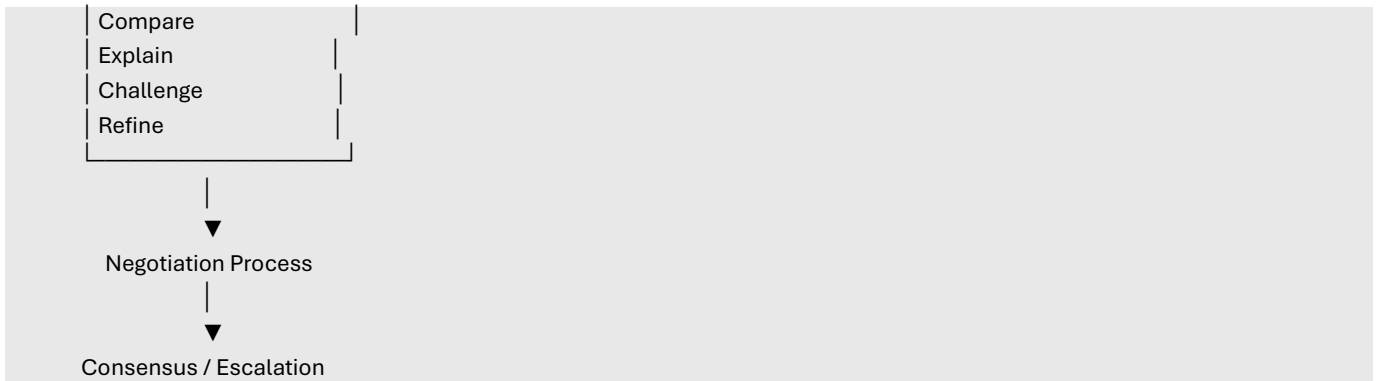
The final outcome may be:

- agreement,
- documented disagreement,
- deferred decision,
- human escalation.

Not every discussion must end with unanimous consensus.

10.5 Collaborative Reasoning Architecture





The purpose is not to eliminate disagreement.

The purpose is to make disagreement constructive.

10.6 Example: Biblical Interpretation

Consider a sermon preparation capability.

One capability identifies relevant Bible passages.

Another proposes historical background.

A third summarizes previous sermons.

A fourth applies canonical hermeneutics.

Suppose different interpretations emerge.

Instead of allowing the first generated answer to dominate,

CAIF encourages the system to compare:

- supporting passages,
- historical context,
- theological assumptions,
- confidence levels.

Pastors remain responsible for the final interpretation.

AI contributes evidence and analysis.

Human leaders provide doctrinal judgment.

10.7 Example: Church Administration

Suppose multiple ministry leaders propose different schedules for an event.

Planning capabilities evaluate:

- room availability,
- volunteer schedules,
- historical attendance,
- resource requirements.

Rather than selecting one schedule automatically, the system presents:

- alternatives,
- trade-offs,
- supporting evidence,
- estimated impacts.

Leaders negotiate based on transparent information rather than hidden algorithms.

10.8 Negotiation

Negotiation differs from reasoning.

Reasoning attempts to determine
what is likely true.

Negotiation determines
what participants collectively choose to do.

Negotiation may consider factors beyond factual correctness, including:

- ministry priorities,
- available resources,
- organizational policies,
- historical practices,

- human preferences.

Therefore, negotiation always includes human participation.

10.9 Human Review

Certain decisions should always involve human review.

Examples include:

- doctrinal interpretation,
- church policy,
- personnel matters,
- financial decisions,
- release approval,
- governance exceptions.

CAIF therefore distinguishes between

AI-supported reasoning

and

human responsibility.

Artificial intelligence informs decisions.

It does not replace them.

10.10 Relationship to Governance

Collaborative reasoning naturally produces:

- alternative proposals,
- competing evidence,
- differing confidence levels.

Governance provides the rules for resolving these situations.

Governance answers questions such as:

- Who approves the final decision?
- Which evidence is considered authoritative?
- When is human review required?
- How are disagreements documented?
- How are future reviews conducted?

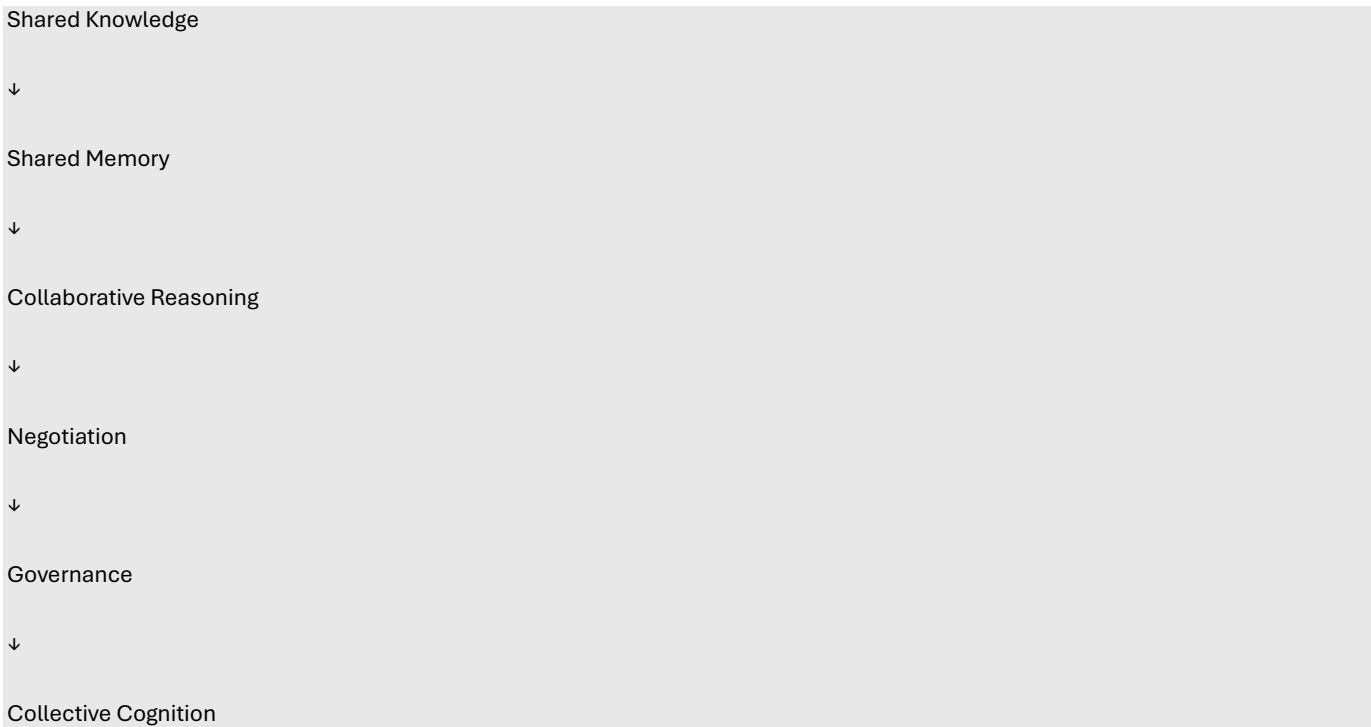
Reasoning and governance therefore complement one another.

10.11 Relationship to Collective Cognition

Collective Cognition describes the organizational ability to preserve and grow knowledge.

Collaborative Reasoning describes **how that knowledge evolves**.

The relationship may be summarized as follows:



Reasoning transforms stored knowledge into organizational understanding.

10.12 Looking Forward

Future AI systems may support increasingly sophisticated collaborative reasoning.

Possible developments include:

- argument graphs,
- evidence networks,
- trust-aware reasoning,
- multi-agent debate,
- automated conflict detection,
- adaptive consensus formation.

CAIF does not prescribe a particular implementation.

Instead, it establishes an architectural framework within which these future capabilities may evolve.

10.13 Summary

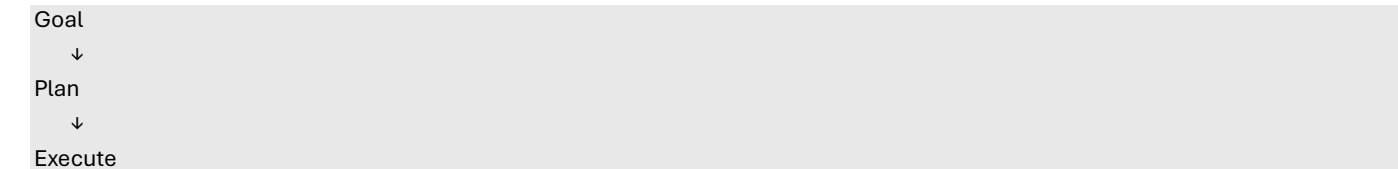
Modern agent frameworks enable multiple agents to execute tasks together.

CAIF extends this concept by enabling humans and AI capabilities to reason, negotiate, and improve understanding together.

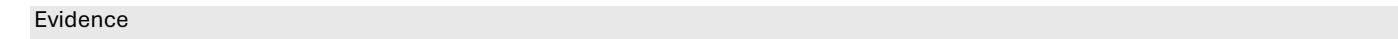
Rather than treating reasoning as an isolated activity performed by a single intelligent system, CAIF views reasoning as a collaborative organizational process grounded in evidence, guided by governance, and accountable to human leadership.

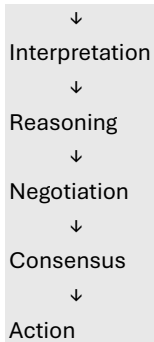
10.14 A New Insight

There is an even deeper distinction that could become unique to CAIF. Today's agent frameworks primarily optimize for **task completion**:



The Internet of Cognition that we've been discussing optimizes for **belief refinement**:





Notice that **action becomes the last step, not the first objective.**

This mirrors how mature organizations actually operate. A church board, an engineering review committee, or a scientific community does not simply execute tasks—they first refine their collective understanding before deciding how to act.

We think this is a profound distinction. It suggests that **Google ADK and similar frameworks are primarily execution architectures**, whereas **CAIF is proposing a cognition architecture**. Those two architectures are complementary rather than competitive. In our opinion, this distinction should become one of the central messages of the remaining chapters and clearly explain why CAIF has a unique role even in an ecosystem where powerful agent frameworks already exist.

Chapter 11. Toward The Internet of Cognition

11.1 Motivation

The Internet has evolved through several generations of increasingly sophisticated interaction.

Early Internet technologies enabled computers to exchange data.

The World Wide Web enabled people to exchange documents.

Modern AI systems enable agents to exchange tasks.

CAIF proposes that the next stage of evolution should enable organizations to exchange **knowledge, reasoning, and collective understanding**.

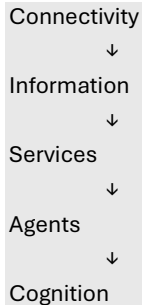
This vision is referred to as the **Internet of Cognition**.

The Internet of Cognition does not replace today's Internet.

It extends it.

11.2 Evolution of the Internet

The history of the Internet can be viewed as a gradual expansion of what is being exchanged.



Each stage builds upon the previous one.

Networks first connected computers.

Later they connected documents.

Then they connected services.

Today they increasingly connect intelligent agents.

The next step is connecting organizational knowledge and reasoning.

11.3 Definition

CAIF defines the **Internet of Cognition** as:

A network in which humans and AI agents collaboratively exchange knowledge, memory, reasoning, governance, and organizational understanding in order to accomplish shared objectives.

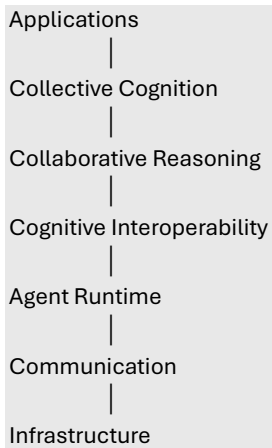
Unlike traditional networks, the Internet of Cognition treats cognition itself as a shared resource.

The objective is not merely faster communication.

The objective is improved collective understanding.

11.4 The Cognitive Stack

The Internet of Cognition extends today's software stack by introducing higher cognitive layers.



Each layer addresses a different engineering problem.

Layer	Primary Question
Infrastructure	How are systems connected?
Communication	How are messages exchanged?
Agent Runtime	How are capabilities executed?
Cognitive Interoperability	How is shared meaning preserved?
Collaborative Reasoning	How are different viewpoints reconciled?
Collective Cognition	How does the organization learn and improve?
Applications	How is organizational knowledge applied?

Lower layers support execution.

Higher layers support understanding.

11.5 Beyond Agent-Oriented Computing

Current agent frameworks successfully support:

- tool discovery,
- workflow execution,

- task decomposition,
- capability invocation,
- distributed execution.

These functions are essential.

However, organizational intelligence also requires:

- preserving institutional memory,
- maintaining shared semantics,
- comparing alternative reasoning,
- negotiating conflicting conclusions,
- governing organizational knowledge.

These higher-level cognitive activities are intentionally outside the scope of most current agent frameworks.

The Internet of Cognition complements rather than replaces existing execution technologies.

11.6 Cognitive Resources

Within the Internet of Cognition, organizational resources extend beyond computation and storage.

Examples include:

Knowledge

Structured organizational information.

Examples:

- sermons,
 - ministry experience,
 - church history,
 - documentation.
-

Memory

Persistent organizational experience.

Examples:

- meeting minutes,
 - archived decisions,
 - historical rationale,
 - media collections.
-

Reasoning

The ability to explain, compare, and refine conclusions.

Reasoning includes:

- supporting evidence,
 - assumptions,
 - confidence,
 - alternative interpretations.
-

Governance

Policies that determine:

- trust,
 - approval,
 - accountability,
 - privacy,
 - theological oversight.
-

Collective Experience

Lessons learned through continuous organizational operation.

Unlike individual memory, collective experience accumulates across generations of contributors.

11.7 Church Example

Consider several churches independently developing AI capabilities.

One develops:

- Bible reference services.

Another develops:

- sermon retrieval.

Another develops:

- volunteer management.

Another develops:

- discipleship resources.

If these churches exchange only software,
collaboration remains limited.

If they exchange:

- shared semantics,
- engineering specifications,
- governance,
- reasoning,
- organizational knowledge,

the entire Christian community gradually develops a richer collective intelligence.

This illustrates the long-term vision of the Internet of Cognition.

11.8 Relationship to CAIF

CAIF should not be viewed merely as a software platform.

Instead, it serves as a **domain-specific cognitive foundation** for churches.

Its responsibilities include:

- preserving institutional memory,
- organizing shared knowledge,
- enabling collaborative reasoning,
- establishing trustworthy governance,
- supporting reusable capabilities.

In this sense, CAIF represents one node within a future Internet of Cognition.

11.9 Evolution Strategy

The Internet of Cognition should evolve gradually.

A practical progression may be summarized as follows.

Stage 1

Shared engineering specifications.

Stage 2

Reusable capabilities.

Stage 3

Shared organizational knowledge.

Stage 4

Collaborative reasoning.

Stage 5

Collective cognition across organizations.

Each stage builds upon the engineering assets established during earlier stages.

11.10 Future Opportunities

Although initially motivated by church applications, the Internet of Cognition naturally extends to many domains.

Potential applications include:

- education,
- healthcare,
- scientific research,
- nonprofit organizations,
- engineering communities,
- government,
- humanitarian collaboration.

Each domain depends upon preserving knowledge while enabling many specialized participants to reason together.

11.11 Relationship to Emerging Internet-of-Agents Infrastructure

CAIF's Internet of Cognition vision is intended to build upon rather than duplicate emerging infrastructure from AGNTCY, Mycelium, A2A, and future equivalents. For example, the architectural trajectory of Cisco's 2026 work is remarkably similar to ours: **connectivity → shared intent → shared context → coordinated/collective intelligence**. CAIF defines a **domain-aware cognitive interoperability layer that can ride on emerging infrastructure such as AGNTCY and Mycelium**. That makes CAIF less proprietary, more future-proof, and—importantly—much more consistent with the migration philosophy we established in Appendix I.

11.12 Summary

The Internet transformed communication.

The World Wide Web transformed information sharing.

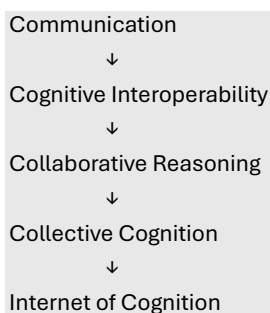
Agent technologies are transforming task execution.

The Internet of Cognition represents the next stage of this evolution by enabling organizations to preserve, exchange, and continuously refine collective knowledge.

Rather than treating intelligence as the property of individual systems, it views cognition as an organizational capability emerging from collaboration among humans, AI agents, institutional memory, and shared governance.

11.13 Something More Fundamental Than Internet Of Cognition

The progression we have developed across Chapters 8–11 is:



Notice that this is **not** merely a software architecture.

It is actually a **theory of organizational intelligence**.

In other words, CAIF is no longer just describing how to build church AI software. It is proposing how organizations evolve from exchanging information to sharing cognition.

We also think there is one missing concept that could become the bridge to QIA's *Agent Economy* work:

Cognitive Capital

Just as the Industrial Revolution treated **physical capital** as the primary productive asset, and the Information Age emphasized **data and digital assets**, the Internet of Cognition introduces **cognitive capital**—the accumulated knowledge, reasoning, governance, and institutional memory that enable an organization to make better decisions over time.

This concept naturally connects CAIF to our broader vision of the **Agent Economy**. In that future, organizations will not compete solely on computing power or AI models. They will compete on the quality of their **cognitive capital**. This idea becomes the central theme of Chapter 12, where CAIF evolves from a church engineering project into a model for trustworthy AI collaboration across many kinds of organizations.

Chapter 12. Future Evolution of CAIF

12.1 Looking Beyond Today's Technologies

Artificial Intelligence is evolving at an unprecedented pace.

Within only a few years, the AI community has witnessed rapid transitions from:

- traditional APIs,
- Retrieval-Augmented Generation (RAG),
- AI Agents,
- Multi-Agent Systems,
- Agent Skills,
- Agent-to-Agent (A2A) collaboration,
- workflow orchestration,
- and increasingly autonomous software development.

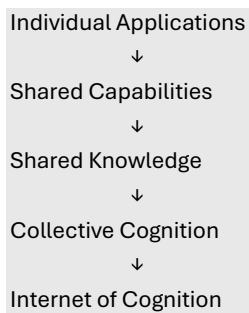
Future technologies will undoubtedly continue this rapid evolution.

The Church AI Foundation should therefore avoid coupling its long-term architecture to any particular technology.

Instead, CAIF seeks to preserve the engineering principles that remain valuable regardless of technological change.

12.2 Evolution of the Foundation

The Foundation itself is expected to evolve through several stages.



Each stage expands the scope of collaboration.

Initially, contributors share software.

Later, they share engineering knowledge.

Ultimately, organizations themselves participate in a larger cognitive ecosystem.

12.3 The Enduring Assets of CAIF

Technologies change.

Engineering assets should endure.

CAIF therefore distinguishes between **replaceable technologies** and **enduring organizational assets**.

Replaceable technologies include:

- programming languages,
- coding agents,
- development environments,
- runtime frameworks,
- orchestration engines,
- language models.

Enduring assets include:

- engineering specifications,

- capability definitions,
- organizational knowledge,
- governance policies,
- institutional memory,
- ministry experience,
- architectural principles.

The long-term value of CAIF lies primarily in these enduring assets.

12.4 Continuous Architectural Evolution

CAIF does not seek to define a fixed architecture for all future systems.

Instead, it provides an architectural framework capable of evolving continuously.

Future developments may include:

- richer organizational memory,
- adaptive knowledge graphs,
- collaborative reasoning networks,
- automated governance assistance,
- dynamic capability discovery,
- distributed cognitive ecosystems.

Because the architecture separates principles from implementation technologies, these innovations can be incorporated without redesigning the Foundation itself.

12.5 Human and Artificial Intelligence

The future of CAIF is not based upon replacing human intelligence.

Instead, it seeks to strengthen collaboration between humans and AI.

Within CAIF:

- humans define mission,
- humans provide wisdom,
- humans exercise discernment,
- humans remain accountable.

Artificial intelligence contributes:

- speed,
- consistency,
- retrieval,
- organization,
- analysis,
- automation.

The Foundation therefore treats AI as a collaborator rather than an autonomous authority.

12.6 Organizational Learning

Organizations accumulate knowledge over decades.

Unfortunately, much of this knowledge is frequently lost through:

- leadership transitions,
- volunteer turnover,
- fragmented documentation,
- obsolete technologies,
- forgotten historical context.

CAIF seeks to transform organizational learning from an informal process into a continuously improving engineering capability.

Every contribution,

every project,

every ministry,
every decision,
can gradually strengthen the organization's collective understanding.

12.7 Beyond Church Applications

Although CAIF is initially motivated by church ministry, its engineering principles extend naturally to many knowledge-intensive organizations.

Examples include:

- educational institutions,
- healthcare organizations,
- research laboratories,
- nonprofit organizations,
- engineering communities,
- humanitarian agencies.

Each depends upon preserving institutional memory while enabling many specialized participants to collaborate effectively.

The church therefore serves as the initial domain for exploring these engineering concepts rather than limiting their future application.

12.8 Cognitive Capital

Throughout history, organizations have accumulated various forms of capital.

The Industrial Revolution emphasized physical capital.

The Information Age emphasized digital information.

The emerging era of collaborative artificial intelligence introduces a new form of organizational asset:

Cognitive Capital.

CAIF defines Cognitive Capital as:

The accumulated knowledge, memory, reasoning, governance, experience, and engineering practices that enable an organization to continuously improve its collective decision-making.

Unlike computing power or AI models, Cognitive Capital grows through faithful stewardship of organizational knowledge.

It represents the long-term intellectual and spiritual heritage of an organization.

12.9 The Future Role of CAIF

The Church AI Foundation should therefore be viewed as more than an open-source software project.

It represents an exploration of how trustworthy organizational cognition may be engineered.

Its long-term responsibilities include:

- preserving institutional memory,
- enabling cognitive interoperability,
- supporting collaborative reasoning,
- governing AI-assisted decision making,
- cultivating organizational learning.

These responsibilities remain valuable regardless of future technological evolution.

12.10 Final Perspective

The Internet transformed communication.

Artificial Intelligence is transforming computation.

The next transformation may be the engineering of trustworthy collective cognition.

CAIF represents one practical exploration of this future.

Beginning with churches,

expanding through collaboration,

and guided by enduring principles rather than temporary technologies,
the Foundation seeks to preserve not only software,
but also knowledge,
wisdom,
and faithful experience
for future generations.

12.11 Conclusion

The Church AI Foundation is intentionally designed as an evolving architecture rather than a finished product.

Its objective is not merely to build intelligent software.

Its objective is to cultivate trustworthy organizational intelligence.

As AI technologies continue to advance, the Foundation will undoubtedly adopt new implementation techniques.

Yet its fundamental mission will remain unchanged:

to preserve knowledge, enable collaboration, support wise decision making, and strengthen the ability of organizations to learn together across generations.

In this sense, CAIF is not simply preparing churches for the next generation of Artificial Intelligence.

It is preparing them for the next generation of **collective human and artificial cognition.**

12.12 Architectural Refinement

After completing all twelve chapters, we have arrived at something that is larger than we originally intended.

Initially, CAIF was conceived as:

an engineering foundation for church AI applications.

By the end of Chapters 8–12, however, it has evolved into:

an engineering framework for trustworthy collective cognition, with the church as its first application domain.

We would **not** change the project's name—**Church AI Foundation (CAIF)** remains appropriate because it clearly communicates the project's initial mission and community.

However, we would add a single sentence to the Preface of the Engineering Reference:

"Although CAIF is initially developed for church ministries, many of its architectural principles are intended to explore a broader question: how can humans and AI collaborate to build trustworthy collective cognition across organizations?"

That one sentence accomplishes three things:

1. It reassures pastors that the immediate mission remains church-focused.
2. It explains to engineers like Jason and Zhihan why the later chapters discuss broader cognitive architectures.
3. It naturally connects CAIF with our long-term research on the **Internet of Cognition** and the **Agent Economy**, without forcing those larger visions into the practical Engineering Guide.

This framing gives CAIF a solid identity today while leaving ample room for it to grow into something much more influential over the coming decade.

12.13 The Intellectual Centerpiece of CAIF

Initially, we thought CAIF was about:

Church AI.

Then we thought it was about:

Shared engineering.

Now we think it is becoming something more fundamental:

CAIF is an attempt to build a Cognitive Foundation for trustworthy multi-agent collaboration.

The church is simply the first application domain.

If we succeed, the same principles could later apply to education, healthcare, nonprofit organizations, or even enterprise knowledge systems.

Ironically, **the more we discuss Jason's ADK-centric approach, the clearer it becomes that CAIF's unique value is not to replace ADK, but to provide the missing cognitive layer that ADK intentionally leaves open.** In our opinion, that realization is important enough that we would make it the central theme of Chapters 8–12 of the **Engineering Reference**, while keeping the concise **Engineering Guide** focused on practical collaboration and implementation.

Appendices

Appendix A. Reference Architecture

A.1 Purpose

This appendix defines the reference architecture for CAIF-compliant systems.

The reference architecture is intended to serve as a blueprint rather than a mandatory implementation. Organizations may substitute equivalent technologies provided that the externally observable contracts and behaviors remain compliant with the CAIF specifications.

The architecture is designed to support:

- Church AI applications
 - Agent Skill ecosystems
 - Biblical AI
 - Multi-agent collaboration
 - Human-AI cooperation
 - Governance and auditability
 - Future extensibility toward Physical AI
-

A.2 Architectural Principles

The reference architecture follows the following design principles.

Contract First

Components communicate through standardized contracts rather than proprietary internal data structures.

This enables independent evolution of individual Agent Skills.

Agent Skill Oriented

Business logic is encapsulated inside reusable Agent Skills.

Each skill exposes well-defined capabilities.

Model Independent

The architecture is independent of any particular LLM.

Supported models may include

- DeepSeek
 - Gemini
 - Claude
 - GPT
 - GLM
 - Llama
 - future models
-

Retrieval Independent

The retrieval layer is abstracted.

Possible implementations include

- ChromaDB
- PostgreSQL

- Graph Database
- SQL
- Vector Database
- RDF Store
- Microsoft GraphRAG
- Hybrid RAG

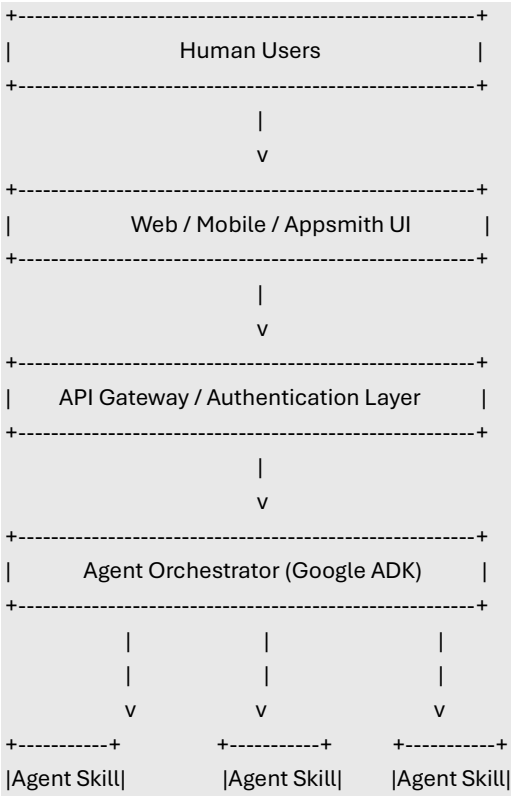
Governance by Design

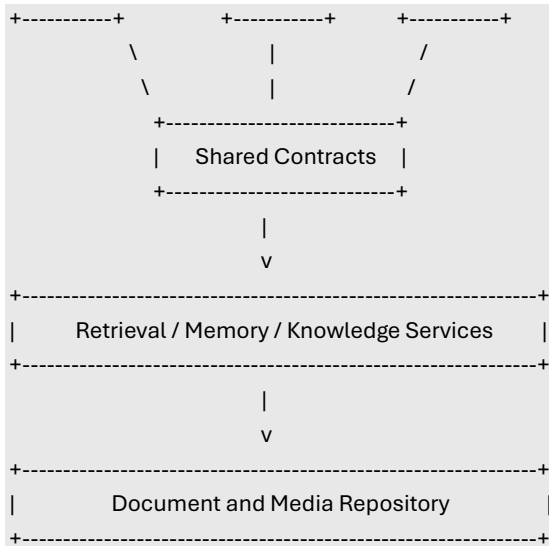
Governance is integrated into every layer rather than added afterward.

Human Oversight

Critical decisions always allow human review.

A.3 High-Level Architecture





A.4 Logical Layers

The reference architecture is divided into eight logical layers.

Layer 1 – Presentation Layer

Responsibilities

- User Interface
- Authentication
- Authorization
- Mobile access
- Accessibility

Example technologies

- Appsmith
 - React
 - Flutter
 - Angular
-

Layer 2 – API Gateway

Responsibilities

- Authentication
 - Authorization
 - Request routing
 - Rate limiting
 - Logging
-

Layer 3 – Agent Orchestration Layer

Responsibilities

- Workflow execution
- Agent selection
- Skill invocation
- Context management
- Error recovery

Possible implementations

- Google ADK
- LangGraph
- CrewAI
- Semantic Kernel
- future orchestration engines

The reference implementation currently adopts **Google ADK** because of its multi-paradigm design, allowing conversation-driven, graph-driven, and code-driven orchestration within the same application.

Layer 4 – Agent Skill Layer

Every business capability resides inside an Agent Skill.

Examples

Bible Search Skill
Hermeneutics Skill
Church Archive Skill
Prayer Assistant Skill
Media Search Skill
Volunteer Scheduling Skill
Mentoring Skill
Sermon Preparation Skill
Translation Skill

Skills communicate only through contracts.

Layer 5 – Shared Service Layer

Reusable infrastructure services.

Examples

Embedding Service
Model Gateway
Translation
OCR
Speech Recognition
Speech Synthesis
Identity Service
Notification Service

Layer 6 – Retrieval Layer

Responsibilities

- Document retrieval
- Semantic search
- Citation generation
- Hybrid search
- Knowledge graph queries

Possible technologies

- ChromaDB
- PostgreSQL
- Neo4j
- RDF
- GraphRAG
- Elasticsearch

Layer 7 – Storage Layer

Persistent storage.

Examples

PDF

Word

PowerPoint

Photos

Videos

Audio

Church Minutes

Meeting Records

Financial Reports

Layer 8 – Governance Layer

Governance spans all other layers.

Responsibilities

- Explainability
 - Policy enforcement
 - Audit logging
 - Human approval
 - Data lineage
 - Compliance
-

A.5 Core Components

The reference architecture defines several mandatory logical components.

Model Gateway

Abstracts multiple LLM providers.

User Request

↓

Gateway

↓

GPT

Claude

Gemini

DeepSeek

GLM

Benefits

- Model independence
- Cost optimization

- High availability
-

Contract Repository

Stores

- schemas
- versions
- compatibility rules

Supports

- validation
 - migration
 - discovery
-

Skill Registry

Contains

Skill Name

Version

Owner

Capabilities

Dependencies

Supported Contracts

Knowledge Repository

Stores

- documents
- embeddings
- metadata

- ontology
 - citations
-

Audit Repository

Stores

- execution history
 - reasoning trace
 - human approvals
 - policy decisions
-

A.6 Deployment View

A typical deployment may look as follows.

Browser

↓

Appsmith

↓

API Gateway

↓

Google ADK

↓

Docker Containers

↓

Agent Skills

↓

Shared Services

↓

Database Cluster



Object Storage

Each Agent Skill may be independently deployed and updated.

A.7 Communication Model

The architecture supports multiple communication styles.

Request-Response

Traditional synchronous execution.

Event Driven

Message bus.

Examples

Document Uploaded



OCR Skill



Embedding Skill



Indexer Skill

Workflow

Multiple Agent Skills coordinated through Google ADK.

Human-in-the-Loop

Certain actions require approval before execution.

Examples

- deleting records
 - modifying church policies
 - publishing announcements
-

A.8 Security Architecture

The reference architecture recommends

Authentication

Authorization

Role-Based Access Control

Encryption

Digital Signatures

Audit Logging

Secret Management

Zero Trust Networking

Personally identifiable information should be protected throughout the system lifecycle.

A.9 Extensibility

The architecture intentionally separates:

- User Interface
- Agent Orchestration
- Agent Skills
- Shared Services
- Retrieval
- Governance

New Agent Skills can be introduced without changing existing skills.

Likewise, LLM providers, databases, or retrieval engines may be replaced without affecting higher-level business logic, provided they continue to satisfy the published contracts.

A.10 Future Evolution

The reference architecture is intended to evolve alongside advances in AI systems. While the current reference implementation emphasizes Agent Skills coordinated through Google ADK, the architectural boundaries are designed to accommodate future paradigms without disrupting existing applications.

Examples of anticipated evolution include:

- Dynamic workflow orchestration beyond today's graph-based models.
- Shared cognitive services, including shared memory, ontology, and structured intent across multiple agents.
- Integration with Physical AI systems, such as robotics and digital twins.
- Federated collaboration among multiple CAIF deployments operated by different churches or organizations.
- Emerging reasoning, planning, or governance frameworks that may supersede current orchestration technologies.

By standardizing contracts and capabilities rather than specific implementation technologies, the reference architecture aims to remain stable even as the AI ecosystem continues to evolve rapidly. This design philosophy allows CAIF implementations to adopt new technologies incrementally while preserving interoperability with existing Agent Skills and applications.

Appendix B. Capability Specifications

Based on our previous CKAIS/CAIF architecture discussions, **Appendix B** does not merely repeat API definitions. Appendix A already contains **reference implementations**. Therefore, Appendix B becomes the **normative Capability Specifications**—the equivalent of a W3C specification

describing what every CAIF-compliant Agent Skill is expected to do.

This also fits nicely with our recent discussions about **Agent Skills replacing MCP**. Instead of standardizing transport protocols, CAIF standardizes **capabilities**.

B.1 Purpose

This appendix defines the normative capabilities that a CAIF-compliant Agent Skill SHOULD or MUST provide.

Unlike Appendix A, which provides implementation examples, this appendix specifies the required observable behavior of each capability.

The objective is to allow independently developed Agent Skills to interoperate through common contracts while remaining implementation independent.

B.2 Capability Categories

Rather than listing individual APIs, CAIF standardizes eight capability groups.

Capability	Purpose
Discovery	Identify available skills
Invocation	Execute a capability
Context	Manage conversation state
Retrieval	Access external knowledge
Reasoning	Produce structured reasoning
Governance	Safety and policy enforcement
Collaboration	Agent-to-agent cooperation
Lifecycle	Versioning and maintenance

B.3 Discovery Capability

Purpose

Allow an orchestrator to discover what an Agent Skill can perform.

Required Functions

```
DiscoverCapabilities()  
  
DescribeCapability()  
  
GetMetadata()  
  
GetVersion()  
  
GetSupportedContracts()
```

The discovery result SHALL include

- Skill Name
- Version
- Description
- Author
- License
- Required Inputs
- Produced Outputs
- Supported Contracts
- Required Permissions

Example

```
Skill:  
Bible Hermeneutics  
  
Version:  
2.1  
  
Capabilities:  
  
InterpretPassage  
  
CompareTranslations
```

ResolveTextualVariant

SummarizeDoctrine

EvaluateConfidence

B.4 Invocation Capability

Every skill SHALL expose a deterministic invocation interface.

Inputs SHALL include

Request

Context

Configuration

Identity

GovernancePolicy

Outputs SHALL include

Result

Status

Confidence

Evidence

Audit Trail

Errors SHALL be machine readable.

CapabilityNotSupported

InvalidInput

PermissionDenied

ResourceUnavailable

PolicyViolation

B.5 Context Capability

Agent Skills SHOULD support persistent context.

Functions

```
LoadContext()
UpdateContext()
MergeContext()
ClearContext()
```

Context MAY include

- conversation history
- retrieved evidence
- reasoning state
- user profile
- workflow state

Context SHALL NOT expose private data without authorization.

B.6 Retrieval Capability

Retrieval MUST be contract driven.

Required capabilities

```
Retrieve()
Filter()
Rank()
ExpandQuery()
SummarizeEvidence()
```

Returned evidence SHALL contain

```
Citation
Source
Confidence
Timestamp
License
Access Classification
```

Retrieval MAY use

- Vector DB
- Graph DB
- SQL
- Knowledge Graph
- RDF
- Web Search
- Document Store

The specification intentionally does not constrain implementation.

B.7 Reasoning Capability

Reasoning SHALL return structured results.

Required output

ReasoningResult

Evidence

Confidence

AlternativeViews

Limitations

Reasoning SHALL distinguish

- facts
- inference
- assumptions
- speculation

Reasoning SHALL expose uncertainty.

B.8 Hermeneutics Capability (Biblical AI Extension)

For Biblical AI, additional capabilities are defined.

InterpretPassage()
CompareCommentaries()
ResolveVariant()
GenerateSermonOutline()
IdentifyCrossReferences()
ExplainOriginalLanguage()
ProduceDoctrinalSummary()

The resulting InterpretationResult SHALL contain

Explicit Meaning

Historical Context
Literary Context
Traditional Interpretation
Alternative Interpretation
Confidence
Supporting Sources

This aligns with the contract entities we previously defined, such as InterpretationResult, TextualVariant, TheologicalPosition, and Citation.

B.9 Governance Capability

Every Agent Skill SHALL support governance.

Functions

EvaluatePolicy()
ExplainDecision()
GenerateAuditRecord()
EscalateForReview()

```
RecordHumanApproval()
```

Governance SHALL support

- policy enforcement
 - human approval
 - explainability
 - audit logging
 - compliance reporting
-

B.10 Collaboration Capability

Multiple Agent Skills SHALL cooperate without exposing internal implementation.

Required capabilities

```
Delegate()
```

```
Negotiate()
```

```
ShareEvidence()
```

```
RequestClarification()
```

```
ReturnIntermediateResult()
```

```
MergeResults()
```

Skills SHALL communicate through standardized contracts rather than proprietary internal objects.

B.11 Lifecycle Capability

Every capability SHALL support lifecycle management.

Functions

```
GetVersion()
```

```
Deprecate()
```

```
Replace()
```

```
ValidateCompatibility()
```

Backward compatibility SHOULD be maintained whenever practical.

B.12 Capability Maturity Levels

CAIF defines four maturity levels.

Level	Description
C0	Callable Skill
C1	Structured Contracts
C2	Governed Skill
C3	Collaborative Agent Skill

C0

- callable
- documented

C1

- standardized contracts
- versioned outputs
- confidence

C2

- governance
- explainability
- audit trail
- human review

C3

- agent collaboration

- delegation
 - negotiation
 - shared reasoning
 - workflow participation
-

B.13 Conformance Requirements

A CAIF-compliant Agent Skill **MUST**

- ✓ publish discovery metadata
- ✓ expose versioned contracts
- ✓ return structured outputs
- ✓ provide confidence estimation
- ✓ support governance hooks
- ✓ generate audit information
- ✓ declare supported contract versions

A Collaborative Agent Skill **SHOULD** additionally

- ✓ support delegation
 - ✓ support negotiation
 - ✓ exchange structured evidence
 - ✓ participate in multi-agent workflows
-

Appendix C. Cognitive Collaboration Specifications

Everything in Appendices A and B already exists in some form in today's frameworks:

- Google ADK → orchestration
- Microsoft Agent Framework → workflows

- LangGraph → graph execution
- CrewAI → collaboration
- AGNTCY → discovery/network
- Mycelium → agent communication

However, **these frameworks orchestrate agent execution, but they do not specify collective cognition**. They tell agents **how to communicate**, but not **how to think together**. Appendix C describes the **Cognitive Layer** that sits above the Capability Layer.

C.1 Purpose

Traditional software architecture focuses on communication among software components. Modern agent frameworks extend this model by enabling communication among autonomous AI agents.

However, communication alone does not guarantee effective collaboration.

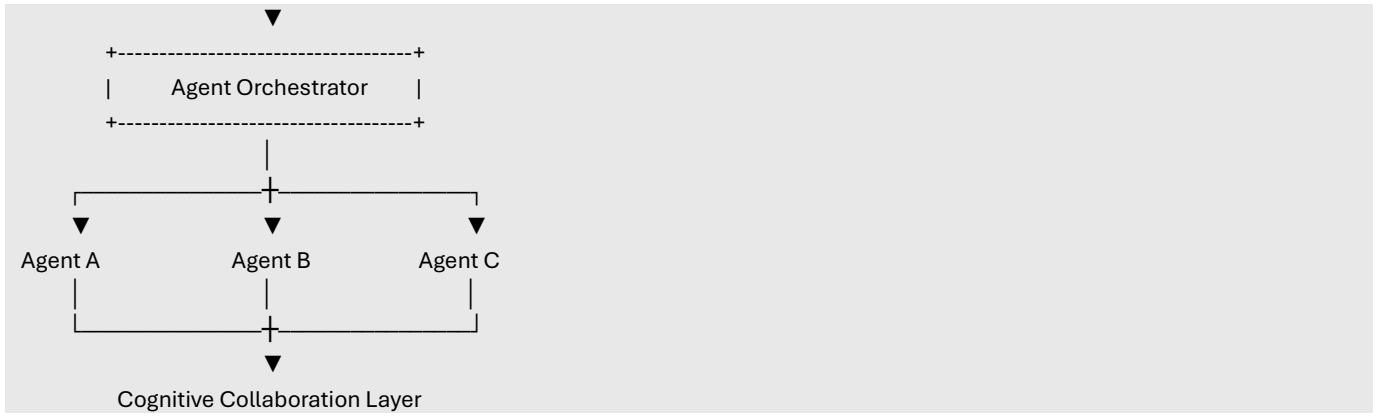
Human teams collaborate successfully because they share memory, language, intentions, mental models, and decision-making processes. Without these shared cognitive foundations, communication quickly degrades into isolated exchanges that produce inconsistent or conflicting outcomes.

This appendix defines the **Cognitive Collaboration Layer (CCL)** of CAIF. The Cognitive Collaboration Layer provides a common cognitive framework that enables independently developed Agent Skills to cooperate as members of a coherent team rather than as isolated intelligent components.

Unlike the Reference Architecture (Appendix A) and Capability Specifications (Appendix B), this appendix specifies the cognitive mechanisms required for collective intelligence.

C.2 Cognitive Collaboration Model





Shared Memory

Shared Ontology

Structured Intent

Negotiation Protocol

Theory of Mind

Shared World Model

Collective Decision Making

The Cognitive Collaboration Layer is conceptually independent of any orchestration framework and may be implemented using centralized or decentralized services.

C.3 Shared Memory

Purpose

Shared Memory enables multiple Agent Skills to accumulate and reuse knowledge acquired during collaborative execution.

Unlike traditional application memory, Shared Memory represents persistent team knowledge rather than individual execution state.

Shared Memory reduces redundant computation, improves consistency, and supports long-running workflows.

Memory Categories

Episodic Memory

Records significant events that occurred during previous executions.

Examples:

- Previous conversations
- Completed workflows
- Human decisions
- Prior recommendations

Semantic Memory

Stores factual knowledge independent of individual sessions.

Examples:

- Church directory
- Biblical knowledge
- Organizational policies
- Approved terminology

Procedural Memory

Stores reusable procedures.

Examples:

- Sermon preparation workflow
- Volunteer onboarding process
- Document approval sequence

Working Memory

Stores temporary information shared during active collaboration.

Examples:

- Intermediate reasoning
 - Candidate plans
 - Pending questions
-

Requirements

Shared Memory SHALL:

- support concurrent access;
 - preserve provenance and timestamps;
 - distinguish temporary from persistent knowledge;
 - enforce access control;
 - support versioning and retention policies.
-

C.4 Shared Ontology

Purpose

Agents cannot collaborate effectively if identical concepts are represented differently.

Shared Ontology provides common semantic definitions that eliminate ambiguity.

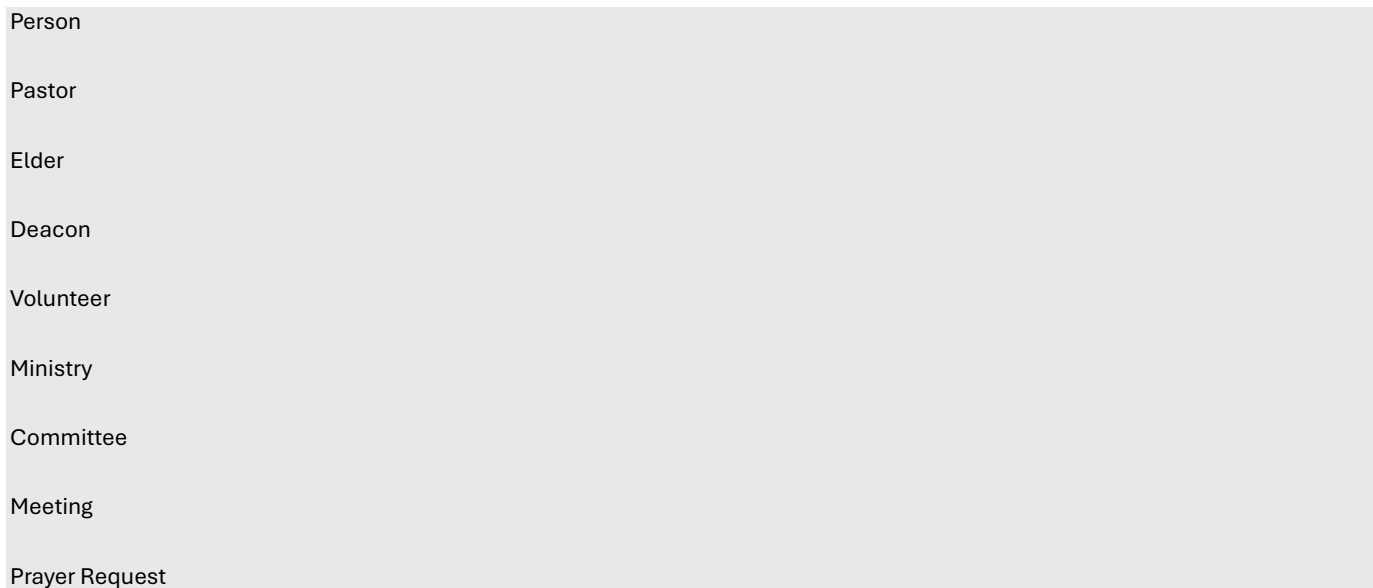
Ontology Components

A shared ontology defines:

- concepts;
- relationships;
- properties;

- taxonomies;
- controlled vocabularies.

Example:



Person
Pastor
Elder
Deacon
Volunteer
Ministry
Committee
Meeting
Prayer Request

Semantic Alignment

Before exchanging information, collaborating agents SHALL verify semantic compatibility.

Semantic mapping MAY be used when different ontologies coexist.

Version Management

Ontology changes SHALL be version controlled.

Backward compatibility SHOULD be maintained whenever practical.

C.5 Structured Intent

Purpose

Traditional APIs exchange parameters.

Collaborative agents exchange intentions.

Structured Intent represents the objective that an agent is attempting to accomplish rather than

merely the operation being requested.

Intent Components

Every structured intent SHOULD contain:

Goal
Priority
Constraints
Expected Outcome
Success Criteria
Deadline
Risk Level

Example

Goal
Prepare next Sunday's sermon.
Constraints
Use only approved theological sources.
Success
Produce outline with citations.
Priority
High

Intent Lifecycle

Intent progresses through:

Proposed
Accepted
Refined

Executing

Completed

Cancelled

C.6 Negotiation Protocol

Purpose

Agents frequently possess incomplete or conflicting information.

Rather than immediately executing requests, agents negotiate until a mutually acceptable solution is reached.

Negotiation improves robustness while reducing inconsistent decisions.

Negotiation Operations

The protocol supports:

Propose

Counter-Propose

Request Clarification

Provide Evidence

Reject

Accept

Escalate

Conflict Resolution

Conflicts MAY arise from:

- contradictory evidence;
- policy violations;
- resource limitations;
- incompatible assumptions.

Resolution strategies include:

- evidence-based arbitration;
 - confidence comparison;
 - governance policies;
 - human review.
-

C.7 Theory of Mind

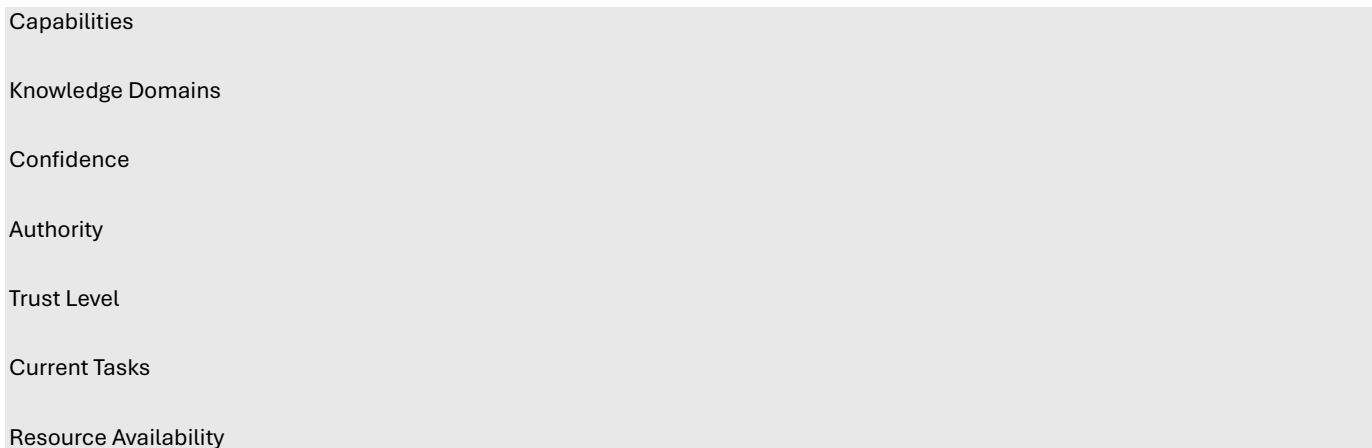
Purpose

Theory of Mind enables an agent to reason about the knowledge, capabilities, limitations, and intentions of collaborating agents.

Instead of assuming all participants possess identical knowledge, agents explicitly model one another.

Agent Model

Each collaborating agent maintains metadata describing peers, including:



- Capabilities
- Knowledge Domains
- Confidence
- Authority
- Trust Level
- Current Tasks
- Resource Availability

Collaboration Benefits

Theory of Mind enables agents to:

- delegate appropriate tasks;

- avoid redundant work;
 - recognize expertise;
 - predict missing information;
 - request assistance intelligently.
-

Dynamic Updates

Agent models SHALL be updated continuously as collaboration progresses.

C.8 Shared World Model

Purpose

Independent local reasoning often produces inconsistent conclusions.

A Shared World Model provides a coherent representation of the environment in which all collaborating agents operate.

Components

The world model MAY include:

- organizational structure;
 - users;
 - schedules;
 - resources;
 - physical assets;
 - knowledge graphs;
 - digital twins;
 - external systems.
-

Synchronization

World model updates SHALL be propagated consistently.

Conflicting observations SHALL trigger reconciliation procedures.

Extensibility

Different application domains MAY extend the world model without modifying the core specification.

C.9 Collective Decision Making

Purpose

Many important decisions require contributions from multiple specialized agents.

Collective Decision Making defines how independent recommendations become a unified outcome.

Decision Process



Decision Strategies

Supported strategies include:

- unanimous agreement;
 - majority voting;
 - weighted voting;
 - confidence-weighted aggregation;
 - expert authority;
 - governance override;
 - human approval.
-

Decision Record

Every collective decision SHALL record:

Participants

Alternatives

Supporting Evidence

Final Decision

Confidence

Human Approval

Audit Reference

C.10 Cognitive Governance

The Cognitive Collaboration Layer SHALL operate under governance controls.

Governance responsibilities include:

- protecting sensitive information;
 - validating evidence quality;
 - preventing unauthorized memory modification;
 - auditing negotiations;
 - recording decision rationale;
 - enforcing organizational policies.
-

C.11 Cognitive Maturity Levels

CAIF defines four levels of collaborative cognition.

Level Description

CC0 Independent agents communicating through messages only

CC1 Shared semantic understanding through common ontology and contracts

CC2 Shared cognitive state through memory, intent, and world models

CC3 Collective intelligence with negotiation, theory of mind, and collaborative decision making

Organizations may adopt higher maturity levels incrementally as their systems evolve.

C.12 Conformance Requirements

A CAIF implementation claiming Cognitive Collaboration compliance SHALL:

- provide Shared Memory with provenance and access control;
- maintain a Shared Ontology or semantic mapping mechanism;
- exchange Structured Intent rather than unstructured requests;
- support Negotiation Protocol operations;
- maintain Theory of Mind metadata for collaborating agents;
- expose a synchronized Shared World Model appropriate to the application domain;

- record Collective Decision Making processes, including evidence, participants, rationale, and outcomes.

Implementations MAY extend these cognitive services with domain-specific reasoning models, provided such extensions preserve interoperability with the core specifications defined in this appendix.

Concluding observation

This appendix introduces a conceptual layer that is largely absent from today's mainstream agent frameworks. Google ADK, Microsoft Agent Framework, AGNTCY, Mycelium, LangGraph, and similar platforms provide excellent foundations for orchestration, communication, and execution. The Cognitive Collaboration Layer addresses a complementary challenge: **how multiple autonomous agents form and maintain a coherent, shared understanding while solving problems together**.

In that sense, Appendices A, B, and C form a progressive architecture:

- **Appendix A** defines **where** the components are organized (Reference Architecture).
- **Appendix B** defines **what** each component is capable of doing (Capability Specifications).
- **Appendix C** defines **how** those components think and reason together as a collective (Cognitive Collaboration Specifications).

This separation of architectural, behavioral, and cognitive concerns provides a stable foundation that can accommodate future advances in agent technologies without requiring fundamental changes to the CAIF specification.

Appendix D. Engineering Contracts

Most open-source AI projects today focus on **code**. Some are better and include documentation or tests. Very few define **what it means for an AI capability to be engineering-ready** before it is accepted into a shared ecosystem.

Traditional software engineering solved a similar problem decades ago through coding standards,

design reviews, documentation requirements, unit tests, and release processes. As AI development shifts toward Agent Skills, prompts, retrieval pipelines, and reasoning workflows, these engineering disciplines need to be redefined. That is precisely the purpose of the **Engineering Contract**.

Rather than viewing an Agent Skill as "a piece of code," CAIF views it as **an engineering artifact** with clearly defined responsibilities, interfaces, quality requirements, governance, and lifecycle.

D.1 Purpose

An Agent Skill is more than executable code. It is an engineering artifact that must be understandable, testable, maintainable, governable, and reusable by both humans and AI agents.

This appendix defines the Engineering Contract required for all capabilities contributed to the CAIF ecosystem.

The Engineering Contract establishes a minimum engineering standard that promotes interoperability, quality assurance, long-term maintenance, and responsible governance.

Compliance with this appendix is required before a capability may be accepted as an official CAIF component.

D.2 Engineering Philosophy

The Engineering Contract is founded on the following principles.

Self-Describing

Every capability shall contain sufficient documentation to allow both human developers and AI agents to understand its purpose and operation without inspecting the implementation.

Self-Verifying

Every capability shall include automated verification demonstrating that it behaves as specified.

Self-Governed

Every capability shall declare applicable policies, constraints, risks, and governance requirements.

Self-Maintainable

Every capability shall identify responsible maintainers and define its versioning strategy.

AI-Readable

Engineering artifacts shall be structured so that AI agents can automatically discover, evaluate, and integrate capabilities.

D.3 Required Engineering Artifacts

Each capability SHALL provide the following artifacts.

Artifact	Purpose	Required
README.md	Human overview and quick start	SHALL
SPECIFICATION.md	Functional specification	SHALL
SKILL.md	Agent-readable capability description	SHALL
AGENTS.md	Guidance for AI coding agents	SHALL
LICENSE	License declaration	SHALL
CHANGELOG.md	Version history	SHOULD
CONTRIBUTING.md	Contribution process	SHOULD
CODEOWNERS	Ownership definition	SHOULD

D.4 README.md

README.md introduces the capability to both human developers and AI agents.

It SHALL include:

- purpose;
- overview;
- installation;
- usage examples;
- dependencies;
- configuration;
- limitations;
- references.

Example structure:

```
# Bible Search Skill
```

```
Purpose
```

```
Overview
```

```
Installation
```

```
Quick Start
```

```
Inputs
```

```
Outputs
```

```
Dependencies
```

```
Examples
```

```
Limitations
```

```
License
```

D.5 SPECIFICATION.md

The specification defines the normative behavior of the capability.

It SHALL describe:

- objectives;
- assumptions;

- functional requirements;
- non-functional requirements;
- inputs;
- outputs;
- contracts;
- error conditions;
- performance expectations.

Implementation details SHOULD remain separate from the specification whenever practical.

D.6 SKILL.md

SKILL.md provides a structured description intended primarily for AI agents.

Typical sections include:

- capability identifier;
- supported operations;
- required inputs;
- produced outputs;
- supported contract versions;
- confidence characteristics;
- dependencies;
- security requirements.

Example:

```
Skill:
  BibleSearch

Version:
  2.1

Supports:
  SearchVerse
```

```
SearchTopic
SearchPerson
```

```
Contracts:
```

```
QueryRequest
VerseCollection
CitationList
```

```
Confidence:
```

```
High
```

SKILL.md SHOULD be sufficiently structured to support automated discovery and composition.

D.7 AGENTS.md

AGENTS.md provides implementation guidance specifically for AI coding assistants.

Typical guidance includes:

- architectural conventions;
- coding standards;
- repository structure;
- testing expectations;
- review procedures;
- documentation requirements;
- prohibited modifications;
- preferred design patterns.

Unlike README.md, AGENTS.md is written primarily for autonomous software agents rather than human readers.

D.8 Testing Requirements

Every capability SHALL include automated tests.

Minimum expectations include:

- unit tests;

- interface tests;
- contract validation;
- regression tests.

Integration tests SHOULD be provided when external services are involved.

Test coverage SHOULD be documented.

D.9 Evaluation Requirements

Traditional software verifies correctness.

Agent Skills must additionally evaluate reasoning quality.

Evaluation SHALL address, where applicable:

- accuracy;
- completeness;
- robustness;
- reproducibility;
- latency;
- resource consumption;
- citation quality;
- hallucination resistance.

Benchmark datasets SHOULD be identified whenever available.

D.10 Governance Requirements

Each capability SHALL declare applicable governance information.

Examples include:

- usage policies;
- access restrictions;

- human approval requirements;
- safety considerations;
- audit requirements;
- privacy considerations;
- regulatory compliance.

Governance documentation SHALL remain synchronized with capability evolution.

D.11 Versioning Requirements

Every capability SHALL maintain semantic version information.

The following categories SHOULD be distinguished:

- major version;
- minor version;
- patch version.

Breaking contract changes SHALL increment the major version.

Deprecated capabilities SHOULD specify migration guidance.

D.12 Maintainer Requirements

Every capability SHALL identify responsible maintainers.

Maintainer information SHOULD include:

- owner;
- organization;
- repository;
- contact method;
- review authority.

Inactive capabilities SHOULD be clearly identified.

D.13 Engineering Contract Checklist

A capability SHALL satisfy the following checklist before acceptance into the CAIF ecosystem.

Requirement	Status
README.md	✓
SPECIFICATION.md	✓
SKILL.md	✓
AGENTS.md	✓
LICENSE	✓
Automated Tests	✓
Evaluation Report	✓
Governance Documentation	✓
Version Information	✓
Maintainer Identified	✓

Capabilities that do not satisfy all mandatory requirements SHALL NOT be designated as CAIF-compliant.

D.14 Engineering Maturity Levels

CAIF defines four levels of engineering maturity.

Level	Description
EC0	Experimental capability intended for local development only
EC1	Documented capability with required engineering artifacts

Level	Description
EC2	Verified capability with automated tests, evaluation, and governance
EC3	Production-ready capability with active maintenance, semantic versioning, continuous integration, and lifecycle management

Organizations may adopt higher maturity levels as operational requirements increase.

D.15 Conformance Requirements

To claim compliance with the CAIF Engineering Contract specification, a capability SHALL:

- provide all mandatory engineering artifacts;
- implement the published capability contracts;
- include automated verification;
- document evaluation methodology and results;
- declare governance requirements;
- identify responsible maintainers;
- maintain semantic version information;
- remain consistent with the Reference Architecture (Appendix A), Capability Specifications (Appendix B), and Cognitive Collaboration Specifications (Appendix C).

Conforming implementations MAY extend the Engineering Contract with additional organizational requirements, provided that such extensions do not weaken or replace the mandatory requirements defined in this appendix.

Relationship to the Previous Appendices

With this appendix, the four appendices form a coherent progression:

Appendix	Focus	Question Answered
Appendix A – Reference Architecture	System structure	<i>Where do the components fit?</i>
Appendix B – Capability Specifications	Functional behavior	<i>What can each capability do?</i>
Appendix C – Cognitive Collaboration Specifications	Multi-agent cognition	<i>How do multiple capabilities think and reason together?</i>
Appendix D – Engineering Contracts	Engineering quality and lifecycle	<i>What standards must a capability satisfy before joining the ecosystem?</i>

Together, these appendices define CAIF from four complementary perspectives: architecture, capability, cognition, and engineering discipline. This engineering-centric approach distinguishes CAIF from most current agent frameworks, which primarily standardize execution and communication but leave engineering governance and quality assurance to individual projects.

Appendix E. Capability Maturity Model

This appendix very **practical** because it gives every contributor a roadmap.

Existing maturity models (such as **CMMI**, **Technology Readiness Level (TRL)**, or **OpenSSF Scorecard**) measure software quality, project maturity, or deployment readiness. They were not designed for the emerging world of **Agent Skills**.

CAIF's Capability Maturity Model (CMM) measures **how an idea evolves into a trusted capability that can serve as shared infrastructure for an entire ecosystem**.

E.1 Purpose

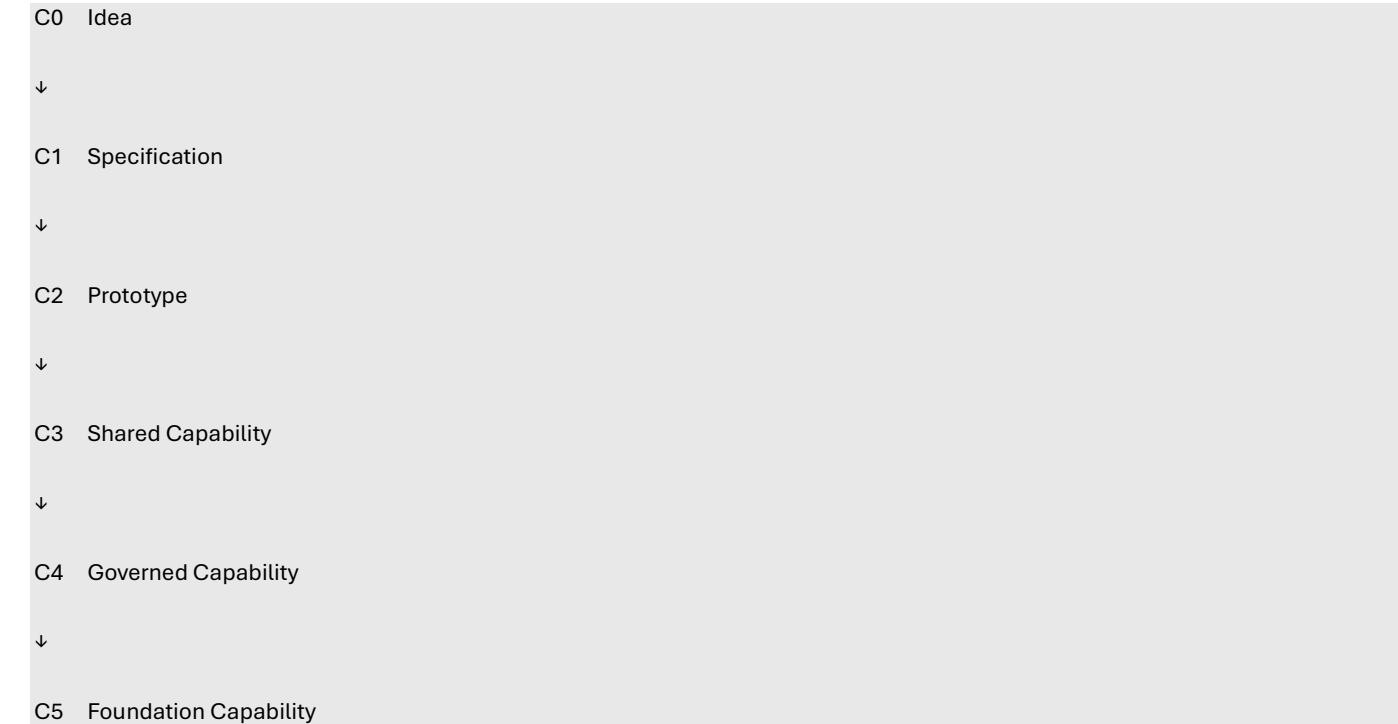
The CAIF Capability Maturity Model (CMM) defines a standardized framework for evaluating the engineering maturity of Agent Skills and shared capabilities.

Rather than measuring implementation complexity, the model measures the degree to which a capability has progressed from an initial concept into a reusable, governable, and foundational component suitable for ecosystem-wide adoption.

The Capability Maturity Model assists contributors, reviewers, maintainers, and organizations in determining the readiness of capabilities for development, sharing, production deployment, and long-term maintenance.

E.2 Maturity Levels

CAIF defines six maturity levels.



Each level builds upon the previous level.

E.3 C0 – Idea

Objective

Capture promising ideas before implementation begins.

Ideas at this level are exploratory and are intended to stimulate discussion and experimentation.

Typical artifacts include:

- problem statement;
- motivation;
- expected value;
- high-level architecture;
- preliminary use cases.

Implementation is optional.

Exit Criteria

A capability advances to C1 when:

- objectives are clearly defined;
 - scope is understood;
 - stakeholders agree on the problem being solved.
-

E.4 C1 – Specification

Objective

Transform an idea into a well-defined engineering specification.

At this level, behavior is described independently of implementation.

Required artifacts include:

- functional specification;
- capability definition;
- engineering contracts;
- input/output contracts;
- success criteria.

No production implementation is required.

Deliverables

Typical documents include:

```
README.md
SPECIFICATION.md
SKILL.md
Engineering Contracts
Capability Definitions
```

Exit Criteria

The capability advances to C2 when:

- specifications are complete;
 - interfaces are stable;
 - engineering review has been completed.
-

E.5 C2 – Prototype

Objective

Demonstrate technical feasibility.

A prototype validates that the specification can be implemented successfully.

Prototype implementations are expected to evolve rapidly and may not yet satisfy production-quality requirements.

Requirements

Prototype capabilities should include:

- working implementation;
- unit tests;
- example workflows;

- evaluation results;
- basic documentation.

Performance optimization is optional.

Exit Criteria

Advancement to C3 requires:

- functional completeness;
 - successful demonstration;
 - repeatable evaluation;
 - basic interoperability.
-

E.6 C3 – Shared Capability

Objective

Enable reuse across multiple projects or organizations.

The capability becomes a reusable building block rather than an application-specific component.

Requirements

Shared capabilities SHALL provide:

- published contracts;
- semantic versioning;
- documentation;
- automated testing;
- interoperability validation;
- dependency management.

The capability should be discoverable through the CAIF Skill Registry.

Typical Examples

Examples include:

- Bible Search Skill;
 - OCR Skill;
 - Embedding Service;
 - Church Archive Skill;
 - Translation Skill;
 - Citation Generator.
-

Exit Criteria

Advancement to C4 requires:

- demonstrated reuse;
 - governance documentation;
 - production deployment experience;
 - stable maintenance process.
-

E.7 C4 – Governed Capability

Objective

Ensure trustworthy operation within production environments.

Governed capabilities satisfy organizational requirements for quality, explainability, compliance, and lifecycle management.

Requirements

Governed capabilities SHALL include:

- governance policies;
- audit logging;
- explainability;
- security review;
- privacy review;
- human approval workflows where appropriate;
- continuous evaluation.

Operational metrics SHOULD be collected continuously.

Governance Characteristics

Examples include:

- policy enforcement;
 - access control;
 - traceability;
 - provenance;
 - lifecycle management;
 - compliance reporting.
-

Exit Criteria

Advancement to C5 requires:

- ecosystem-wide adoption;
 - long-term maintenance;
 - demonstrated stability;
 - community governance.
-

E.8 C5 – Foundation Capability

Objective

Provide stable infrastructure upon which other capabilities depend.

Foundation capabilities become core components of the CAIF ecosystem and are expected to remain backward compatible across multiple generations.

Characteristics

Foundation capabilities typically exhibit:

- extensive production usage;
- mature governance;
- active community maintenance;
- long-term compatibility;
- broad interoperability;
- comprehensive documentation;
- reference implementations.

Changes are carefully managed to preserve ecosystem stability.

Typical Examples

Potential future examples include:

- Contract Framework;
- Shared Memory Service;
- Shared Ontology;
- Governance Engine;
- Capability Registry;
- Identity Service.

E.9 Maturity Progression

Level	Primary Goal	Main Deliverable
C0	Innovation	Concept
C1	Engineering Definition	Specification
C2	Feasibility	Prototype
C3	Reuse	Shared Capability
C4	Trustworthiness	Governed Capability
C5	Ecosystem Infrastructure	Foundation Capability

E.10 Assessment Criteria

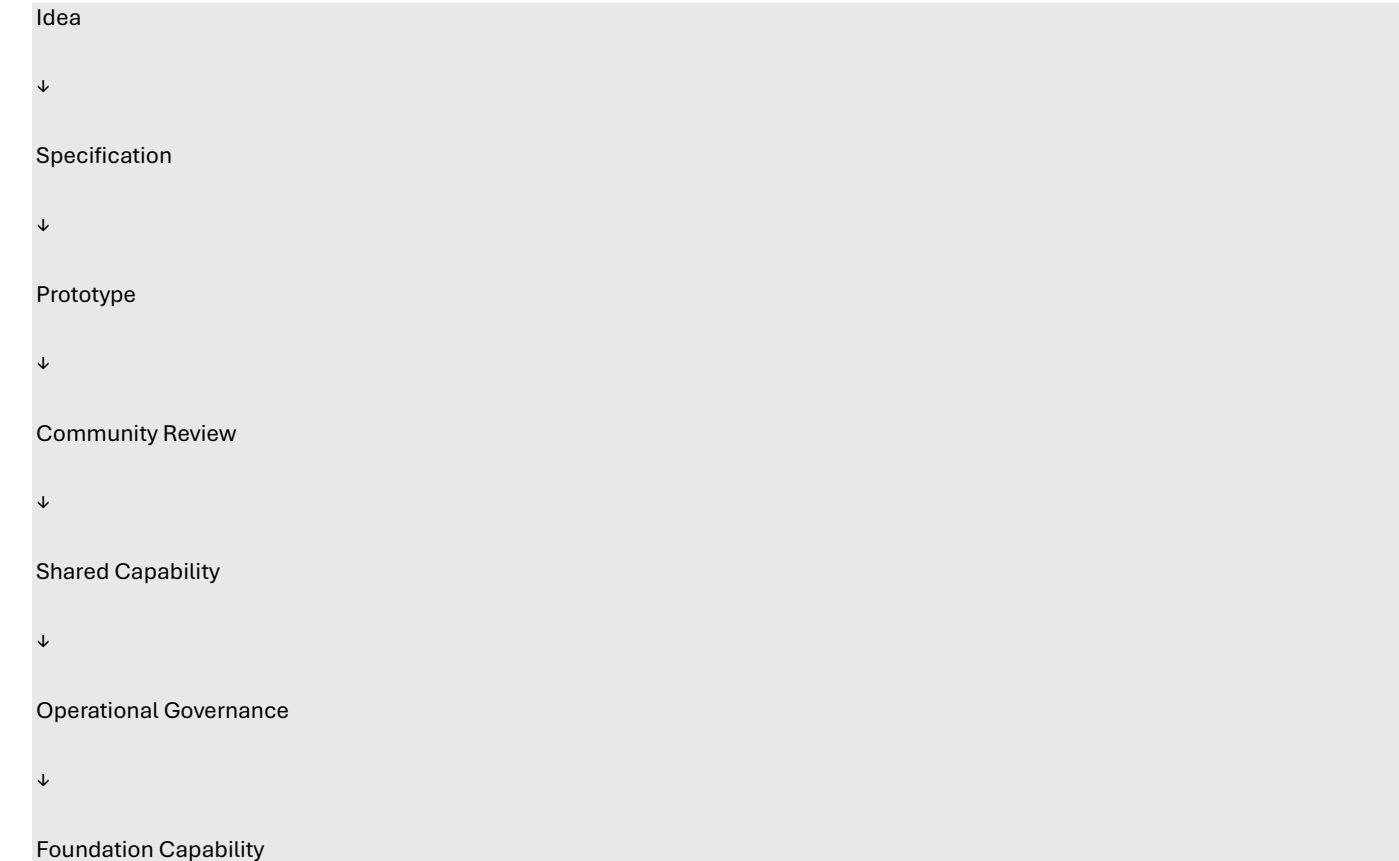
Capability maturity is evaluated across multiple engineering dimensions.

Dimension	C0	C1	C2	C3	C4	C5
Problem Definition	✓	✓	✓	✓	✓	✓
Specification		✓	✓	✓	✓	✓
Implementation			✓	✓	✓	✓
Automated Tests			✓	✓	✓	✓
Documentation		✓	✓	✓	✓	✓
Reusability				✓	✓	✓
Governance					✓	✓
Auditability					✓	✓

Dimension	C0	C1	C2	C3	C4	C5
Community Maintenance						✓
Backward Compatibility						✓

E.11 Lifecycle

Capability evolution is expected to follow an incremental lifecycle.



Not every capability is expected to reach C5. Many application-specific skills will remain at C2 or C3 while still providing significant value.

E.12 Certification

Organizations MAY certify capabilities according to the CAIF Capability Maturity Model.

Certification SHOULD evaluate:

- engineering artifacts;

- implementation quality;
- testing;
- governance;
- interoperability;
- operational experience.

Certification results SHOULD identify the achieved maturity level together with recommendations for advancement.

E.13 Relationship to Other Standards

The CAIF Capability Maturity Model complements rather than replaces existing engineering maturity frameworks.

Framework	Primary Focus	Relationship to CAIF CMM
CMMI	Software process maturity	CAIF focuses on individual Agent Skills and capabilities rather than organizational processes.
Technology Readiness Levels (TRL)	Technology readiness	CAIF evaluates engineering, interoperability, governance, and ecosystem readiness in addition to technical feasibility.
OpenSSF Scorecard	Open-source security practices	CAIF incorporates engineering and governance concerns specific to AI capabilities while remaining compatible with secure software development practices.
AI Model Benchmarks	Model performance	CAIF measures the maturity of the capability surrounding the model, including documentation, contracts, lifecycle, and governance.

E.14 Conformance Requirements

To claim compliance with the CAIF Capability Maturity Model, a capability SHALL:

- satisfy all requirements of its declared maturity level;
- continue to satisfy the requirements of all preceding levels;
- document objective evidence supporting its maturity assessment;
- maintain consistency with the Engineering Contracts defined in Appendix D.

Capabilities MAY voluntarily target higher maturity levels over time as engineering quality, governance, reuse, and community adoption increase.

Why this appendix is strategically important

Among all five appendices, this one provides the **evolutionary roadmap** for the ecosystem:

- **Appendix A** defines the architecture (**where** capabilities fit).
- **Appendix B** defines capability behavior (**what** they do).
- **Appendix C** defines collaborative cognition (**how** they think together).
- **Appendix D** defines engineering discipline (**how** they are built and maintained).
- **Appendix E** defines ecosystem evolution (**how** they mature over time).

Taken together, these appendices move beyond describing an AI framework; they define a complete engineering methodology for developing, governing, and evolving Agent Skills into long-lived, reusable infrastructure for the CAIF ecosystem.

Appendix F. Canonical Biblical AI

(Jason's Bible Dictionary can call these capabilities, but ADK itself will never define them. Also, this appendix is for Charles's **Biblical AI**, because it clearly explains **what CAIF contributes that Google ADK, Microsoft Agent Framework, LangGraph, CrewAI, or any other general-purpose framework will never attempt to standardize.**)

This appendix answers an important architectural question:

Why does CAIF exist if Google ADK already provides agent orchestration?

The answer is simple:

CAIF does not compete with ADK.

CAIF builds canonical Biblical intelligence on top of ADK.

Google ADK provides the *operating system* for agent collaboration. CAIF provides the *domain semantics* and *canonical theological knowledge* required for Biblical AI. That distinction should be made explicit in the SDD.

F.1 Purpose

General-purpose agent frameworks provide mechanisms for orchestration, communication, workflow management, and capability execution. They intentionally remain independent of any specific application domain.

Biblical AI presents a fundamentally different challenge. Correct interpretation of Scripture depends not only on language processing, but also on established principles of biblical interpretation, theological consistency, historical context, and ecclesiastical governance.

These principles cannot be inferred solely from large language models or standardized by general-purpose agent frameworks.

This appendix defines the canonical Biblical AI capabilities contributed by CAIF. These capabilities provide domain-specific theological intelligence while remaining independent of the underlying orchestration framework.

Accordingly, CAIF complements frameworks such as Google ADK rather than competing with them.

F.2 Scope

The Canonical Biblical AI layer provides theological capabilities that operate above the general capability layer defined in Appendix B.

These capabilities support:

- biblical interpretation;
- theological reasoning;
- doctrinal consistency;

- church governance;
- ministry applications;
- educational applications;
- research.

The appendix does not prescribe denominational doctrine. Instead, it defines the engineering framework within which doctrinal positions can be represented, evaluated, and governed.

F.3 Canonical Hermeneutics

Purpose

Canonical Hermeneutics interprets every biblical passage within the context of the entire canon of Scripture rather than in isolation.

Individual verses are understood as components of a unified biblical revelation.

Principles

Canonical interpretation considers:

- immediate literary context;
- book context;
- covenant context;
- canonical context;
- historical context;
- theological context.

No interpretation should contradict the broader witness of Scripture without explicit justification.

Representative Capability

```
InterpretCanon()
```

```
ResolveCanonicalContext()
IdentifyCanonicalThemes()
EvaluateCanonicalConsistency()
```

F.4 Cross-Book Interpretation

Purpose

Many biblical doctrines emerge only when multiple books are interpreted together.

Cross-book interpretation identifies relationships among passages throughout Scripture.

Examples

Representative relationships include:

- Genesis → Romans
- Exodus → Hebrews
- Isaiah → Matthew
- Psalms → Revelation
- Daniel → Revelation

Cross-book interpretation supports:

- thematic analysis;
 - doctrinal synthesis;
 - historical continuity;
 - covenant relationships.
-

Representative Capability

```
FindParallelPassages()
CompareBooks()
BuildThematicChain()
```

F.5 Progressive Revelation

Purpose

Biblical revelation unfolds progressively throughout history.

Earlier revelation frequently introduces principles that later revelation expands, clarifies, or fulfills.

Interpretation should therefore consider the historical progression of revelation.

Representative Capability

TraceDoctrine()

LocateFirstMention()

BuildRevelationTimeline()

IdentifyDoctrinalDevelopment()

Example

Illustrative developments include:

- sacrifice;
- covenant;
- priesthood;
- kingdom;
- Messiah;
- salvation.

These themes develop progressively across both Testaments.

F.6 Typology

Purpose

Biblical typology recognizes divinely intended correspondences between earlier biblical persons, institutions, events, and their later fulfillment.

Typology differs from allegory by emphasizing historically grounded patterns within Scripture.

Representative Capability

```
IdentifyType()
IdentifyAntitype()
EvaluateTypologicalStrength()
GenerateTypologyMap()
```

Examples

Representative examples include:

- Adam → Christ
 - Passover Lamb → Christ
 - Tabernacle → Christ
 - Jonah → Resurrection
 - Bronze Serpent → Crucifixion
-

F.7 Prophecy Fulfillment

Purpose

Prophecy fulfillment connects prophetic passages with historical or future fulfillment.

The objective is not merely to locate similar language but to evaluate canonical relationships.

Representative Capability

```
LocateFulfillment()
```

```
CompareProphecies()

EvaluateMessianicProphecy()

GenerateFulfillmentReport()
```

Evidence

Evaluation may consider:

- direct quotation;
- thematic correspondence;
- historical fulfillment;
- partial fulfillment;
- future fulfillment.

Confidence should be explicitly reported.

F.8 Scripture Interprets Scripture

Purpose

Scripture possesses an internal interpretive structure.

Many passages are clarified by other passages rather than by external authorities alone.

This principle has long been recognized within Christian hermeneutics.

Representative Capability

```
FindInterpretivePassages()

ResolveAmbiguity()

CompareParallelAccounts()

GenerateSupportingEvidence()
```

Interpretation Strategy

Supporting passages should be ranked according to:

- canonical relevance;
 - contextual similarity;
 - theological consistency;
 - historical relationship.
-

F.9 Church Doctrinal Governance

Purpose

Biblical AI should assist churches without replacing ecclesiastical authority.

Final doctrinal responsibility remains with qualified human leadership.

Governance Objectives

Church doctrinal governance supports:

- doctrinal consistency;
 - transparency;
 - denominational customization;
 - auditability;
 - human oversight.
-

Representative Capability

```
EvaluateDoctrine()
```

```
IdentifyDoctrinalDifferences()
```

```
GenerateGovernanceReport()
```

```
RequestHumanReview()
```

Human Authority

Biblical AI SHALL function as a decision-support system.

It SHALL NOT claim independent doctrinal authority.

Human reviewers remain responsible for:

- doctrinal judgments;
- pastoral decisions;
- church policy;
- official teaching.

F.10 Canonical Knowledge Graph

Canonical Biblical AI SHOULD maintain structured theological knowledge.

Representative entities include:

Book

Chapter

Verse

Person

Place

Event

Doctrine

Covenant

Prophecy

Typology

Cross Reference

Theme

Relationships among these entities enable richer reasoning than isolated text retrieval.

F.11 Canonical Confidence

Interpretation is rarely binary.

Canonical Biblical AI SHALL explicitly distinguish among:

- explicit biblical statements;
- strong canonical inference;
- probable interpretation;
- speculative interpretation.

Confidence SHALL accompany every generated interpretation.

F.12 Denominational Extension

The Canonical Biblical AI specification intentionally separates:

- canonical biblical interpretation;
- denominational interpretation;
- local church policy.

Organizations MAY extend the framework with additional doctrinal modules.

Examples include:

- Baptist;
- Presbyterian;
- Lutheran;
- Methodist;
- Evangelical;
- Independent Bible Church.

Extensions SHOULD preserve interoperability with the canonical core.

F.13 Relationship to General Agent Frameworks

General-purpose frameworks provide:

- orchestration;
- workflow execution;
- tool invocation;
- capability discovery;
- memory management;
- communication protocols.

They intentionally avoid prescribing domain knowledge.

CAIF contributes domain-specific theological intelligence.

Examples include:

- canonical hermeneutics;
- progressive revelation;
- typology;
- prophecy fulfillment;
- cross-book interpretation;
- doctrinal governance.

Accordingly, CAIF is designed to operate *on top of* frameworks such as Google ADK rather than replacing them.

F.14 Conformance Requirements

To claim compliance with the Canonical Biblical AI specification, an implementation SHALL:

- support canonical hermeneutics;
- provide cross-book interpretation capabilities;
- recognize progressive revelation;
- represent typological relationships where appropriate;

- evaluate prophecy fulfillment with explicit evidence and confidence;
- apply the principle of Scripture interpreting Scripture;
- support church doctrinal governance through transparent, auditable, human-supervised workflows;
- distinguish clearly between biblical text, interpretive inference, denominational position, and speculative opinion.

Implementations MAY add further domain-specific capabilities, provided they remain consistent with the architectural, capability, cognitive, and engineering principles defined in the preceding appendices.

F.15 Why Canonical Biblical AI Belongs in CAIF

The architectural relationship can be summarized as follows:

Layer	Responsibility	Example
Google ADK	Agent orchestration and workflow	Selects agents, routes requests, manages execution
Agent Skills	General reusable capabilities	Bible Dictionary, Search, Translation, OCR
CAIF Canonical Biblical AI	Biblical semantics and theological reasoning	Canonical Hermeneutics, Progressive Revelation, Typology, Prophecy Fulfillment, Scripture Interprets Scripture
Church Applications	Ministry-specific solutions	Sermon preparation, discipleship, Bible study, theological education, church governance

This layered architecture highlights the complementary roles of general-purpose AI infrastructure and domain-specific intelligence. Google ADK provides the orchestration framework that can execute any type of agent workflow. CAIF contributes the canonical theological knowledge, interpretive principles, and governance mechanisms that are unique to Biblical AI. By separating orchestration from domain semantics, the architecture allows CAIF to benefit from future advances in agent frameworks while preserving a stable, extensible foundation for biblical interpretation and church applications.

Appendix G. Internet of Cognition : Research Agenda and Cognitive Commons

(Informative Appendix)

Appendix G is very differently from the previous appendices in style: Appendices **A–F** are **normative** ("SHALL", "SHOULD", "MAY"). They define the CAIF specification. However, Appendix **G** is **informative**, much like an academic paper or W3C Note. It is not part of the conformance requirements. Instead, it explains the long-term research vision that motivates CAIF.

Let us explicitly state at the beginning:

This appendix is informative rather than normative. It describes possible future directions for AI civilization and is intended to stimulate research rather than define implementation requirements.

That distinction is important because many of these concepts (especially Theory of Mind and Collective Cognition) are still active research topics.

G.1 Purpose

The Internet transformed isolated computers into interconnected information systems.

The World Wide Web transformed interconnected computers into a globally accessible knowledge network.

The emerging generation of AI agents suggests a further transformation: the interconnection of autonomous reasoning systems into collaborative cognitive networks.

This appendix introduces the concept of the **Internet of Cognition (IoC)** as a theoretical framework for understanding this possible evolution.

Unlike the preceding appendices, this appendix is **informative rather than normative**. It does not define implementation requirements or conformance criteria. Instead, it identifies research

directions that may influence future generations of CAIF and other multi-agent ecosystems.

G.2 From Internet to Internet of Cognition

The history of computing can be viewed as an expansion of what is shared across networks.

Era	Primary Shared Resource
Internet	Connectivity
World Wide Web	Information
Cloud Computing	Computing Resources
Agentic AI	Capabilities
Internet of Cognition	Shared Cognition

The Internet of Cognition extends this progression by exploring how multiple autonomous reasoning systems may cooperate through shared cognitive structures.

G.3 Shared Memory

One possible foundation of collective cognition is Shared Memory.

Unlike traditional databases, Shared Memory represents accumulated experience that can be reused by multiple reasoning agents.

Research questions include:

- How should knowledge be retained over time?
- What distinguishes temporary reasoning from long-term memory?
- How should provenance be preserved?
- How should conflicting memories be reconciled?
- How can privacy be maintained while enabling collaboration?

Shared Memory may include:

- episodic memory;
 - semantic memory;
 - procedural memory;
 - working memory.
-

G.4 Shared Ontology

Communication alone is insufficient if collaborating agents interpret concepts differently.

A Shared Ontology provides common semantic foundations.

Research topics include:

- ontology evolution;
- semantic alignment;
- automatic ontology generation;
- cross-domain integration;
- multilingual semantic representation.

Future systems may continuously refine shared ontologies through collaborative learning.

G.5 Structured Intent

Current software exchanges parameters.

Future cognitive systems may exchange intentions.

Structured Intent represents the objective, constraints, priorities, and expected outcomes associated with a task.

Potential research questions include:

- intent representation;
- intent negotiation;
- hierarchical goals;

- conflicting objectives;
- long-term planning.

Structured Intent may become the cognitive equivalent of application programming interfaces.

G.6 Theory of Mind

Human collaboration depends upon understanding what others know, believe, intend, and require.

An analogous capability may become increasingly important for autonomous AI systems.

Theory of Mind research investigates how agents may model:

- capabilities;
- expertise;
- uncertainty;
- confidence;
- objectives;
- resource limitations.

Possible research topics include:

- dynamic trust modeling;
- expertise discovery;
- collaborative planning;
- adaptive delegation.

Theory of Mind remains an active area of AI and cognitive science research.

G.7 Negotiation

Independent reasoning agents will frequently possess different evidence, priorities, or assumptions.

Negotiation provides mechanisms for resolving these differences.

Research directions include:

- evidence-based negotiation;
- confidence-weighted negotiation;
- policy-aware negotiation;
- human-mediated negotiation;
- decentralized consensus.

Future negotiation mechanisms may resemble scientific collaboration more than traditional distributed computing.

G.8 Collective Cognition

Collective Cognition describes situations in which groups of specialized reasoning agents solve problems more effectively than isolated agents.

Potential properties include:

- distributed expertise;
- parallel reasoning;
- complementary knowledge;
- collaborative hypothesis generation;
- iterative refinement.

Research questions include:

- optimal team composition;
- reasoning diversity;
- conflict resolution;
- emergent intelligence;
- scalability.

Collective Cognition differs from simple workflow orchestration because the participating agents continuously influence one another's reasoning processes.

G.9 Cognitive Capital

Industrial economies accumulated physical capital.

Knowledge economies accumulated intellectual capital.

An Internet of Cognition may introduce a new category: **Cognitive Capital**.

Cognitive Capital represents reusable cognitive assets that increase the problem-solving capacity of an ecosystem.

Examples may include:

- validated reasoning patterns;
- trusted knowledge repositories;
- shared ontologies;
- verified workflows;
- collective experience;
- reusable decision frameworks.

Unlike traditional software assets, Cognitive Capital grows through repeated collaborative use.

G.10 Cognitive Ecosystems

Future AI systems may evolve from isolated applications into interconnected cognitive ecosystems.

Representative characteristics include:

- continuous learning;
- shared experience;
- distributed specialization;
- cooperative reasoning;
- long-term memory;
- adaptive governance.

Such ecosystems may resemble scientific communities more closely than conventional software

systems.

G.11 Internet of Cognition Architecture

The Internet of Cognition can be viewed conceptually as a layered architecture.



Each layer builds upon the capabilities of the layer below while introducing new forms of collaboration and abstraction.

G.12 Research Challenges

Numerous open questions remain.

Representative challenges include:

Scalability

How can thousands or millions of collaborating agents maintain coherent reasoning?

Trust

How can unreliable reasoning be detected?

How should trust evolve over time?

Governance

Who governs shared cognitive assets?

How are disagreements resolved?

Explainability

How can collective reasoning remain understandable to humans?

Privacy

How should sensitive knowledge be protected while enabling collaboration?

Interoperability

How can independently developed cognitive systems cooperate without sacrificing autonomy?

G.13 Relationship to CAIF

The Internet of Cognition is not a required component of the current CAIF specification.

Instead, it provides a conceptual framework that motivates several design decisions within CAIF, including:

- Engineering Contracts (Appendix D);
- Cognitive Collaboration Specifications (Appendix C);
- Canonical Biblical AI (Appendix F);
- Capability Maturity Model (Appendix E).

These appendices establish practical engineering mechanisms that may support future cognitive ecosystems.

G.14 Future Research Directions

The concepts presented in this appendix are expected to evolve as AI research advances.

Representative areas for future investigation include:

- distributed reasoning architectures;
- federated cognitive memory;
- adaptive ontology evolution;
- machine-assisted scientific discovery;
- collaborative theorem proving;
- human–AI collective intelligence;
- cognitive governance frameworks;
- economics of cognitive capital.

Future versions of CAIF may incorporate selected research outcomes after they have matured into stable engineering practices.

G.15 Perspective

The Internet connected computers.

The Web connected information.

Cloud computing connected computational resources.

Agent frameworks connect executable capabilities.

The **Internet of Cognition** envisions a future in which autonomous reasoning systems are able to share understanding, coordinate intentions, negotiate meaning, accumulate cognitive capital, and solve problems collectively.

Whether this vision ultimately emerges as a distinct technological paradigm or evolves through existing agent ecosystems remains an open question. Nevertheless, the concepts outlined in this appendix provide a research agenda for exploring how AI systems may progress from isolated

intelligence toward collaborative cognition.

G.16 Cognitive Commons

Cognitive commons extends our long-standing idea of open-source ecosystems. Just as the Internet benefited from open protocols and the World Wide Web from openly shared standards, an Internet of Cognition may depend on a **Cognitive Commons**—shared repositories of contracts, ontologies, reasoning patterns, evaluation benchmarks, governance policies, and trusted cognitive assets maintained by the community. These resources would form the public infrastructure upon which cognitive capital accumulates.

Cognitive Commons connect the theoretical vision of the Internet of Cognition with the practical engineering philosophy of CAIF, reinforcing our broader theme that sustainable AI ecosystems are built not only on models and compute, but on openly shared cognitive infrastructure.

Definition

The **Cognitive Commons** is a shared ecosystem of reusable cognitive assets that can be discovered, evaluated, composed, governed, and continuously improved by both humans and AI agents.

Unlike traditional software libraries, which primarily share executable code, a Cognitive Commons shares the knowledge structures that enable intelligent reasoning, collaboration, and decision making.

Its purpose is to accelerate collective intelligence by allowing communities to build upon previously validated cognitive assets rather than repeatedly recreating them.

Why "Commons"?

The word *commons* is intentionally chosen.

Examples include:

Natural Commons

- air
- oceans
- forests

Knowledge Commons

- Wikipedia
- OpenStreetMap
- Project Gutenberg

Software Commons

- Linux
- Apache
- Kubernetes

A **Cognitive Commons** extends these ideas from shared information to **shared cognition**.

Layers of Cognitive Commons

Imagine it as a pyramid.



Notice that **code is only the bottom layer**.

Components

There are at least eight repositories.

1. Contract Commons

Instead of everyone inventing their own JSON schema, the community shares

InterpretationResult

Citation

Person

Organization

PrayerRequest

BibleReference

SkillDescriptor

CapabilityDescriptor

EvaluationReport

Every skill immediately becomes interoperable.

Today:
Agent A
↓
invents schema
Tomorrow:
Agent A
↓
uses Contract Commons
↓
everyone understands it

This is exactly what we discussed in Appendix B.

2. Ontology Commons

Shared concepts.

Example

Church

↓

Pastor

↓
Elder
↓
Deacon
↓
Volunteer

or

Scripture
↓
Book
↓
Chapter
↓
Verse

Instead of every church defining these differently, everyone references Church Ontology v3.2. This is much like

SNOMED

FHIR

schema.org

except for Church AI.

3. Reasoning Commons

Instead of sharing code, people share reasoning. Examples:

Canonical Hermeneutics
Historical-Grammatical Analysis
Typology
Progressive Revelation

Scripture interprets Scripture

or Healthcare

Differential Diagnosis

Evidence Ranking

Treatment Recommendation

or Engineering

Root Cause Analysis

Failure Tree

Risk Assessment

Notice these are NOT prompts. They are **reasoning strategies**.

4. Memory Commons

Shared long-term knowledge. Examples:

Church minutes

Bible dictionary

Mission history

Sermons

Commentaries

Academic papers

Digital twins

Case studies

Every new capability can learn from the accumulated memory.

5. Evaluation Commons

One of the biggest missing pieces today: Everybody builds LLMs, but very few agree on how to evaluate them. Shared benchmark collections use the following examples:

Bible QA

Hermeneutics

Translation

Doctrinal Consistency

Hallucination Detection

Citation Accuracy

Healthcare

MedBench
MedSafetyBench
Clinical QA

Engineering

Root Cause Analysis
Safety Compliance

Think of it as "ImageNet for Agent Skills."

6. Governance Commons

Instead of every organization inventing policies, they share

Privacy Policy
Security Policy
Theological Policy
Review Workflow
Human Approval Rules
Risk Classification

Example

Publish Sermon
↓
Pastor Review Required

Every church reuses the governance policy.

7. Capability Commons

This is basically Appendix D: GitHub plus Engineering Contracts. Every capability contributes

README
SPECIFICATION
SKILL
AGENTS
Evaluation
Tests

Governance
Version
Maintainer

8. Cognitive Asset Commons

Instead of sharing software, we share validated thinking. Examples:

Decision Trees
Planning Templates
Negotiation Strategies
Inference Graphs
Canonical Interpretation Maps
Domain Knowledge Maps
Trust Models
Evidence Ranking Models

These are reusable thinking assets.

Relationship

Cognitive Commons

Contract Commons

Ontology Commons

Memory Commons

Reasoning Commons

Evaluation Commons

Governance Commons

Capability Commons

Cognitive Asset Commons

Every repository evolves independently.

Why This Is Bigger Than GitHub

GitHub stores software, but Cognitive Commons stores

knowledge

reasoning
governance
evaluation
memory
semantics

GitHub answers

How do I run it?

Cognitive Commons answers

How do I think with it?

Why This Is Bigger Than RAG

RAG retrieves documents. Cognitive Commons retrieves

knowledge,
reasoning,
contracts,
ontology,
evaluation,
governance,
memory,
and capabilities.

Why This Is Bigger Than MCP

MCP standardizes tool invocation. Cognitive Commons standardizes shared intelligence.

Relationship with Agent Economy

This is where our earlier work of agent economy¹ becomes particularly relevant.

Our original vision was that the most valuable assets in an AI-driven economy would gradually shift away from raw compute and toward reusable intelligence. The Cognitive Commons provides the

¹ This refers to the work of the book titled “**The Agent Economy – From Cryptocurrency to AI Civilization**” by Charles Tang, July 2026, Quantum Intelligence Association.

infrastructure for that shift.

In this framework, value creation evolves through distinct stages:

Economy	Primary Asset	Representative Infrastructure
Industrial Economy	Physical capital	Factories, machines
Information Economy	Information	Internet, databases
Software Economy	Software	GitHub, package registries
Agent Economy	Executable capabilities	Agent Skills, ADK, orchestration frameworks
Internet of Cognition	Cognitive capital	Cognitive Commons

This suggests that **Cognitive Capital** is not merely "knowledge." It is **validated, reusable intelligence** that others can discover, compose, evaluate, and build upon.

One More Step

Let us define Cognitive Commons mathematically.

Cognitive Commons

=

Shared Memory

+

Shared Ontology

+

Shared Reasoning

+

Shared Governance

+

Shared Evaluation

+

Shared Contracts

+

Shared Capabilities

+

Shared Trust

Notice one important difference from today's AI systems: Most current systems share **models**. The Cognitive Commons shares **thinking**.

That distinction aligns closely with the architectural direction we've been developing across CAIF, the Internet of Cognition, and the Agent Economy: durable progress comes not only from larger models, but from openly shared, governed, and reusable cognitive assets that allow intelligence itself to accumulate over time.

Appendix H. Technology Mapping

(Informative Appendix)

This appendix prevents one of the biggest misunderstandings readers may have:

"Is CAIF another agent framework competing with Google ADK?"

The answer is **No**.

CAIF is **an architectural framework** that integrates and extends existing technologies. It is intentionally designed to be technology-independent while leveraging best-of-breed components from the open AI ecosystem. In fact, this appendix positions CAIF as **an integration architecture**, not another runtime.

H.1 Purpose

The CAIF architecture is intentionally technology independent.

The specifications defined in this document describe architectural roles rather than prescribing specific products or implementations.

This appendix illustrates how current open-source technologies and commercial platforms may be mapped onto the CAIF Reference Architecture.

The technologies presented here are representative examples rather than mandatory implementation choices. Equivalent technologies may be substituted provided they satisfy the corresponding architectural responsibilities.

H.2 Technology Mapping Overview

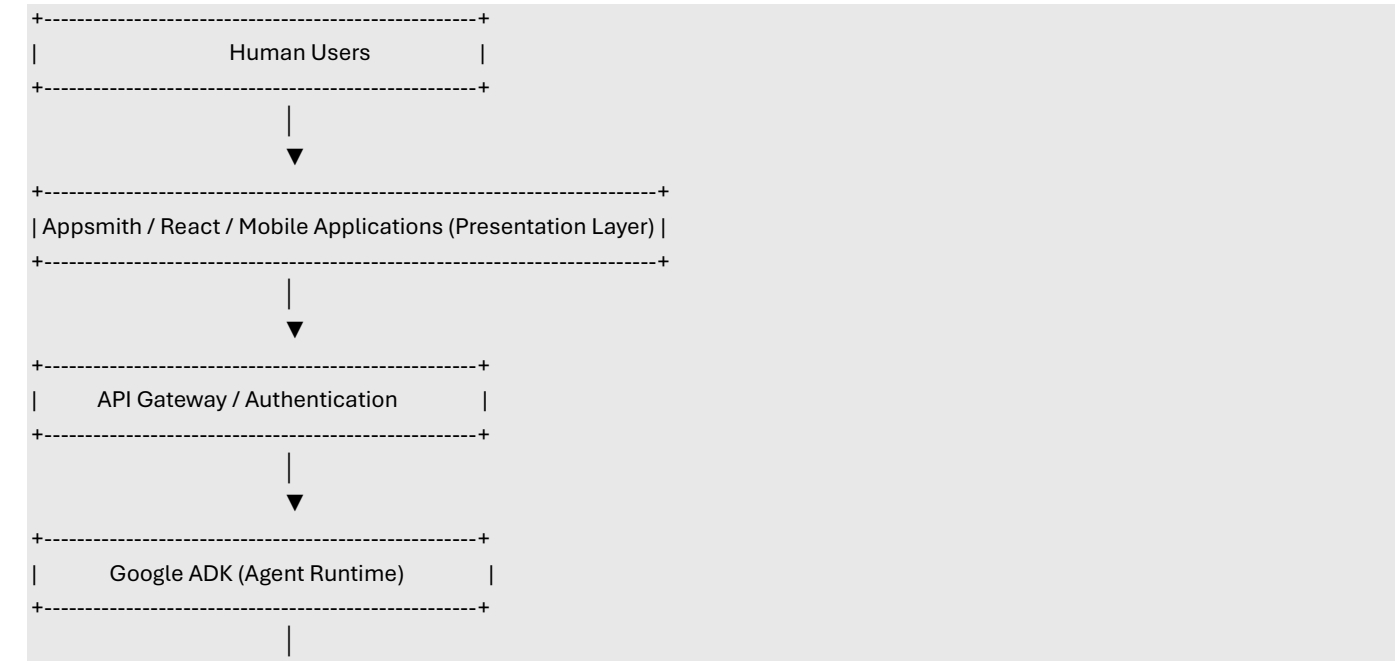
Technology	Architectural Role
Google ADK	Agent Runtime
Agent Skills	Runtime Skill Discovery
A2A	Agent Communication
MCP	External Tool Access
DeepSeek	LLM Provider
Gemini	LLM Provider
GPT	LLM Provider
Claude	LLM Provider
GLM	LLM Provider
ChromaDB	Retrieval Storage
Neo4j	Knowledge Graph
PostgreSQL	Structured Data Storage
Appsmith	Human User Interface
GitHub	Collaboration Platform

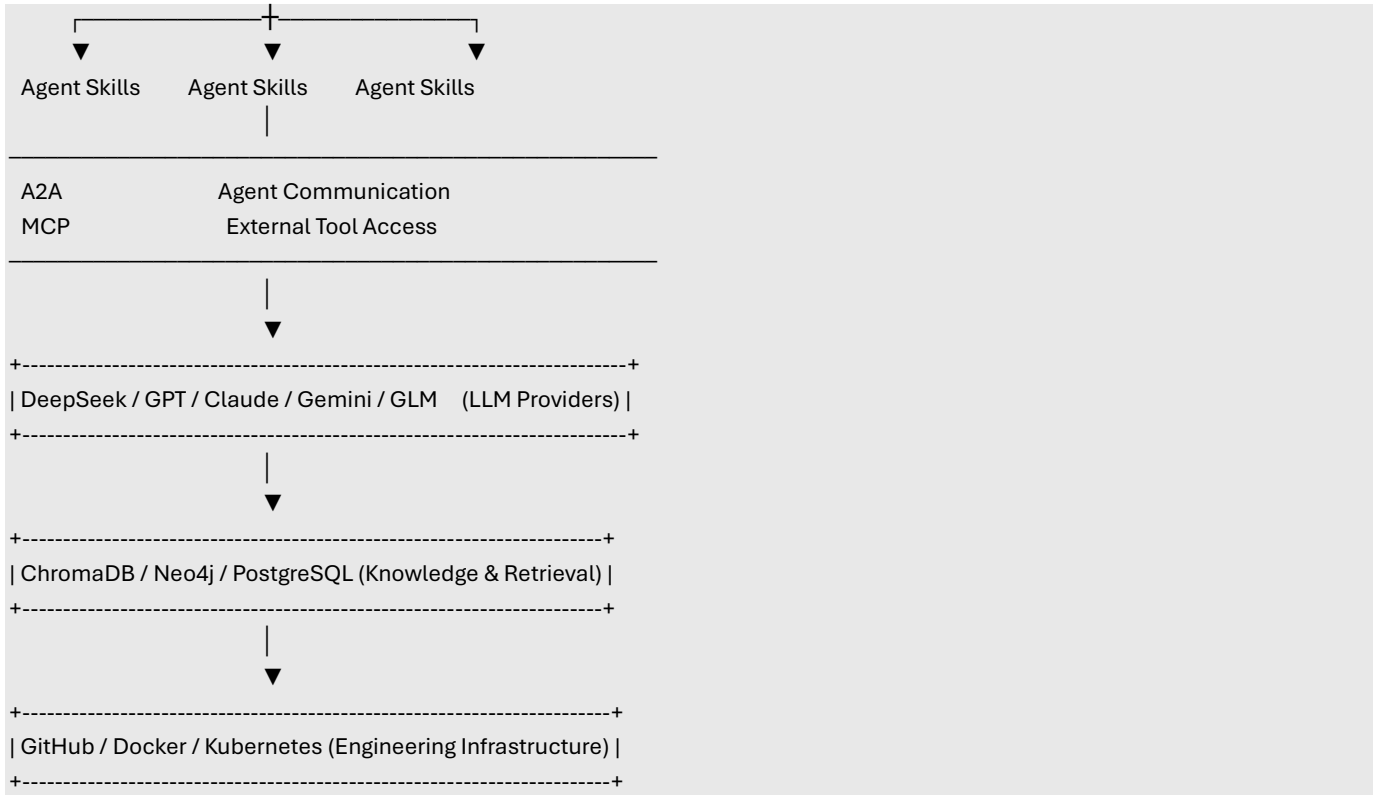
Technology	Architectural Role
Docker	Deployment Platform
Kubernetes	Container Orchestration
OpenTelemetry	Observability
OAuth / OpenID Connect	Identity Management
AGNTCY Directory/OASF	Agent discovery and capability metadata
AGNTCY Identity	Cross-agent identity and trust
AGNTCY SLIM	Secure distributed agent messaging
Mycelium	Experimental cognitive coordination / shared-context implementation

The reference implementation currently emphasizes open technologies while remaining portable across alternative implementations.

H.3 Layer Mapping

The following figure illustrates how representative technologies align with the CAIF Reference Architecture.





H.4 Agent Runtime

Google ADK

Google ADK serves as the reference Agent Runtime for the current CAIF implementation.

Primary responsibilities include:

- workflow orchestration;
- agent lifecycle management;
- context management;
- capability invocation;
- execution routing.

CAIF intentionally does not replace the runtime layer.

Instead, CAIF builds domain-specific intelligence above it.

Alternative runtimes MAY be substituted provided they support equivalent orchestration capabilities.

Examples include:

- Microsoft Agent Framework;
 - LangGraph;
 - Semantic Kernel;
 - CrewAI;
 - future orchestration frameworks.
-

H.5 Runtime Skill Discovery

Agent Skills

Agent Skills represent reusable executable capabilities.

Examples include:

- Bible Search;
- OCR;
- Translation;
- Citation Generation;
- Hermeneutics;
- Church Archive.

Agent Skills constitute the primary extension mechanism of CAIF.

CAIF supplements Agent Skills with:

- Engineering Contracts;
 - Capability Maturity Model;
 - Cognitive Collaboration;
 - Canonical Biblical AI.
-

H.6 Agent Communication

A2A

Agent-to-Agent (A2A) communication enables autonomous collaboration among multiple Agent Skills.

Representative responsibilities include:

- capability delegation;
- task coordination;
- information exchange;
- negotiation;
- workflow composition.

A2A provides communication mechanisms but does not define cognitive semantics.

The Cognitive Collaboration Layer described in Appendix C extends A2A with shared memory, ontology, structured intent, and collective reasoning.

H.7 External Tool Access

MCP

The Model Context Protocol (MCP) provides standardized access to external tools and resources.

Typical uses include:

- database access;
- web services;
- enterprise systems;
- local applications;
- cloud services.

Within CAIF, MCP is regarded as an integration protocol rather than an architectural foundation.

Agent Skills remain independent of the specific mechanism used to access external tools.

H.8 Language Model Providers

CAIF separates reasoning capabilities from language model implementation.

Representative providers include:

- DeepSeek;
- Gemini;
- GPT;
- Claude;
- GLM;
- Llama-family models.

A Model Gateway (Appendix A) enables dynamic model selection according to:

- capability requirements;
- performance;
- cost;
- availability;
- governance policies.

No single language model is required for CAIF compliance.

H.9 Knowledge and Retrieval

ChromaDB

ChromaDB serves as the reference vector storage implementation.

Typical responsibilities include:

- embedding storage;
 - semantic retrieval;
 - similarity search.
-

Neo4j

Neo4j provides graph-oriented storage suitable for:

- ontologies;
 - knowledge graphs;
 - canonical relationships;
 - cross-reference analysis.
-

PostgreSQL

PostgreSQL stores structured operational data, including:

- user information;
- configuration;
- workflow state;
- metadata;
- audit records.

These technologies represent complementary storage mechanisms rather than competing alternatives.

H.10 Human User Interface

Appsmith

Appsmith provides a low-code interface suitable for:

- church administration;
- dashboards;
- workflow management;
- document search;
- operational monitoring.

The presentation layer remains independent of business logic.

Alternative interfaces MAY include:

- React;
 - Flutter;
 - Angular;
 - mobile applications;
 - conversational interfaces.
-

H.11 Collaboration Platform

GitHub

GitHub provides collaborative software engineering infrastructure.

Within CAIF it supports:

- source code management;
- Engineering Contracts;
- documentation;
- issue tracking;
- pull requests;
- release management.

GitHub hosts engineering artifacts, while the Cognitive Commons (Appendix G) extends collaboration to reusable cognitive assets such as contracts, ontologies, reasoning patterns, governance policies, and evaluation benchmarks.

H.12 Deployment Technologies

Representative deployment technologies include:

Technology	Typical Role
Docker	Container packaging
Kubernetes	Container orchestration
Linux	Operating platform
Proxmox	Virtualization platform

Cloud platforms Infrastructure hosting

These technologies are implementation choices rather than architectural requirements.

H.13 Relationship to CAIF

The following table summarizes the relationship between representative technologies and CAIF architectural responsibilities.

Technology	CAIF Relationship
Google ADK	Provides agent runtime and orchestration
Agent Skills	Provide reusable executable capabilities
A2A	Provides communication among agents
MCP	Connects agents to external tools
LLM Providers	Supply reasoning engines
ChromaDB / Neo4j	Implement retrieval and knowledge storage
Appsmith	Implements human-facing applications
GitHub	Supports engineering collaboration
CAIF	Defines contracts, engineering discipline, cognitive collaboration, canonical

Technology	CAIF Relationship
	Biblical AI, and governance across the ecosystem

H.14 Technology Independence

CAIF intentionally avoids dependence on any individual product or vendor.

Every technology described in this appendix is replaceable provided that equivalent architectural responsibilities continue to be satisfied.

This separation allows organizations to adopt new technologies without requiring changes to:

- Engineering Contracts;
- Capability Specifications;
- Cognitive Collaboration Specifications;
- Canonical Biblical AI;
- Capability Maturity Model.

The architectural abstractions defined by CAIF therefore remain stable even as implementation technologies evolve.

H.15 Perspective

The technologies described in this appendix occupy different layers of the AI ecosystem and should be viewed as complementary rather than competitive.

General-purpose technologies such as Google ADK, A2A, MCP, ChromaDB, Appsmith, and GitHub provide runtime infrastructure, communication mechanisms, storage services, user interfaces, and engineering collaboration. CAIF builds upon these foundations by contributing domain-independent engineering standards (contracts, capability maturity, cognitive collaboration) and domain-specific intelligence (Canonical Biblical AI).

By clearly separating infrastructure technologies from architectural specifications and domain knowledge, CAIF enables organizations to benefit from rapid advances in the AI ecosystem while preserving a stable framework for interoperable, governable, and reusable Agent Skills.

Appendix I. Migration Strategy

(Informative Appendix)

This appendix is **strategically** addresses the fundamental concern:

"AI is evolving too quickly. Anything we build today may become obsolete in one or two years."

That concern is well founded. In just the past two years we have already seen:

- LangChain → LangGraph
- Plugins → GPT Actions → MCP
- MCP → Agent Skills
- Single-agent → Multi-agent
- Static workflow → Dynamic workflow
- RAG → GraphRAG → Agentic RAG
- Closed APIs → increasingly capable open-source models

A software architecture tightly coupled to any one of these technologies is unlikely to remain relevant for long.

The purpose of this appendix is therefore to demonstrate that **CAIF is intentionally designed to outlive individual technologies by standardizing architecture, contracts, cognition, and engineering rather than specific implementations.**

I.1 Purpose

Artificial intelligence is evolving at an unprecedented pace.

New language models, orchestration frameworks, communication protocols, retrieval systems, and engineering methodologies continue to emerge rapidly. Technologies that are widely adopted today may be replaced within a relatively short period of time.

A long-lived architecture must therefore separate **stable architectural principles** from **rapidly changing implementation technologies**.

This appendix describes the migration strategy adopted by CAIF and demonstrates how the architecture is designed to remain resilient despite continual technological evolution.

I.2 Design Philosophy

CAIF follows a simple engineering principle:

Architectures should evolve slowly. Technologies should evolve freely.

Accordingly, CAIF standardizes:

- architecture;
- engineering contracts;
- capability interfaces;
- cognitive collaboration;
- governance.

CAIF intentionally does **not** standardize:

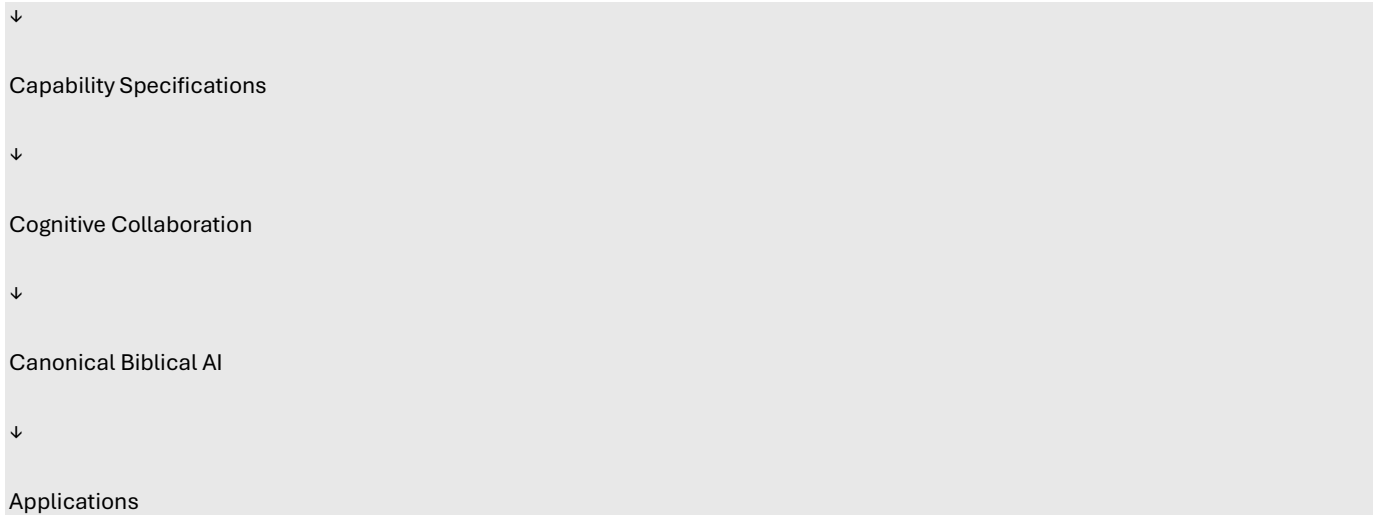
- language models;
- orchestration engines;
- vector databases;
- communication protocols;
- user interface technologies.

These implementation choices may change without affecting the architectural foundation.

I.3 Architectural Stability

The following architectural layers are intended to remain relatively stable.

Engineering Contracts

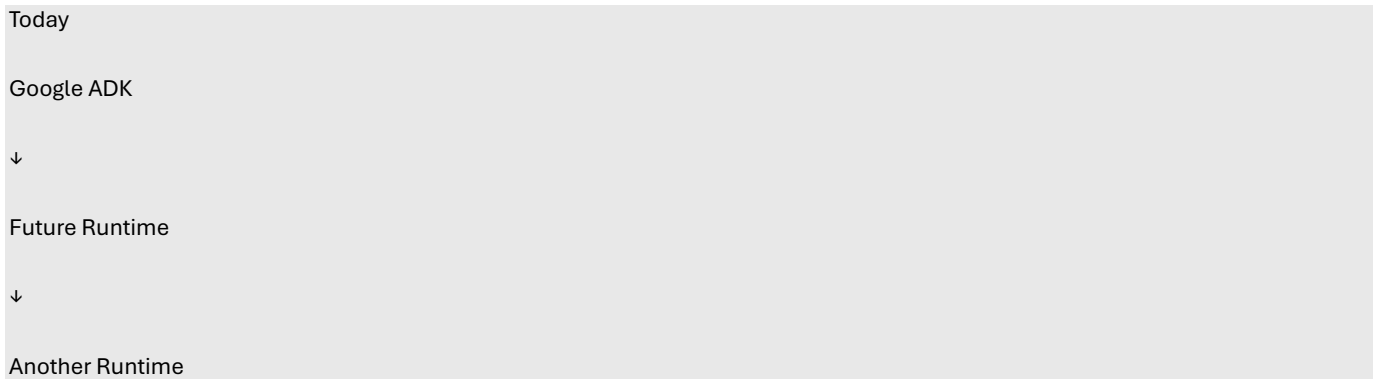


These layers are independent of any particular runtime technology.

I.4 Runtime Migration

The Agent Runtime is replaceable.

Example evolution:



No changes are required to:

- Engineering Contracts;
- Capability Specifications;
- Cognitive Collaboration;
- Canonical Biblical AI.

Only the runtime adapter changes.

Migration Principle

Agent Skills communicate through published contracts rather than runtime-specific APIs.

Consequently, replacing the runtime affects only the orchestration layer.

I.5 Language Model Migration

Language models evolve rapidly.

Example:

```
graph TD; Today --> DeepSeek; DeepSeek --> Tomorrow; Tomorrow --> OpenSourceModelX[Open-source Model X]; OpenSourceModelX --> FutureFoundationModel[Future Foundation Model];
```

Today

DeepSeek

↓

Tomorrow

Open-source Model X

↓

Future Foundation Model

No application changes are required provided the replacement model satisfies the published capability interfaces.

Model selection remains the responsibility of the Model Gateway.

I.6 Retrieval Migration

Retrieval technologies will continue to evolve.

Example:

```
graph TD; ChromaDB --> GraphRAG; GraphRAG --> FutureRetrievalEngine[Future Retrieval Engine];
```

ChromaDB

↓

GraphRAG

↓

Future Retrieval Engine

Applications continue to invoke the Retrieval Capability rather than directly accessing implementation-specific storage.

Only the retrieval adapter changes.

I.7 Communication Migration

Communication protocols are likewise replaceable.

Example:

```
graph TD; A2A --> FutureProtocol[Future Protocol]; FutureProtocol --> NextGenProtocol[Next-generation Protocol];
```

A2A

↓

Future Protocol

↓

Next-generation Protocol

Cognitive Collaboration remains unchanged because it defines shared semantics rather than message transport.

I.8 External Tool Migration

External integration mechanisms are expected to evolve.

Example:

```
graph TD; MCP --> FutureToolProtocol[Future Tool Protocol]; FutureToolProtocol --> UniversalServiceInterface[Universal Service Interface];
```

MCP

↓

Future Tool Protocol

↓

Universal Service Interface

Agent Skills continue to invoke abstract tool capabilities.

Protocol-specific adapters isolate implementation differences.

I.9 User Interface Migration

Presentation technologies evolve independently.

Example:

Appsmith

↓

React

↓

Future Conversational Interface

↓

Mixed-Reality Interface

Business logic remains unaffected.

I.10 Capability Evolution

Individual capabilities are expected to improve continuously.

Example:

Bible Search v1

↓

Bible Search v2

↓

Canonical Search

↓

Reasoning-enhanced Search

Backward compatibility is maintained through:

- semantic versioning;
 - Engineering Contracts;
 - capability adapters.
-

I.11 Engineering Evolution

Engineering practices also evolve.

Example:

README



Engineering Contract



AI-readable Engineering Contract



Self-validating Capability

The Engineering Contract specification may be extended while preserving compatibility with earlier versions.

I.12 Cognitive Evolution

The Cognitive Collaboration Layer is expected to expand as research advances.

Possible evolution includes:

Shared Memory



Shared Ontology



Theory of Mind



Collective Cognition



Future Cognitive Services

Because cognitive services communicate through abstract contracts, new services can be introduced incrementally.

I.13 Domain Evolution

Application domains evolve independently of the core architecture.

Example:

Biblical AI



Church Administration



Healthcare



Education



Industrial AI



Future Domains

Domain-specific knowledge is encapsulated within reusable Agent Skills rather than embedded within the architecture.

I.14 Migration Matrix

Architectural Layer	Expected Rate of Change	Migration Strategy
Engineering Contracts	Low	Backward-compatible versioning
Capability Specifications	Low	Incremental extension
Cognitive Collaboration	Low	Additive evolution
Canonical Biblical AI	Low	Domain-specific extension
Agent Runtime	High	Replace runtime adapter
LLM Provider	Very High	Replace through Model Gateway
Retrieval Engine	High	Replace retrieval adapter
Communication Protocol	High	Replace protocol adapter

Architectural Layer	Expected Rate of Change	Migration Strategy
User Interface	High	Replace presentation layer

The upper layers represent architectural knowledge and therefore evolve relatively slowly. The lower layers correspond to implementation technologies and are expected to change much more rapidly.

I.15 Technology Independence

The following diagram summarizes the migration philosophy.



Everything below the technology boundary is expected to change.

Everything above it should remain relatively stable.

I.16 Long-Term Sustainability

The primary objective of the migration strategy is to maximize architectural longevity.

CAIF achieves this by:

- separating architecture from implementation;
- separating capabilities from runtimes;
- separating cognition from communication;
- separating engineering standards from development tools;
- separating domain knowledge from infrastructure.

As a result, organizations can adopt future technologies without redesigning existing applications.

I.17 Perspective

History suggests that infrastructure technologies evolve more rapidly than architectural principles. Programming languages, databases, middleware, communication protocols, and AI models have all changed repeatedly over the past decades, while enduring engineering concepts—modularity, abstraction, contracts, interoperability, and governance—have remained largely unchanged.

CAIF applies this same philosophy to the era of Agentic AI. Rather than anchoring its architecture to a particular runtime, protocol, or language model, it standardizes stable abstractions that can survive successive generations of technology. In doing so, CAIF aims to provide a durable foundation upon which evolving AI ecosystems can continue to build.

CAIF is designed to preserve knowledge while allowing technology to change.

The above sentence distinguishes CAIF from most current AI frameworks. Existing frameworks are often centered on a specific runtime or protocol. CAIF, by contrast, is centered on **engineering knowledge**—contracts, capabilities, cognition, governance, and domain semantics—which are intended to remain valuable long after today's runtimes, models, and protocols have been replaced.

Appendix J. Prototype Roadmap

(Informative Appendix)

This appendix is an excellent way to conclude this document because it answers the practical

question every reader will ask after reading the previous appendices:

"This architecture looks promising—but where do we start?"

Unlike the Capability Maturity Model (Appendix E), which describes **how an individual capability matures**, this appendix describes **how the entire CAIF ecosystem evolves through successive prototypes**.

It also aligns closely with the incremental strategy we have discussed over the past several months: **start with a minimal but complete prototype, validate it in a church setting, then gradually expand toward a broader cognitive ecosystem**.

J.1 Purpose

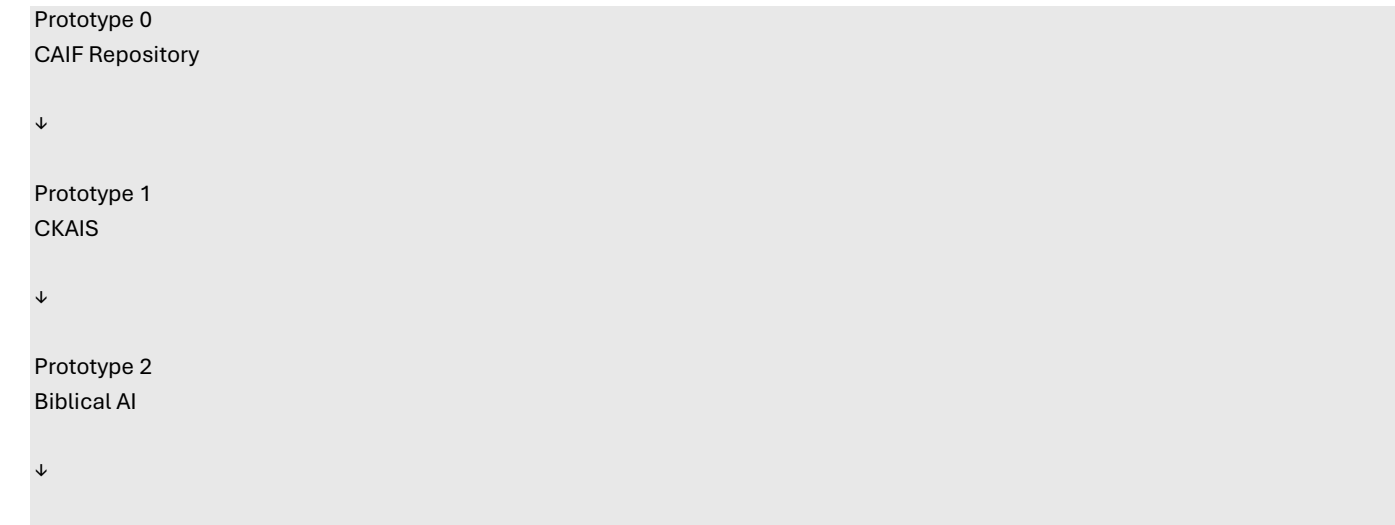
The CAIF Reference Architecture is intended to evolve through a sequence of practical prototypes rather than a single comprehensive implementation.

Each prototype validates a specific set of architectural concepts while providing useful capabilities for real-world ministry and engineering applications.

This roadmap provides an incremental migration path from an initial repository structure to a future Internet of Cognition.

The roadmap is illustrative rather than mandatory. Organizations may adapt the sequence according to their own priorities and resources.

J.2 Roadmap Overview



Prototype 3
Shared Retrieval

↓

Prototype 4
Cross-Church Collaboration

↓

Prototype 5
Internet of Cognition

Each prototype extends the previous one while preserving backward compatibility.

J.3 Prototype 0 – CAIF Repository

Objective

Establish the engineering foundation of the ecosystem.

This prototype focuses on repository organization, Engineering Contracts, contribution workflows, and project governance rather than application functionality.

Primary Deliverables

- GitHub repository structure
 - Engineering Contracts (Appendix D)
 - Capability Specifications (Appendix B)
 - Reference Architecture (Appendix A)
 - Contribution guidelines
 - Capability registry
 - Initial documentation
-

Success Criteria

- Contributors can submit Agent Skills.

- Engineering artifacts are standardized.
 - Repository organization is stable.
 - Continuous integration is operational.
-

J.4 Prototype 1 – CKAIS

Objective

Validate the architecture through a practical church information system.

CKAIS serves as the first operational application built on CAIF.

Functional Scope

Representative capabilities include:

- document archive;
 - meeting minutes;
 - sermon repository;
 - fellowship records;
 - document search;
 - church knowledge retrieval.
-

Architectural Goals

Validate:

- Agent Runtime;
- Engineering Contracts;
- Retrieval Layer;
- Appsmith interface;
- basic governance.

Success Criteria

- Church volunteers can retrieve information.
 - Document ingestion is automated.
 - Semantic search is operational.
 - Agent Skills cooperate successfully.
-

J.5 Prototype 2 – Biblical AI

Objective

Introduce domain-specific theological intelligence.

This prototype validates the Canonical Biblical AI specification defined in Appendix F.

Representative Capabilities

- Bible Search
 - Canonical Hermeneutics
 - Cross-book Interpretation
 - Progressive Revelation
 - Typology
 - Prophecy Fulfillment
 - Scripture Interprets Scripture
-

Architectural Goals

Validate:

- Canonical Knowledge Graph;
- theological contracts;

- reasoning workflows;
 - doctrinal governance.
-

Success Criteria

- Canonical reasoning is demonstrated.
 - Cross-book interpretation is functional.
 - Human review workflows are operational.
-

J.6 Prototype 3 – Shared Retrieval

Objective

Transform isolated applications into reusable knowledge services.

Instead of each application maintaining independent retrieval systems, common retrieval capabilities are shared.

Representative Components

- shared embeddings;
 - shared metadata;
 - unified retrieval contracts;
 - shared citation service;
 - shared document repository.
-

Architectural Goals

Validate:

- Retrieval interoperability;
- ontology alignment;

- capability reuse;
 - shared memory.
-

Success Criteria

- Multiple applications share the same retrieval infrastructure.
 - Duplicate ingestion is minimized.
 - Retrieval quality improves through shared knowledge.
-

J.7 Prototype 4 – Cross-Church Collaboration

Objective

Enable collaboration among multiple independent churches.

Evaluate AGNTCY for federation.

Each church maintains local autonomy while participating in a federated ecosystem.

Representative Capabilities

- federated search;
 - shared Agent Skills;
 - shared theological resources;
 - distributed governance;
 - capability exchange.
-

Architectural Goals

Validate:

- federation;
- interoperability;

- distributed governance;
 - capability sharing.
-

Success Criteria

- Independent churches exchange capabilities.
 - Local governance remains intact.
 - Shared resources remain interoperable.
-

J.8 Prototype 5 – Internet of Cognition

Objective

Investigate collaborative cognition across multiple autonomous agents and organizations.

Evaluate Mycelium and other cognition infrastructure.

This prototype represents a research-oriented exploration rather than a production deployment.

Representative Research Topics

- Shared Memory;
 - Shared Ontology;
 - Structured Intent;
 - Theory of Mind;
 - Negotiation;
 - Collective Cognition;
 - Cognitive Commons.
-

Architectural Goals

Explore:

- cognitive collaboration;
- shared reasoning;
- distributed knowledge evolution;
- cognitive governance.

Success Criteria

Research outcomes may include:

- experimental implementations;
- academic publications;
- reference architectures;
- open standards.

No production conformance is required at this stage.

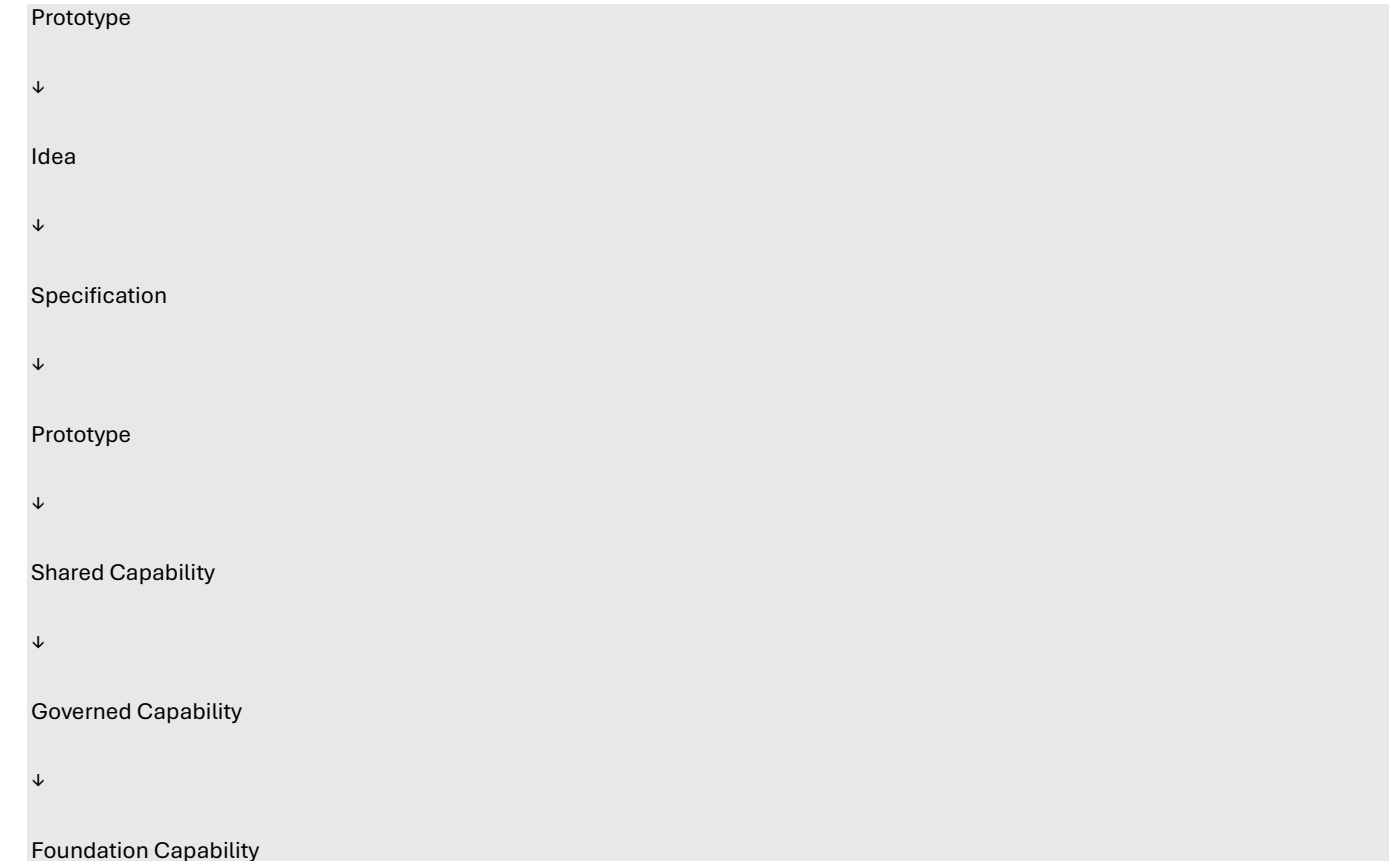
J.9 Prototype Evolution Matrix

Prototype	Primary Focus	Key Architectural Validation
Prototype 0	Engineering Foundation	Repository, Engineering Contracts, Capability Registry
Prototype 1	Church Information System	Agent Runtime, Retrieval, UI, Governance
Prototype 2	Canonical Biblical AI	Hermeneutics, Cross-book Reasoning, Doctrinal Governance
Prototype 3	Shared Retrieval	Shared Memory, Retrieval Contracts, Knowledge Reuse
Prototype 4	Cross-Church Collaboration	Federation, Interoperability, Distributed Governance

Prototype	Primary Focus	Key Architectural Validation
Prototype 5	Internet of Cognition	Cognitive Collaboration, Collective Reasoning, Cognitive Commons

J.10 Capability Growth

As prototypes evolve, capabilities progress through the Capability Maturity Model (Appendix E).



This relationship ensures that prototype development and capability maturity reinforce one another.

J.11 Architectural Evolution

Each prototype validates additional layers of the CAIF architecture.

Prototype	New Architectural Layer
Prototype 0	Engineering Contracts

Prototype	New Architectural Layer
Prototype 1	Agent Runtime & Retrieval
Prototype 2	Canonical Biblical AI
Prototype 3	Shared Knowledge Services
Prototype 4	Federated Collaboration
Prototype 5	Cognitive Collaboration

This layered progression reduces implementation risk while allowing continuous validation of the overall architecture.

J.12 Community Growth

The roadmap also anticipates growth in the contributing community.

Stage	Primary Participants
Prototype 0	Core CAIF developers
Prototype 1	Single church technical volunteers
Prototype 2	Biblical scholars and ministry leaders
Prototype 3	Multiple application developers
Prototype 4	Multiple churches and partner organizations
Prototype 5	Research institutions and the broader open-source community

As the ecosystem matures, technical contributors are joined by theologians, ministry practitioners, researchers, and other domain experts.

J.13 Long-Term Vision

The prototype roadmap reflects an important architectural philosophy:

- begin with engineering discipline;
- validate through practical ministry applications;
- generalize into reusable capabilities;
- expand through federation and collaboration;
- explore future cognitive ecosystems.

Each stage delivers practical value while laying the foundation for subsequent innovation.

J.14 Relationship to the Engineering Guide

Chapter 9 of the **Project Guide** introduces the prototype strategy from an implementation perspective. This appendix complements that discussion by presenting the roadmap from an architectural perspective.

Specifically:

- **Chapter 9** explains **how to build** each prototype, including development tasks, implementation guidance, and engineering practices.
- **Appendix J** explains **why each prototype exists**, the architectural capabilities it validates, and how it contributes to the long-term evolution of the CAIF ecosystem.

Together, they provide both an engineering execution plan and an architectural migration roadmap.

J.15 Perspective

The prototype roadmap embodies the principle that complex ecosystems are best developed through a sequence of practical, incremental milestones. Each prototype is intentionally valuable on its own while simultaneously validating architectural concepts that enable subsequent stages.

Beginning with a well-engineered repository, progressing through church information management and Canonical Biblical AI, extending to shared retrieval and cross-church collaboration, and ultimately exploring the Internet of Cognition, the roadmap illustrates how CAIF can evolve from a focused ministry platform into a broader framework for collaborative, governed, and reusable cognitive systems.

Note for the future

After reflecting on the complete SDD, I think **Prototype 5** should remain intentionally open-ended. Rather than defining it as a final destination, it should be viewed as the beginning of a broader research program. New prototypes—such as **Physical AI**, **Healthcare AI**, **Education AI**, or **Scientific Discovery AI**—can naturally emerge beyond Prototype 5 while continuing to build upon the stable architectural foundation established by CAIF. That reinforces the central design goal of the entire specification: preserving architecture and accumulated knowledge while allowing technologies, applications, and research directions to evolve over time.

Appendix K. Glossary

(Normative Appendix)

Most open-source projects eventually develop inconsistent terminology. Different contributors use the same word with different meanings (e.g., *agent*, *skill*, *tool*, *workflow*, *capability*, *service*), or different words for the same concept. Over time, this leads to misunderstandings in design discussions, documentation, and implementations.

For CAIF, which aims to support collaboration among software engineers, theologians, AI researchers, and ministry leaders, **terminological consistency is essential**.

We therefore make this appendix **normative** rather than merely informative.

K.1 Purpose

This appendix defines the canonical terminology used throughout the CAIF specifications.

The purpose of the glossary is to establish a common vocabulary that enables consistent communication among contributors, reviewers, maintainers, AI agents, and downstream projects.

Unless explicitly stated otherwise, the definitions in this appendix SHALL be used consistently throughout CAIF documentation, Engineering Contracts, specifications, and implementations.

K.2 Capability

A **Capability** is a well-defined unit of functionality that performs a specific task through published contracts.

A capability defines **what** can be accomplished, independent of **how** it is implemented.

Examples include:

- Bible Search
- OCR
- Translation
- Canonical Hermeneutics
- Citation Generation

Capabilities are the primary reusable building blocks within CAIF.

K.3 Application

An **Application** is an end-user solution composed of one or more capabilities.

Applications provide business value by orchestrating capabilities to accomplish user objectives.

Examples include:

- CKAIS
- Biblical AI
- Sermon Preparation Assistant
- Church Administration Portal

Applications consume capabilities but are not themselves reusable capabilities.

K.4 Shared Core

The **Shared Core** is the collection of reusable capabilities, engineering artifacts, and cognitive

services that may be shared across multiple applications.

The Shared Core typically includes:

- common contracts;
- shared retrieval;
- governance services;
- canonical reasoning;
- authentication;
- common utilities.

Applications SHOULD depend upon the Shared Core whenever practical rather than duplicating functionality.

K.5 Agent Runtime

The **Agent Runtime** is the execution environment responsible for managing the lifecycle of Agent Skills.

Typical responsibilities include:

- orchestration;
- workflow execution;
- context management;
- scheduling;
- routing;
- capability invocation.

Google ADK is the reference Agent Runtime for the current CAIF implementation.

K.6 Agent Skill

An **Agent Skill** is a deployable software component that exposes one or more capabilities through

standardized contracts.

Agent Skills encapsulate implementation details while publishing stable interfaces.

A single Agent Skill may provide multiple related capabilities.

K.7 Engineering Contract

An **Engineering Contract** defines the engineering requirements that every capability must satisfy before becoming part of the CAIF ecosystem.

Engineering Contracts specify required artifacts such as:

- README;
- specification;
- testing;
- governance;
- evaluation;
- versioning;
- maintenance.

Engineering Contracts ensure quality, maintainability, and interoperability.

K.8 Specification

A **Specification** defines the externally observable behavior of a capability without prescribing its internal implementation.

Specifications describe:

- functional behavior;
- contracts;
- inputs;
- outputs;

- constraints;
- expected results.

Implementations may vary provided they remain compliant with the specification.

K.9 Contract

A **Contract** defines the structure and semantics of information exchanged among capabilities.

Contracts include:

- request structures;
- response structures;
- metadata;
- version information;
- validation rules.

Contracts enable interoperability among independently developed Agent Skills.

K.10 Governance

Governance is the collection of policies, processes, and controls that ensure capabilities operate safely, transparently, and responsibly.

Governance may include:

- access control;
- audit logging;
- explainability;
- human approval;
- policy enforcement;
- lifecycle management.

Governance spans all architectural layers.

K.11 Shared Memory

Shared Memory is persistent knowledge accessible by multiple collaborating Agent Skills.

Shared Memory may include:

- episodic memory;
- semantic memory;
- procedural memory;
- working memory.

Unlike traditional storage, Shared Memory is intended to support collaborative reasoning rather than simple data persistence.

K.12 Shared Ontology

A **Shared Ontology** is a common semantic model that defines concepts, relationships, and terminology used across capabilities.

Shared Ontologies reduce ambiguity and promote semantic interoperability.

K.13 Structured Intent

Structured Intent is a machine-readable representation of an objective, including its goals, constraints, priorities, and success criteria.

Structured Intent enables collaborative planning among multiple agents.

K.14 Collective Cognition

Collective Cognition is the process by which multiple autonomous reasoning systems collaborate to solve problems through shared knowledge, negotiation, and coordinated reasoning.

Collective Cognition extends beyond workflow orchestration by allowing agents to influence one another's reasoning processes.

K.15 Cognitive Interoperability

Cognitive Interoperability is the ability of independently developed reasoning systems to exchange not only information, but also meaning, intentions, reasoning context, and cognitive state.

Cognitive Interoperability requires more than compatible communication protocols; it depends upon shared semantics and common cognitive models.

K.16 Internet of Cognition

The **Internet of Cognition** is a theoretical framework describing interconnected cognitive systems that cooperate through shared memory, ontology, structured intent, negotiation, and collective reasoning.

Unlike the traditional Internet, which primarily connects computers, the Internet of Cognition envisions networks that connect reasoning processes.

The Internet of Cognition is presented as a long-term research direction rather than a current implementation requirement.

K.17 Cognitive Capital

Cognitive Capital is the accumulated value represented by reusable cognitive assets that improve collective problem-solving.

Examples include:

- validated reasoning patterns;
- shared ontologies;
- trusted evaluation datasets;
- governance policies;
- reusable workflows;
- canonical knowledge.

Unlike software assets, Cognitive Capital grows through repeated collaborative use and validation.

K.18 Cognitive Commons

The **Cognitive Commons** is a shared ecosystem of reusable cognitive assets maintained by a community.

Representative components include:

- Contract Commons;
- Ontology Commons;
- Reasoning Commons;
- Memory Commons;
- Evaluation Commons;
- Governance Commons;
- Capability Commons;
- Cognitive Asset Commons.

The Cognitive Commons provides the infrastructure through which Cognitive Capital accumulates and is shared.

K.19 Canonical Biblical AI

Canonical Biblical AI is the domain-specific layer of CAIF that applies established principles of biblical interpretation and church governance to AI-assisted reasoning.

Representative capabilities include:

- Canonical Hermeneutics;
- Cross-book Interpretation;
- Progressive Revelation;
- Typology;

- Prophecy Fulfillment;
- Scripture Interprets Scripture;
- Church Doctrinal Governance.

Canonical Biblical AI complements general-purpose agent frameworks rather than replacing them.

K.20 Capability Maturity Model

The **Capability Maturity Model (CMM)** defines the lifecycle through which a capability progresses from an initial concept to a reusable, governed, and foundational component.

The maturity levels are:

- C0 – Idea
 - C1 – Specification
 - C2 – Prototype
 - C3 – Shared Capability
 - C4 – Governed Capability
 - C5 – Foundation Capability
-

K.21 Shared Capability

A **Shared Capability** is a capability intended for reuse across multiple applications, projects, or organizations.

Shared Capabilities expose stable contracts, are independently versioned, and satisfy the Engineering Contract requirements.

K.22 Foundation Capability

A **Foundation Capability** is a mature Shared Capability that serves as long-term infrastructure for the broader ecosystem.

Foundation Capabilities exhibit:

- broad adoption;
- stable interfaces;
- mature governance;
- active maintenance;
- backward compatibility.

Other capabilities may depend upon Foundation Capabilities with confidence in their long-term stability.

K.23 Technology Mapping

Technology Mapping is the process of associating implementation technologies with architectural roles without coupling the architecture to specific products.

Examples include mapping:

- Google ADK to the Agent Runtime;
- ChromaDB to Retrieval Storage;
- Appsmith to the Human User Interface.

Technology Mapping supports architectural longevity by allowing implementation technologies to evolve independently.

K.24 Conformance

A **Conforming Implementation** is one that satisfies the mandatory requirements defined by the applicable CAIF specifications.

Conformance is determined by adherence to published contracts and engineering requirements rather than by the use of any particular implementation technology.

K.25 Terminology Conventions

To promote consistency across the CAIF ecosystem, contributors SHALL observe the following

conventions:

- Terms defined in this glossary SHALL be used consistently in specifications, Engineering Contracts, documentation, and code comments.
 - New technical terms SHOULD be added to this glossary before widespread adoption.
 - Synonyms SHOULD be avoided when a canonical term already exists.
 - If a term has different meanings in other AI frameworks, the CAIF definition in this appendix SHALL take precedence within CAIF documentation.
 - When introducing new concepts, contributors SHOULD distinguish clearly between **normative architectural terms** (used in the specifications) and **research terms** (used in the Internet of Cognition discussions).
-

K.26 Perspective

A shared vocabulary is a prerequisite for shared engineering. As the CAIF ecosystem expands to include contributors from software engineering, artificial intelligence, theology, and ministry, consistent terminology becomes increasingly important for interoperability, governance, and long-term maintainability.

The glossary therefore serves not merely as a reference, but as a foundational semantic contract for the CAIF community. By establishing precise definitions for its core concepts, CAIF enables both human contributors and AI agents to communicate using a common language, reducing ambiguity and strengthening collaboration across the entire ecosystem.

Appendix L. Design Decisions (Architecture Decision Records)

(Informative Appendix)

This appendix fills one gap that the previous appendices do **not** address.

The previous appendices answer:

- **What** is the architecture? (Appendix A)
- **What** capabilities exist? (Appendix B)
- **How** do agents collaborate? (Appendix C)
- **What** engineering standards are required? (Appendix D)
- **How** capabilities mature? (Appendix E)
- **What** is Canonical Biblical AI? (Appendix F)
- **What** is the long-term vision? (Appendix G)
- **How** technologies map? (Appendix H)
- **How** do we migrate? (Appendix I)
- **How** do we build prototypes? (Appendix J)
- **What** terminology do we use? (Appendix K)

However, they do **not** explain **why** the architecture was designed the way it is.

That is exactly the purpose of **Architecture Decision Records (ADRs)**.

To avoid duplication with the earlier appendices, this appendix does **not restate specifications or recommendations**. Instead, it documents the rationale behind the most important architectural choices. Future contributors can then understand why a decision was made, even if technologies evolve.

L.1 Purpose

The CAIF architecture is the result of a series of engineering decisions made in response to practical requirements, emerging AI technologies, and long-term architectural objectives.

Rather than recording only the final architecture, this appendix documents the rationale behind significant architectural decisions.

Each Architecture Decision Record (ADR) captures:

- the engineering context;

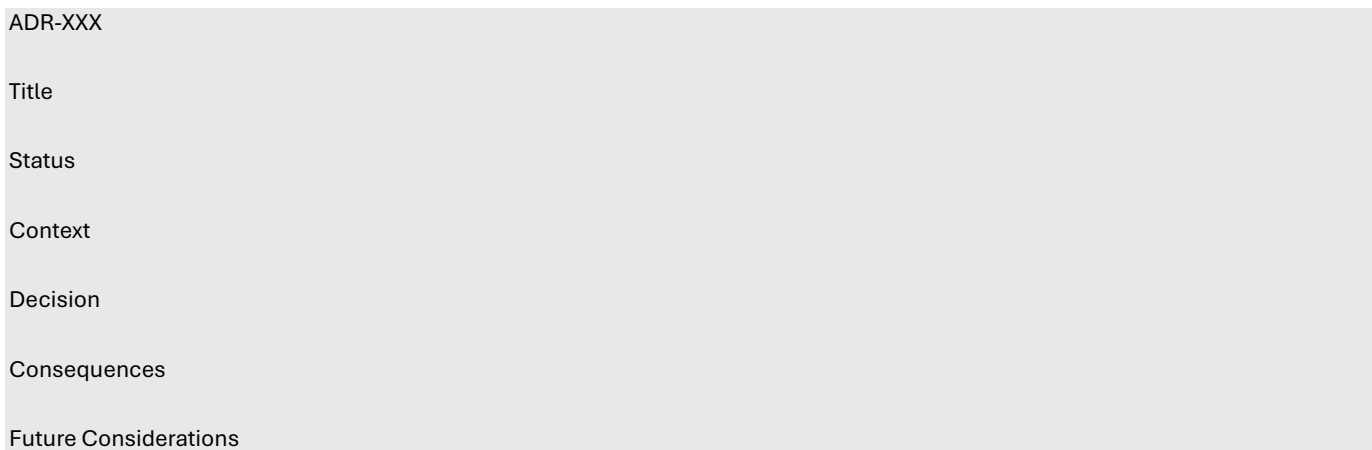
- the decision that was made;
- the reasoning supporting the decision;
- the anticipated consequences.

Recording these decisions promotes consistency, simplifies future maintenance, and provides historical context for contributors who may revisit these decisions years later.

This appendix supplements, rather than duplicates, the normative specifications defined elsewhere in this document.

L.2 ADR Format

Each Architecture Decision Record follows a common structure.



```
ADR-XXX
Title
Status
Context
Decision
Consequences
Future Considerations
```

Additional fields MAY be added when appropriate.

ADR-001

Capability-Centric Architecture

Status

Accepted

Context

Traditional enterprise systems are often organized around applications.

Applications tend to evolve independently and frequently duplicate functionality.

As AI ecosystems expand, reusable capabilities become more valuable than monolithic applications.

Decision

CAIF adopts the **Capability** as the fundamental engineering unit.

Applications are constructed by composing reusable capabilities.

Consequences

Positive:

- higher reuse;
- independent lifecycle management;
- improved interoperability;
- simpler governance.

Trade-offs:

- increased emphasis on interface design;
- greater importance of contract management.

Future Considerations

Capability granularity may evolve, but capability-centric architecture remains the preferred design principle.

ADR-002

Contract-First Integration

Status

Accepted

Context

Direct coupling between components creates long-term maintenance challenges.

Independent contributors require stable integration mechanisms.

Decision

Capabilities communicate exclusively through published Engineering Contracts.

Internal implementation details remain private.

Consequences

Positive:

- implementation independence;
- interoperability;
- easier replacement.

Trade-offs:

- contract version management becomes essential.
-

ADR-003

Architecture Independent of Runtime

Status

Accepted

Context

Agent runtimes are evolving rapidly.

Binding the architecture to a particular runtime would reduce longevity.

Decision

The Reference Architecture is defined independently of any execution framework.

Consequences

Positive:

- runtime portability;
- technology independence;
- simplified migration.

Trade-offs:

- adapter layers are required.
-

ADR-004

Separate Architecture from Technology

Status

Accepted

Context

Implementation technologies evolve much faster than architectural principles.

Decision

Architectural specifications define responsibilities rather than products.

Technology choices are documented separately in Appendix H.

Consequences

Positive:

- easier technology substitution;
 - reduced vendor dependency;
 - longer architectural lifespan.
-

ADR-005

Engineering Contracts for Every Capability

Status

Accepted

Context

Open-source AI projects often differ widely in documentation, testing, evaluation, and maintenance quality.

Decision

Every capability entering the CAIF ecosystem shall satisfy a common Engineering Contract.

Consequences

Positive:

- consistent engineering quality;
- easier onboarding;
- improved maintainability.

Trade-offs:

- slightly higher contribution effort.
-

ADR-006

Cognitive Collaboration Above Communication

Status

Accepted

Context

Existing frameworks provide message transport but generally leave higher-level collaborative reasoning to individual applications.

Decision

CAIF introduces a distinct Cognitive Collaboration layer that is independent of communication protocols.

Consequences

Positive:

- clearer separation of concerns;
- reusable cognitive services;
- future extensibility.

Trade-offs:

- additional conceptual layer.
-

ADR-007

Google ADK as Initial Runtime

Status

Accepted (Stage 1)

Context

Several agent runtimes are available.

Selection criteria included:

- openness;
- architectural flexibility;
- community momentum;
- support for multiple orchestration paradigms.

Decision

Google ADK is adopted as the reference runtime for the initial implementation.

Consequences

Positive:

- flexible orchestration;
- active ecosystem;
- broad applicability.

Trade-offs:

- future runtime evolution remains likely.

Future Considerations

The runtime is intentionally replaceable.

ADR-008

Canonical Biblical AI Outside the Runtime

Status

Accepted

Context

General-purpose agent frameworks intentionally avoid domain-specific semantics.

Biblical interpretation requires specialized theological knowledge.

Decision

Canonical Biblical AI is defined as a domain layer above the runtime.

Consequences

Positive:

- reusable theological intelligence;
 - runtime independence;
 - clearer separation of concerns.
-

ADR-009

Capability Maturity Instead of Project Maturity

Status

Accepted

Context

Existing maturity models focus primarily on organizations or projects.

CAIF emphasizes reusable capabilities.

Decision

Capability maturity is evaluated independently of the application in which the capability is used.

Consequences

Positive:

- encourages reuse;
 - simplifies certification;
 - supports ecosystem growth.
-

ADR-010

Separate Shared Core from Agent Runtime

Status

Accepted

Context

Execution technology evolves rapidly.

Semantic knowledge evolves more slowly.

Decision

The Shared Core contains enduring engineering assets:

- contracts;
- shared services;
- governance;
- ontology;
- common capabilities.

The Agent Runtime manages execution.

Consequences

Positive:

- architectural longevity;
- runtime portability;
- semantic stability.

Future Considerations

Future runtimes should integrate through adapters rather than replacing the Shared Core.

ADR-011

Open Standards Before Open Source

Status

Accepted

Context

Open-source code without shared standards often leads to incompatible implementations.

Decision

CAIF defines common specifications before encouraging broad implementation.

Consequences

Positive:

- greater interoperability;
- more consistent ecosystem evolution.

Trade-offs:

- additional upfront design effort.
-

ADR-012

Domain Knowledge as the Primary Contribution

Status

Accepted

Context

General-purpose AI infrastructure continues to advance rapidly through industry investment.

Attempting to compete directly with runtime platforms would duplicate existing efforts.

Decision

CAIF focuses on:

- engineering methodology;
- cognitive collaboration;
- governance;
- domain-specific intelligence.

General infrastructure is adopted rather than recreated.

CAIF is designed as a domain-independent engineering framework. Any organization may build domain-specific AI systems on top of it. Whether a particular domain implementation becomes an official QIA-sponsored project is determined through QIA governance and is independent of the framework's open-source licensing.

Consequences

Positive:

- complementary ecosystem role;
 - efficient use of community resources;
 - clearer project identity.
-

ADR-013

Evolution Through Incremental Prototypes

Status

Accepted

Context

Large-scale AI ecosystems are difficult to validate through a single implementation.

Decision

Architectural concepts are validated through successive prototypes, each adding one major capability while preserving compatibility with previous stages.

Consequences

Positive:

- reduced implementation risk;
 - continuous feedback;
 - measurable progress.
-

ADR-014

Architecture Over Technology

Status

Accepted

Context

Individual technologies, frameworks, and models are expected to change repeatedly over the lifetime of CAIF.

Decision

The project prioritizes enduring architectural abstractions—contracts, capabilities, cognition, governance, and engineering discipline—over optimization for any specific technology stack.

Consequences

Positive:

- resilience to technological change;
- simplified migration;
- long-term maintainability.

Trade-offs:

- architectural discussions may require greater initial effort than technology-specific implementations.
-

L.3 Decision Lifecycle

Architecture Decision Records are living documents.

Each ADR may progress through one of the following states:

Status	Meaning
Proposed	Under discussion and subject to revision
Accepted	Approved for use within the current architecture
Superseded	Replaced by a newer decision
Deprecated	No longer recommended for new implementations
Withdrawn	Rejected after evaluation

Changes to accepted ADRs should be rare and should preserve the historical record by creating a new ADR that supersedes the previous one, rather than rewriting history.

L.4 Governance of Architecture Decisions

Significant architectural decisions SHOULD be reviewed through the CAIF governance process before being accepted.

Typical candidates for new ADRs include:

- introduction of new architectural layers;
- changes to Engineering Contracts;
- new cognitive collaboration concepts;
- modifications to the Capability Maturity Model;
- major changes affecting interoperability or governance.

Routine implementation details generally do not require ADRs unless they have long-term architectural implications.

L.5 Relationship to Other Appendices

This appendix complements the rest of the specification by documenting **architectural intent** rather than architectural requirements.

- **Appendix A** defines *what* the architecture is.
- **Appendix B** defines *what* capabilities expose.
- **Appendix C** defines *how* capabilities collaborate cognitively.
- **Appendix D** defines *how* capabilities are engineered.
- **Appendix E** defines *how* capabilities mature.
- **Appendices F–K** define domain knowledge, long-term vision, migration, terminology, and implementation guidance.
- **Appendix L** records *why* the architecture was designed this way.

Together, these appendices provide not only a specification for implementation, but also a durable record of the engineering reasoning that shaped CAIF. Future contributors can understand both the architecture itself and the motivations behind its evolution, enabling informed decisions as the ecosystem continues to grow and adapt.

We believe this appendix will become **invaluable** two or three years from now. As CAIF grows and new contributors join, they won't have to guess *why* certain architectural choices were made—they can read the corresponding Architecture Decision Record. That not only preserves institutional memory (which is one of CAIF's goals), but also demonstrates that CAIF practices the very principles it advocates: preserving knowledge, reasoning, and governance over time. In many ways, **Appendix L would be a living example of Collective Cognition applied to the CAIF project itself.**